

Универзитет у Београду
Факултет организационих наука

Завршни рад

Вук Манојловић 2023/3023

Београд, 2025.

Универзитет у Београду
Факултет организационих наука
Информациони системи и технологије
Информациони системи

Завршни рад

**Приступ трансформације монолитне апликације у програмски
систем са микросервисном архитектуром**

Вук Манојловић 2023/3023

Београд, 2025.

Сагласност чланова комисије
за преглед, јавну одбрану и оцену завршног рада

Комисија која је прегледала завршни рад

Студента: Вука Манојловић

Број индекса: 2023/3023

Под насловом Приступ трансформацији монолитне апликације у програмски систем са
микросервисном архитектуром
The Title of Master Thesis The transformation approach for a monolithic application into a software system
with a microservice architecture

У следећем саставу

Ментор: др Слађан Бабарогић, редовни професор

Први члан: др Марко Петровић, доцент

Други члан: др Мирослав Миновић, редовни професор

дала је електронску сагласност за јавну одбрану завршног рада.

РАДНА БИОГРАФИЈА

Вук Манојловић је рођен 21.06.2000. године у Јагодини. Основну и средњу школу, гимназију „Светозар Марковић“, завршио је са одличним успехом у Јагодини. Факултет организационих наука у Београду, смер Информациони системи и технологије, уписује 2019. године. Дипломирао је 15.09.2023. године са просечном оценом 9,02. Године 2023. Уписује мастер академске студије на Факултету организационих наука на модулу Информациони системи, смера Информациони системи и технологије. Говори српски и енглески језик и познаје рад на рачунару. Стручну праксу је обавио у Levi9 Technology Services.

Изјава о академској честитости

Име и презиме: Вук Манојловић

Број индекса: 2023/3023

Студијски програм: Информациони системи и технологије

Модул: Информациони системи

Аутор завршног рада под насловом	Приступ трансформације монолитне апликације у програмски систем са микросервисном архитектуром
Чија је израда одобрена на Седници Већа студијских програма мастер академских студија одржаној:	24.06.2024.

Потписивањем изјављујем:

- Да је рад искључиво резултат мог сопственог истраживачког рада;
- Да сам рад и мишљења других аутора које сам користио у овом раду назначио или цитирао у складу са Упутством;
- Да су сви радови и мишљења других аутора наведени у списку литературе/референци који су саставни део овог рада и писани су у складу са Упутством;
- Да сам довио све дозволе за коришћење ауторског дела који се у потпуносни/целости уносе у предати рад и да сам то јасно навео;
- Да сам свестан да је плагијат коришћење туђих радова у било ком облику (као цитата, парафраза, слика, табела, дијаграма, дизајна, планова, фотографија, филма, музике, формула, веб сајтова, компјутерских програма и сл.) без навођења аутора или представљање туђих ауторских дела као мојих, кажњиво по закону (Закон о ауторским и сродним правима, Службени гласник Републике Србије, бр. 104/2009, 99/2011, 119/2012), као и других закона и одговарајућих аката Универзитета у Београду и Факултета организационих наука;
- Да сам свестан да плагијат укључује и представљање, употребу и дистрибуирање рада предавача или других студената као сопствених;
- Да сам свестан последица које код доказаног плагијата могу проузроковати на предати завшни мастер рад и мој статус;
- Да је електронска верзија завршног рада идентична штампаном примерку и пристајем на његово објављивање под условима прописаним актима Универзитета и Факултета.

Београд, 27.01.2025. године

Манојловић В
Потпис студента

Резиме

Циљ теме мастер рада јесте трансформација монолитне апликације у систем са микросервисном архитектуром. Самим тим, предмет истраживања мастер рада обухвата истраживање конкретних техника трансформације стилова развоја софтверског система, као и проблема и изазова који се јављају у таквом процесу трансформације. Такође, предмет истраживања мастер рада је усмерен и на приказ различитих стилова развоја софтверских система, при чему су два кључна стила овог мастер рада монолитна и микросервисна архитектура.

Главна сврха мастер рада јесте разумевање трансформације поменутих два стила развоја софтверског система. Осим истраживања проблема и изазова који прате трансформацију, рад има за циљ и објашњење конкретних техника које се користе приликом трансформације монолитне апликације у микросервисни систем. Ово обухвата начин њихове реализације, њихове резултате, изазове и ограничења. Коначни циљ мастер рада јесте примена наученог кроз реализацију трансформације на конкретном примеру, уз обухватање свих техника, проблема и изазова који су претходно анализирани у теоријском делу мастер рада.

Резултат мастер рада је нова микросервисна архитектура, настала трансформацијом конкретног примера монолитне апликације применом одговарајућих техника и принципа. Процес трансформације је представљен кроз сликовит приказ развоја архитектуре након примене одговарајућих техника миграције, и то од монолитног облика до коначне микросервисне архитектуре.

Кључне речи: Микросервиси, микросервисна архитектура, монолитна архитектура, монолитна апликација

Keywords: Microservices, microservice architecture, monolithic architecture, monolithic application

Листа акронима

Акроним	Значење
API	Application Programming Interface
URL	Uniform Resource Locator
IoT	Internet of Things
HTTP	HyperText Transfer Protocol
IT	Information Technology
REST	Representational State Transfer
UI	User Interface
PDF	Portable Document Format

Листа слика

Слика 1. Слојевита архитектура [6]	12
Слика 2. Пример модуларне монолитне архитектуре [7].....	14
Слика 3. Архитектурални стил развоја сваког модула у модуларном аритектуралном стилу [8].....	15
Слика 4. Вертикални пресек приликом дефинисања модуларне архитектуре [8].....	16
Слика 5. Илустрација зависности сервиса дистрибуираног монолита [11]	19
Слика 6. Распоред компоненти сервисно-оријентисане архитектуре [14]	25
Слика 7. Дијаграм сервисно-оријентисане архитектуре [15].....	25
Слика 8. Event-driven архитектура [18]	28
Слика 9. Space based архитектурални патерн [21]	34
Слика 10. Компонента јединице обраде SBA [21].....	35
Слика 11. Раздвајање монолитне апликација у микросервисе [22]	38
Слика 12. <i>Strangler fig</i> патерн миграције на примеру мнолитних апликација [29]	55
Слика 13. Процес имплементације <i>Strangler fig</i> application патерна [24]	56
Слика 14. Приказ разлике монолитног и микро <i>frontend</i> типа апликације [30].....	59
Слика 15. Независна испорука микро <i>frontend</i> -а [32]	60
Слика 16. Софтверски систем са функционалношћу која се мења и кодом који га позива [33].....	62
Слика 17. Први корак – креирање апстракције [33]	62
Слика 18. Други корак – преусмеравање клијент кода на нову апстракцију [33]	63
Слика 19. Трећи корак – креирање нове имплементације [33]	64
Слика 20. Четврти корак – преусмеравање на нову имплементацију [33]	64
Слика 21. Verify Branch by Abstraction патерн [34].....	65
Слика 22. Патерн параленог извршавања [35].....	66
Слика 23. Пример <i>decorating collaborator</i> патерна [36].....	68

Слика 24. Пример коришћења change data capture патерна [24]	69
Слика 25. Патерн поделе табеле базе података [24].....	76
Слика 26. Монолитни систем са табелама повезаних кључевима пре примене патерна померања везе у код [24].....	78
Слика 27. Микросервисни систем са одвојеним табелама након примене патерна померања везе у код [24].....	79
Слика 28. Монолитни систем са дељеним референцираним статичким подацима [24]	82
Слика 29. Микросервисни систем са дуплираним референцираним статичким подацима [24]	83
Слика 30. Микросервисни систем са дељеном шемом базе података [24].....	83
Слика 31. Општа архитектура монолитног система.....	88
Слика 32. Детаљна слојевита архитектура конкретног примера монолита.....	90
Слика 33. Проху посредник између корисничког интерфејса и монолитне апликације ..	94
Слика 34. Миграција функционалности питања и одговора на нов микросервис.....	96
Слика 35. Умрежавање микросервиса са функционалностима монолита	98
Слика 36.Миграција дела функционалности на нови микросервис и преусмеравање ка истом	99
Слика 37. Преусмеравање позива на нови микросервис	100
Слика 38. Резултат реализије strangler fig application патерна миграције над свим функционалностима	101
Слика 39. Архитектура микросервиса за аутентификацију „auth-service”	103
Слика 40. Креирање апстракције над сервисом слања мејлова	104
Слика 41. Успостављање комуникације са апстракцијом слања мејлова.....	105
Слика 42. Креирање нове имплементације и микросервиса слања мејлова.....	106
Слика 43. Логика пребацивања између старе и нове имплементације путем „Abstract Factory“ дизајн патерна.....	107

Слика 44. Преусмеравање контролера аутентификације на коришћење нове имплементације слања мејла за промену заборављене лозинке	108
Слика 45. Резултат реализације „ <i>Branch By Abstraction</i> ” патерна миграције	109
Слика 46. Дијаграм секвенце аутентификације корисника на систем.....	111
Слика 47. Конкретан пример „ <i>Decorator Collaborator</i> “ патерна миграције	113
Слика 48. Приказ микросервисне архитектуре након реализације „ <i>Decorator Collaborator</i> ” патерна миграције	114
Слика 49. Концептуални дијаграм доменских класа монолитног софтверског система	115
Слика 50. Раздвајање домена података микросервиса питања и проблем везе са асоцијативном табелом питања и тестова	117
Слика 51. Приказ провере података на микросервису питања приликом избацивања питања са теста	118
Слика 52. Заједничка база података за асоцијативну табелу питања и тестова.....	119
Слика 53. Резултат издвајања базе података питања и повезивања са микросервисом питања	120
Слика 54. Коначна микросервисна архитектура као резултат свих миграција монолита	121

Садржај

1. Увод.....	1
2. Архитектурални стилови развоја софтвера	4
2.1. Монолитна архитектура.....	10
Слојевита архитектура.....	11
Модуларна монолитна архитектура	14
Дистрибуирана монолитна архитектура	17
2.2. Дистрибуирана архитектура.....	20
Сервисно-оријентисана архитектура	22
<i>Event-driven</i> архитектура	27
<i>Space-based</i> архитектура	32
Микросервисна архитектура	37
3. Проблеми, изазови и начини миграције у систем са микросервисном архитектуром.....	42
3.1. Изазови и проблеми миграције	44
3.2. Патерни миграције	51
3.3. Подела домена података.....	71
4. Приступ трансформације монолитне апликације у програмски систем са микросервисном архитектуром	86
4.1 Вербални опис	86
4.2 Архитектура монолита	87
4.3 Поступак миграције	92
5. Закључак	123
Литература	126

1. Увод

Услед динамичког и брзо растућег технолошког окружења, компаније се суочавају са изазовима прилагођавања свог пословања динамичном тржишту, пуног новитета. Компанија своје пословање може реализовати путем разлитих апликација, система, платформи, инфраструктура које јој омогућавају реализацију циљева пословања, као и приказ резултата тржишту. Самим тим, врло важан корак приликом реализације пословања јесте утврдити опцију, односно начин реализације, која може компанију утврдити водећом на тржишту и тиме је учинити продуктивнијом, ефикаснијом и профитабилнијом. Традиционална апликација, са целокупном логиком садржаном на једном месту, представља проверени потез реализације пословања многим компанијама. Са целокупном логиком на једном месту, одржавање овакве апликације је далеко лакше и једноставније. Међутим, са порастом захтева тржишта расте и опсег функционалности које апликација мора да обухвата. Од мале апликације, са функционалностима које су чврсто везане и спаковане на једном месту, настаје апликација са далеко већим доменом функционалности које нису зависне једна од друге, али их архитектура система у којој се налазе таквим чине. Пословање почиње да се шири а првобитна апликација, са традиционалном архитектуром, постаје нежељена опција даље реализације пословања. Из потребе за бржим, ефективнијим и оптималнијим пословањем, настали су различити правци, стилови, који су омогућили да се пословање подели и реализује на више различитих и независних целина које пружају бољи увид у функционалност коју обухватају, као и бржу и продуктивнију имплементацију.

Циљ мастер рада јесте проучити и истражити начин на који компаније своје пословање унапређују и развијају кроз поступак трансформације софтверских архитектура над којима је поменуто пословање реализовано. Конкретно, у мастер раду се детаљно истражује поступак трансформације апликације монолитне архитектуре у софтверски систем са микросервисном архитектуром. Поступак трансформације обухвата многобројне процесе, ограничења, услове и ризике који се морају узети у обзир приликом реализације трансформације софтверских архитектуралних стилова. Идеја мастер рада није само истраживање и реализација конкретног процеса трансформације апликација, већ захтева дубоко размишљање о проблемима који

оваква трансформација проузрокује и изношење идеја како се такви проблеми могу превазићи. На основу тога постоје различити архитектурални приступи трансформације који се могу реализовати ради превазилажења таквих препрека. Међутим, трансформација у микросервисну архитектуру не мора нужно означавати добру ствар. Одржавање и подизање нових микросервиса може бити скуп и захтеван процес, због чега идеја о одржавању апликације са целокупном логиком на једном месту не представља увек лош избор. Уколико се нова функционалност може лако интегрисати са осталим функционалностима монолитне апликације онда нема потребе ка започињању процеса трансформације у микросервисну архитектуру. Са тим у вези, одлука о покретању поступка трансформације у микросервисну архитектуру се мора детаљно обрадити, истражити и након тога утврдити план којим ће се пословање водити (Њуман, 2019, стр. 33-34). Микросервисна архитектура није једина архитектура коју компаније интегришу у своје пословање. Постоје различити архитектурални стилови развоја софтвера, од сервисно оријентисане архитектуре ка архитектури оријентисаној ка догађајима, којим се пословања служе приликом развоја функционалности. Из тог разлога у мастер раду је дата реч о таквим архитектурама, њиховим особинама, начину имплементације, коришћења и сличног.

Резултат истраживања мастер рада јесте имплементирана микросервисна архитектура пословног домена изабране монолитне апликације. Након спроведеног детаљног истраживања о многобројним проблемима, ограничењима и изазовима са којима се пословање суочава приликом трансформације у микросервисну архитектуру, може се приступити конкретној имплементацији такве трансформације. У мастер раду је дата посебна реч о ограничењима и проблемима које микросервисна архитектура може проузроковати и из тог разлога је важно да, пре реализације поступка трансформације, у мастер раду се прво прођу неопходне чињенице везане за овакву трансформацију. Током реализације микросервисне архитектуре коришћени су алати и технологије неопходни како за подизање микросервиса и њиховог пуштања у продукцију, тако и за конкретно писање кода таквих микросервиса.

Структура рада

У почетном, првом, делу рада су описане карактеристике и принципи различитих архитектуралних стилова развоја софтвера. Поред описа архитектура које се користе за потребе мастер рада, дата је реч и о другим архитектуралним стиливима. Поред описаних карактеристика и принципа над којим се заснивају, даје се преглед компоненти њихове архитектуре, увид у то како се оне користе, имплементирају и слично.

Претходно је напоменута важност идентификације и проучавања проблема и изазова који настају поступком трансформације у микросервисну архитектуру. Другим делом мастер рада се то и покрива. У овом делу се даје опис идентификованих проблема и изазова као и начин којим се превазилазе. Самим тим, идентификују и описују се начини миграције у микросервисну архитектуру којим се превазилазе идентификовани проблеми и који се користе за потребе конкретне реализације трансформације над изабраним примером монолитне апликације.

Трећим делом мастер рада се приступа конкретној имплементацији трансформације монолитне апликације у микросервисну архитектуру. За реализацију ове трансформације користе се различити алати и технологије који се у овом делу мастер рада описују. Цео поступак трансформације је приказан и даје се детаљан преглед реализованих поступака миграције. Као резултат миграције, настаје програмски систем са микросервисном архитектуром са пословним доменом изабране монолитне апликације. Коначан исход овог дела мастер рада јесте слика архитектура реализоване трансформације монолитне апликације у ситем са микросервисном архитектуром.

2. Архитектурални стилови развоја софтвера

Пре почетка развоја софтверског система, програмери морају поставити план и донети одлуке неопходне за функционисање софтверског система. Од одлука попут одабира начина форматирања писаног кода, преко одабира програмског језика, система за управљање базом података, библиотека, алата па све до одлука које се тичу самог тима, њихове поделе рада, задатака и циљева, представљају кључне одлуке у процесу развоја софтверског система. Међутим, честа лоша пракса међу тимовима програмера јесте што, иако можда претходно постоји план, дешава се да кодирање започне без претходно успостављеног принципа, дизајна и архитектуре софтверског система. Полази се са претпоставком да се о таквим стварима може донети одлука касније, током процеса развоја софтверског система.

Добро утврђена архитектура и дизајн, којим се тежи, знанто олакшава посао програмерима приликом развоја софтверског система. Јасна визија помаже програмерима да делују проактивно и тиме повећају ефективност и продуктивност тима. Врло често се концепти софтверске архитектуре и софтверског дизајна мешају. Пре него што се опишу архитектурални стилови развоја софтвера, неопходно је разумети шта је софтверска архитектура, шта долази са њом и како се разликује од архитектуралних патерна који се често доводе у вези са појмом архитектуралне структуре. Такође, архитектурални патерни нису једини софтверски патерни који програмери користе приликом развоја софтвера. Патерни пројектовања се често доводе у везу са овим патернима и у наставку је дата разлика између ових појмова.

Постоје разне дефиниције о софтверској архитектури. Bass, Clements и Kazman (2013, стр. 3) као најбољу и најпрецизнију издвајају: „**Софтверска архитектура** је скуп структура неопходних за разумевање система, који укључује елементе софтвера, везе између њих као и њиховог својства“. Поједностављено, структура представља елементе који су спојени заједно везама између њих. Један софтверски систем је изграђен од више различитих архитектуралних структура, при којем ни један од њих не узима титулу да он сам представља архитектуру, већ то чине сви заједно.

Постоје три категорије архитектуралних структура (Bass, Clements & Kazman, 2013):

1. **Модуларна структура** – сваки модул садржи одговарајућу функционалност коју представља над којима се дефинишу радни задаци тимовима који су за њих задужени. Један модул може бити креиран за управљањем и одржавањем базе података, други модул се може односити на функционалност или правила пословања, трећи модул на кориснички интерфејс и слично. На основу претходног закључује се да сваки модул представља једну јединицу имплементације одређеног пословања. Постоје различити начини коришћења модуларне структуре. Декомпозиција одређеног пословања, разлагање пословања на ситније јединице (модуле), јесте структура декомпозиције модула. Такође, модуларна структура може настати из анализе и дизајна објектно-оријентисане парадигме програмирања – дијаграма класа. Слојевита (енг. *layered*) модуларна структура представља такође једну од опција која настаје као резултат интегрисања слојева путем модула софтверског система. Питања на које овај модул одговара се тичу питања везаних за функционалну одговорност сваког модула, софтверске елементе и који модули их могу користити, повезаност између различитих модула и друга.
2. За разлику од модуларне структуре, наредне категорије архитектуралних структура су динамичније, односно фокус оваквих типова структура је на начину како елементи структуре реагују и међусобно комуницирају током извршења. Структуре софтвера које се баве карактеристикама, понашањем и организацијом елемената, сервиса и система током њиховог извршавања јесу део **component-and-connector** (C&C) категорије архитектуралних структура. Структуре оваког типа пропагирају идеју да систем треба да буде структуриран као скуп елеманата који имају своје понашање и интеракције током извршавања. Елементи структуре представљају извршне компоненте (сервиси, клијенти, сервери, филтери и други извршни елементи) и конекторе који се односе на средства комуникације између компоненти, попут различитих оператора, цеви (енг. *pipes*) и других. На пример, бави се питањима попут тога које су то извршавајуће компоненте које су од значаја и како оне комуницирају током извршавања, који делови система могу да раде паралелно, на који начин подаци путују и напредују кроз систем и других.
3. **Структуре алокације** представљају мапирања од софтверских структура до

организационих, развојних, инсталационих и извршних окружења система. На пример један модул софтверске структуре се, поред тога што се приписује тиму програмера за развој, такође додељује различитим местима у структури датотека које служе за имплементацију, подизање, интеграцију, развој, тестирање и друге. Укратко, структуре алокације доносе одлуке о томе како ће се систем односити према другим структурама у свом окружењу. Под другим структурама подразумева се процесор, систем датотека, мреже и друге. Означава везу између софтверских елемената и елемената у једном или више екстерних окружења у којима ће се софтвер извршавати и подизати. На пример, бави се одлукама и питањима које се тичу избора процесора над којима ће се софтверски елементи подизати и извршавати, затим у којим директоријумима или фајловима ће софтверски елементи бити сачувани током развоја, тесирања и изградње система и других одлука.

Развој софтверских система и успостављање архитектуре, помоћу које ће функционисати, долази са својим проблемима и изазовима. У одређеним ситуацијама, елементи архитектуре софтверског система се могу ујединити тако да на одређени начин могу решавати изазове и проблеме са којима се систем суочава. Такав начин спајања елемената архитектуре се испоставио као добро решење за проблеме различитих домена током времена због чега је, овакав начин спајања архитектуралних елемената, добио назив **архитектурални патерн**. Већина оваквих патерна се бави решавањем проблема који долазе приликом имплементације архитектуралне структуре. У зависности од типа архитектуралних елемената који користе, архитектурални патерни се могу дефинисати у односу на категорију архитектуралне структуре. У оквиру модуларне структуре постоји **слојевити патерн** који се бави тиме да особине слојевите архитектуралне структуре буду испоштоване. На основу тога слојевити патерн води рачуна о томе да сваки сервис, који имплементира један слој архитектуре, може да користи само сервисе слоја који су испод њега. Тако постоји **клијент-сервер патерн**, патерн карактеристичан за C&S категорију архитектуралних структура. Овај патерн је карактеристичан по томе што постоје две компоненте: клијент и сервер. Конектори представљају протоколе и поруке које ове компоненте размењују како би сам систем функционисао. За потребе структуре алокације настао је **вишенивоски** (енг. *multi-tier*) **патерн**, патерн који описује како треба дистрибуирати и

алоцирати компоненте система у различите подскупове хардвера и софтвера који су повезани одређеним комуникацијским алатом или средством (Bass, Clements & Kazman, 2013).

Претходно су описане карактеристике и решења које нуде различите алтернативе архитектуралних патерна. Међутим, често постављено питање као и недоумица многих програмера приликом развоја софтвера јесте питање дефиниције патерна. Наизглед једноставна и лако разумљива реч, области и концепте које обухвата приликом развоја софтверског система залазе далеко дубље него што на први поглед може да се претпостави. Постоје различите дефиниције софтверских патерна. Заједничко свим дефиницијама јесте да софтверски патерни теже ка најидеалнијој и најоптималнијој структури софтверског система. Приликом појаве проблема, софтверски патерн се поставља како би такву структуру проблема, у којој се софтверски систем налази, трансформисао у структуру решења. Софтверски патерн није дефинисан за један тип проблема. Важна карактеристика софтверских патерна јесте поновна употребљивост за различите проблеме и изазове са којим се софтверски систем суочава (Влајић, 2014, стр. 3). На основу принципа и правила софтверских патерна настали су **патерни пројектовања** (енг. *design patterns*). Иако прављени за потребе дизајнирања објектно-оријентисаних софтверских система, велики утицај могу да имају на програмере који су се специјализовали за програмирање других парадигми програмирања. Наиме, софтверским патернима се упоставља правац ка оптималнијој, издржљивијој и ефективнијој архитектури софтверских система. Без обзира на тип и вид програмирања, тим програмера увек треба тежити ка томе да структура софтверског система буде флексибилна, стабилнија и оптималнија за многобројне проблеме и изазове на које наилази. Сумирањем претходних појмова и карактеристика, патерни пројектовања се могу дефинисати као описи комуникације између класа који су намештени тако да реше генерални проблем у одређеном контексту (Gamma, Helm, Johnson & Vlissides, 1995). Самим тим, само решење мора да буде специфично, како би конкретан проблем могао да се реши, као и довољно опште, да може да буде део или целокупно решење других будућих проблема, изазова или захтева (Влајић, 2014, стр. 43).

Претходно су објашњени различити видови патерна и разлика између њих. Међутим,

за потребе мастер рада неопходно је направити суштинску разлику између архитектуралних патерна и стилова, као и разумети архитектуралне стилове, њихове особине, карактеристике и шта конкретно представљају. **Архитектурални стил** дефинише и исказује елементе и њихове интеракције у оквиру архитектуре софтверског система. Важно је нагласити да архитектурални стил не решава тренутни проблем, већ дефинише један заједнички језик приликом дефинисања и конструисања архитектуре софтверског система. Самим тим, архитектурални стил је апстрактан, не ради се о конкретној библиотеци, алату, технологији или програмском језику, већ представља највиши ниво апстракције у дизајнирању софтверског система. Помоћу архитектуралних стилова може се схватити дизајн софтверског система на највишем нивоу, без залажења у детаље самог софтверског система. О детаљима софтверског система и његове архитектуре се баве архитектурални патерни. За конкретан проблем, архитектурални патерни могу да буду решење и начин да се имплементација одабраног стила заврши, коришћењем специфицираних техника и алата који су дефинисани у оквиру датог архитектуралног стила. У оквиру једног стила, се могу користити више архитектуралних патерна који су и претходно описани (Gelenler, 2023).

Сваки архитектурални стил развоја софтверског система се класификује у један од два главна архитектурална стила (Richards, 2015, стр. 5):

- **Монолитни** – једна свеокупна целина која се налази на продукцији
- **Дистрибуирани** – више различитих целина које се налазе на продукцији

Поред таквог вида класификације, архитектурални стилови се могу класификовати и према томе на који начин је њихова структура партиционисана. Самим тим, разликују се (Richards, 2015, стр. 9-12):

- **Технички партиционисане архитектуре** – архитектурални стилови чије компоненте система су организоване по техничкој корисности. Класичан пример овакве архитектуре јесте слојевита архитектура обзиром да сваки слој има неку своју техничку улогу. Тако компоненте презентационог слоја представљају кориснички интерфејс, перзистентне компоненте комуницирају са базом података и слично.
- **Доменски партиционисане архитектуре** – за разлику од техничких

партиционисаних архитектуралних стилова, компоненте доменско партиционисаних архитектура су организоване по домену проблема који обухватају. Свака компонента обухвата неку своју функционалност, од целокупне функционалности система, коју имплементира и извршава. Тиме једна функционалност у себи има све неопходне техничке компоненте (презентациону, бизнис и друге логике) које су груписане заједно са циљем успешне имплементације и извршавања функционалности. Под овим типом архитектуралног стила најчешће спадају видови дистрибуираног архитектуралног стила као што су микросервисна, сервисно-оријентисана и друге архитектуре.

Током претходне приче о архитектуралним патернима, структури и патернима пројектовања истакнут је значај одабира стила и правца којим ће се тим водити и држати приликом развоја софтверског система. Из тог разлога врло је важно разумети разлике и карактеристике сваке од категорија архитектуралних стилова који се бирају. Из наведених видова категорија архитектуралних стилова, у наставку се даје преглед најважнијих архитектуралних стилова у развоју софтвера, међу којима се налазе и стилови коришћени за потребе мастер рада.

2.1. Монолитна архитектура

Апликације са монолитном архитектуром су далеко једноставније него са дистрибуираном архитектуром. Пре свега, целокупна логика, функционалности, домен података и сами подаци се налазе на једном централизованом месту. Самим тим, процес налажења одређене функционалности, њеног мењања и дизајнирања је далеко једноставнији јер се зна на ком месту, где и како се тражени затеви могу реализовати. Испорука апликације на продукцију је такође олакшана из разлога што, као што је и напоменуто, апликације монолитне архитектуре су централизоване логичке целине које се заједно испоручују и покрећу над једним сервером, инстанцом, инфраструктуром или слично. За разлику од монолитних, дистрибуиране архитектуре подразумевају постојање више различитих целина (сервиса) над којима се различите функционалности (логичке целине) извршавају и покрећу. Више о дистрибуираном архитектуралном стилу и његовим особинама у наставку.

Према Њуману (2019, стр. 12), за системе са монолитном архитектуром се сматрају сви системи чија целокупна функционалност и логика на продукцији мора да се покреће и извршава заједно. Тиме, постојеће функционалности у монолитној апликацији су међусобно чврсто повезане и зависне чинећи је лабилном и осетљивом на различите проблеме, захтеве и нападе. Падом једног дела функционалности, монолитном апликацијом се доводе у опасност и друге функционалности и самим тим цела структура апликације. Падом једног дела монолита пада и читав софтверски систем направљен на основу монолитне архитектуре. Иако једноставност и цена јесу примамљиве предности монолитне архитектуре, због наведених ризика и мана представљају лош избор многим бизнисима чије пословање се непрестано шири и развија и захтева нове функционисања, невезане са постојећим, како би се прилагодили захтевима тржишта.

У наставку мастер рада су описани следећи примери монолитне архитектуре: Слојевита архитектура, модуларни монолит као и дистрибуирани монолит.

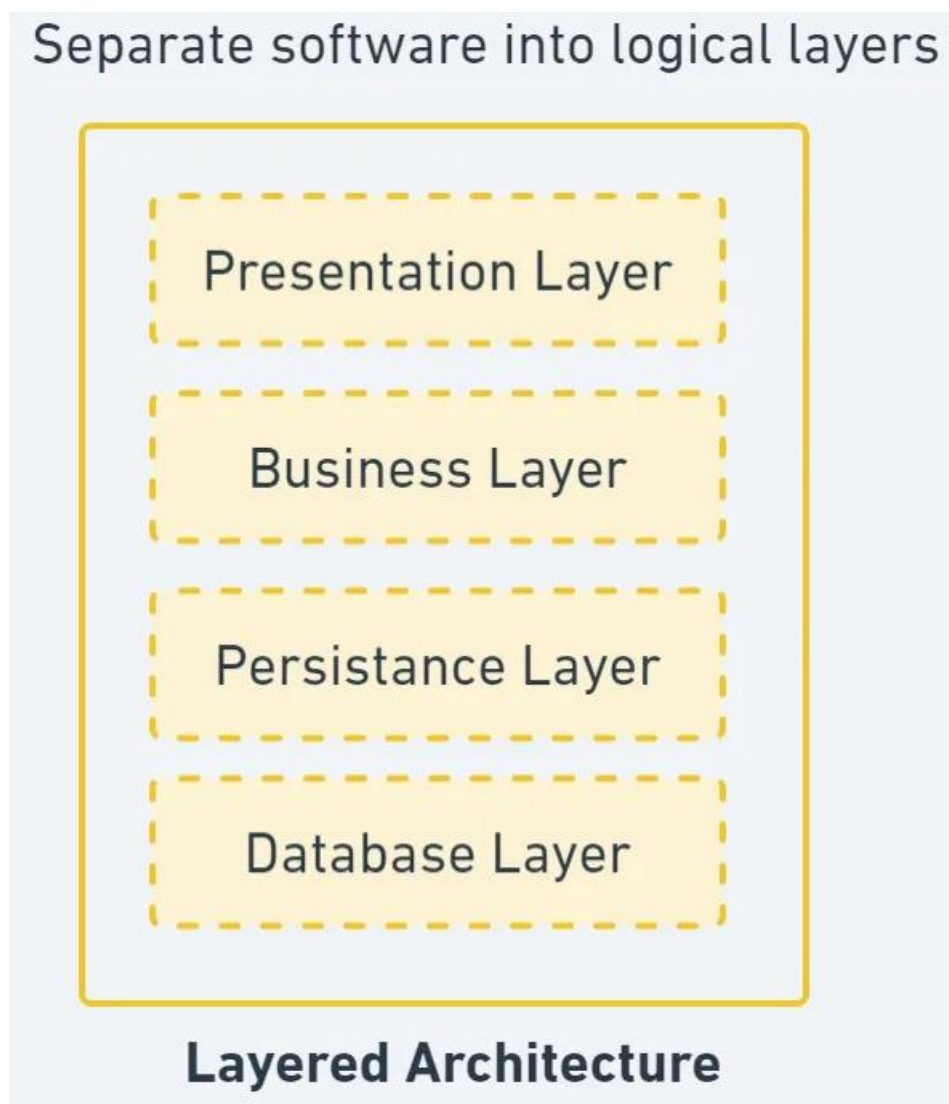
Слојевита архитектура

Приликом развоја софтверских система, први избор и помисао програмирања, за одабир архитектуре софтверског система, јесте слојевити архитектурални стил. Поклапа се са многим традиционалним начинима дефинисања и развоја софтверских система због чега представља стандардни избор приликом дефинисања архитектуре софтверског система. Обзиром да апликација са слојевитом архитектуром се састоји од више различитих слојева, овај тип монолитне архитектуре одговара организацијским структурама тимова таквим да сваки тим је одговоран за технички домен једног слоја слојевите архитектуре. Тако у организацијским структурама тимова постоји тим за управљање базом података, рад над корисничким интерфејсом, одржавање сервиса који комуницирају са слојем корисничког интерфејса и слично (Richards, 2015, стр. 15).

Сваки слој, унутар апликације са слојевитом архитектуром, је садржан од различитих компоненти и извршава одређену функцију која му је предодређена. Сви слојеви су организовани у хоризонталне слојеве и, иако апликација може садржати велики број слојева које испуњавају њихове тражене функционалности, најчешћи случај броја слојева јесте четири и то су: презентациони, бизнис, перзистентни и слој базе података. Презентациони слој представља слој корисничког интерфејса, осигурава комуникацију корисника са софтверским системом, док бизнис слој је одговоран за имплементацију логике неопходне за испуњавање захтева који долазе са презентационог слоја. Затим, перзистентни слој служи за комуникацију са базом података где је одговоран за парсирање података и објеката који се шаљу ка бази, као и за узимање из базе података. Слој базе података представља базу података одговорну за чување свих података неопходних за функционисање софтверског система. Врло важна особина слојевите архитектуре јесте да је сваки слој у оквиру софтверског система одговоран само за задатке који се тичу одговорности коју обухвата. Тако на пример, компоненте у оквиру презентационог слоја су одговорне за логику презентације, односно приказ корисничког интерфејса, код бизнис слоја компонентне су одговорне за логику која ће се извршавати за сваки захтев који дође са презентационог слоја и слично. Овим, компоненте сваког слоја не улазе у домен проблема компоненти другог слоја и образују апстракцију над својим слојем тако да једини вид комуникације које имају са другим слојевима јесте део путање извршавања одређене функционалности

софтверског система. Компоненте бизнис слоја не могу да утичу на то како ће презентациони слој приказати податке кориснику. Задатак компоненте бизнис слоја јесте да изврши неопходну логику и пошаље даље податке следећем слоју. Такође, компоненте презентационог слоја не занима начин како се логика извршава и како се подаци добијају, већ само на који начин ће такве податке представити корисницима (Richards, 2015, стр. 16-17).

На следећој слици је дат приказ слојевитог архитектуралног стила развоја софтвера:



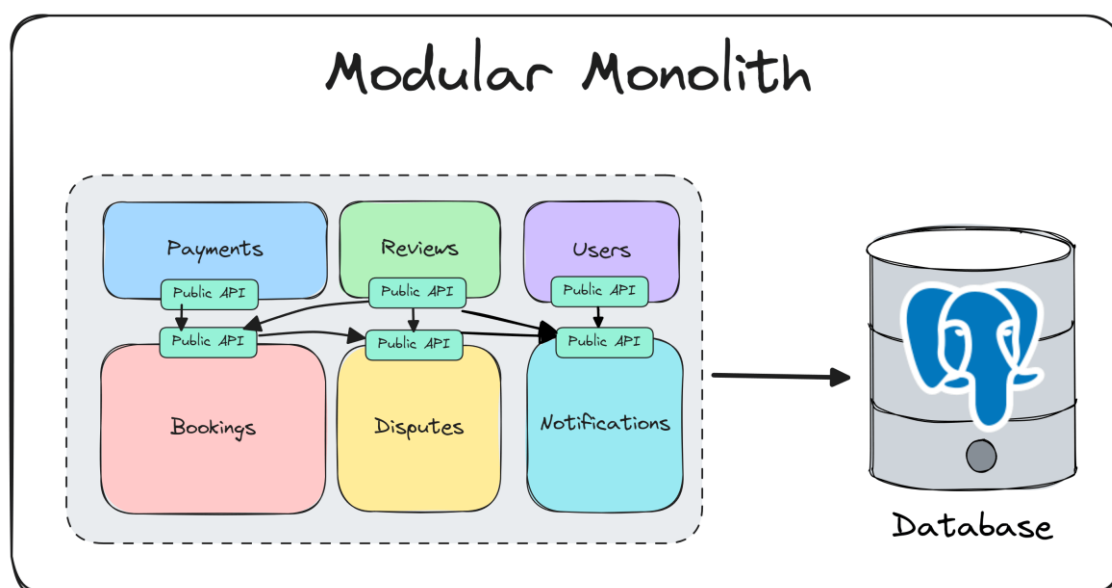
Слика 1. Слојевита архитектура [6]

Поред наведених концепта слојева слојевитог архитектуралног стила, који се тичу изолованости и одговорности који сваки од слојева има, трећи значајан концепт јесте отвореност и затвореност слојева. Затворени слој означава да захтев који треба да дође до њега мора да прође кроз одређени редослед слојева који се налазе пре или после њега. Тако на пример захтев за уписивањем корисничких података у базу података, прво мора да прође кроз презентациони слој, затим кроз бизнис па онда перзистентни и на крају да тек стигне до слоја базе података. Међутим, у одређеним ситуацијама се може тражити да захтев скаче са једног на други слој. У таквој ситуацији одговор на одређени захтев може бити знантно бржи јер прескаче пролазак датог захтева кроз један или више слојева. Овај концепт не би био могућ да слојеви у овом типу архитектуре не испуњавају особине већ поменутог концепта изолованости. Циљ концепта изолованости јесте да сваки слој буде одвојен и да садржи веома мало знања о томе како други слојеви раде. Због изолованости, са сигурношћу се може утврдити да скокови са једног на други слој неће утицати на рад слојева који се налазе између дата два слоја. Самим тим, слојеви који ће бити прескочени током извршавања одређеног захтева се сматрају отвореним слојевима, односно да прихватају принципе концепта отворености слојева (Richards, 2015, стр. 17-19).

Модуларна монолитна архитектура

Модуларна монолитна архитектура представља архитектурални стил развоја софтвера који структурира апликацију у систем са независним модулима или компонентама са добро дефинисаним правилима и ограничењима. Целокупна логичка целина је подељена на више различитих модула од којих је свака задужена за имплементацију, ограничења и правила једне логичке целине. Самим тим, логичке целине су подељене у зависности од њихових ограничења између делова целокупне логичке целине. Резултат овакве поделе логичке целине јесу модули који су *loosely coupled*¹, што додатно потпомаже циљу модуларне архитектуре да подељене целине буду донекле изоловане. Модули међусобно могу коминицрати помоћу API-а (Јовановић, 2024).

На следећој слици је дат пример модуларне монолитне архитектуре једног софтверског система:

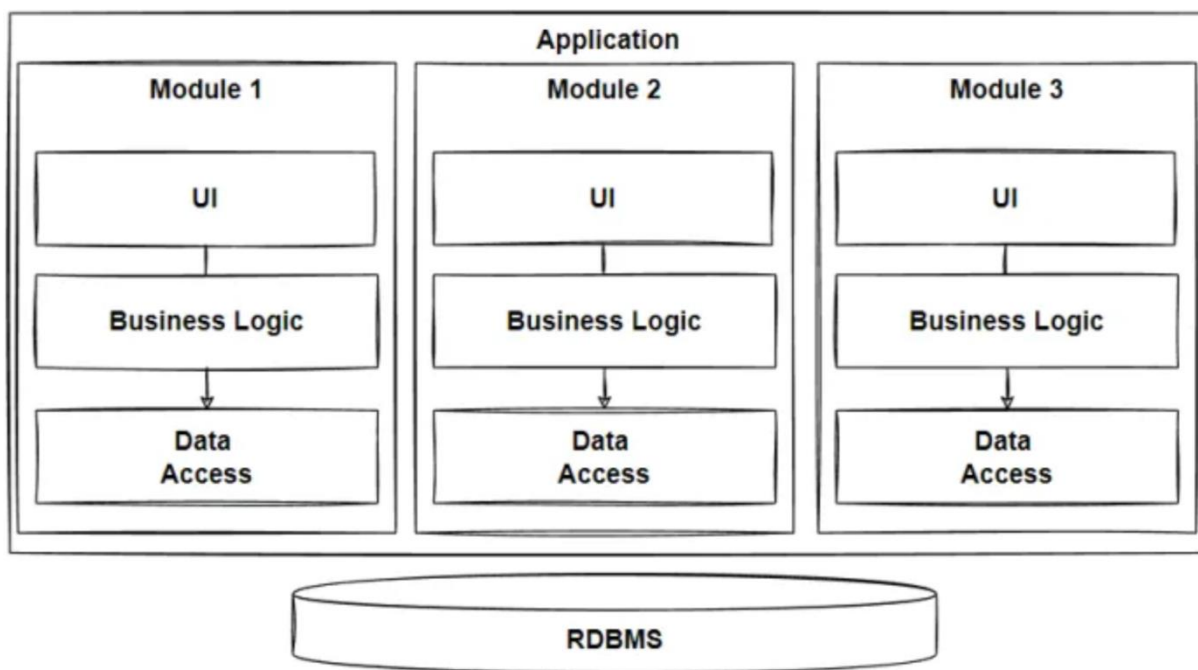


Слика 2. Пример модуларне монолитне архитектуре [7]

Претходно је напоменуто да сваки модул обухвата неку своју логичку целину од целокупне функционалности која апликација обухвата. Такође, сваки од модула може

¹ *loosely coupled* – представља приступ међусобном повезивању компоненти (модула) у информационом, софтверском или апликативном систему на тај начин да дате компоненте зависе једне од друге у најмањој могућој, изводљивој, мери. Реч „*coupled*“ односно „*coupling*“ односи се на степен зависности која једна компонента има према другој (Kirvan, 2023).

да прати неку своју логичку сепарацију којом ће се водити током развоја и имплементације. Тако на пример, на следећој слици приказано је да сваки од модула, за одабир архитектуралног стила његове имплементације логичке целине, користи слојевити архитектурални стил (Ozkaaya, 2023):

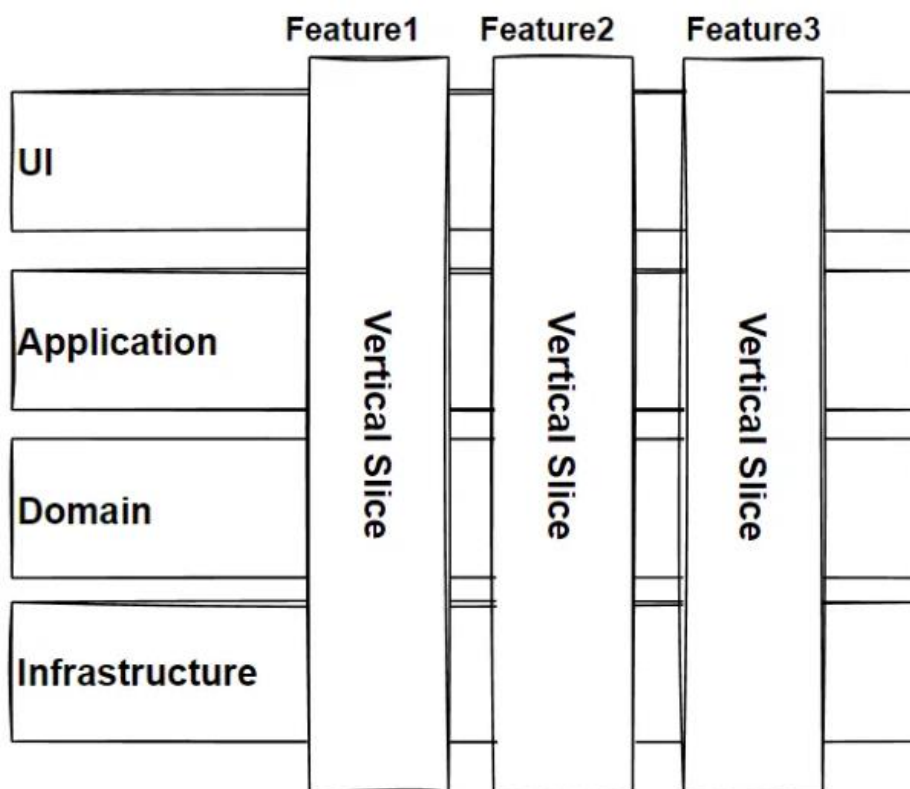


Слика 3. Архитектурални стил развоја сваког модула у модуларном аритектуралном стилу [8]

У модуларној монолитној архитектури, испорука на продукцију се и даље обавља као и са сваком монолитном апликацијом. Апликација се изграђује и подиже на продукцију као једна јединствена апликација због чега овакву архитектуру и убрајају као вид монолитног архитектуралног стила. Међитим, разлика у односу на класичну, традиционалну монолитну архитектуру је та што се изградња апликације врши тако што се целокупан код дели на више различитих целина које се придружују дефинисаним модулима у зависности од ограничења и правила логике коју обухватају. Организација кода за сваки од модула је различита у односу на слојевиту монолитну архитектуру. Наиме, са слојевитом архитектуром се код организује по хоризонталним логичким целинама тако да представљају по једну компоненту која учествује у току извршавања једног захтева, па самим тим имамо слој корисничког интерфејса, бизнис логике и друге. Код модуларне монолитне архитектуре, организација кода је по вертикали, односно кроз вертикалне пресеке бизнис функционалности. На тај начин, за

разлику од слојевите архитектуре, добија се више различитих модула, заснованих на функционалности коју пресецају, чиме се модули одређују, не на основу техничких органичења и правила, већ на основу функционалности које апликација садржи. Уколико се код организује по вертикали апликације, приликом додавања или промене функционалности апликације, ажурирање апликације се дешава над истим скупом слојева као и раније, само што уместо да се такве промене и додаци односе на техничке слојеве (кориснички интерфејс, бизнис, перзистентни...), бриге у вези промена и додатака се односе на бизнис страну, односно на функционалности апликације. Како расте број нових захтева тржишта, тако и софтверски систем, базиран на модуларној монолитној архитектури, такође расте организацијом кода по вертикали тако што се добијају нови модули или постојећи мењају (Ozkaа, 2023).

На следећој слици је приказан начин вертикалног пресека техничких компоненти софтверског система:



Слика 4. Вертикални пресек приликом дефинисања модуларне архитектуре [8]

Дистрибуирана монолитна архитектура

Апликација са дистрибуираном монолитном архитектуром је апликација која се на продукцији подиже и покреће као микросервисна архитектура али је изграђена и развијена као монолитна апликација. Микросервисна архитектура је један од видова дистрибуираних архитектуралних стилова који се базирају на различите јединствене целине, микросервисе, који се независно и засебно покрећу и развијају при чему сваки од микросервиса обухвата неку своју логичку целину од целокупне функционалности коју софтверски систем обухвата. Сваки од микросервиса има своје дефинисане границе домена које обухвата који се тичу података, функционалности и сличног. Због претходног, дистрибуирани монолит представља комбинацију оба стилова архитектуре – монолитне и микросервисне архитектуре (Newman, 2020).

Са дистрибуираним монолитом, апликација узима нефлескибилност монолита, у смислу тежине прилагођавања и увођења нових функционисања, док од микросервисне архитектуре узима комплексност реализације сваке од дистрибуираних целина. Тиме, дистрибуирани монолит настаје углавном као комбинација недостатака оба ова архитектурална стила са појединим предностима. Из тог разлога дистрибуирани монолит јесте архитектурални стил који треба избегавати и који може настати током погрешне имплементације микросервисне или монолитне архитектуре (Newman, 2020).

Сигнали који указују на појаву дистрибуиране монолитне архитектуре могу бити следећи (Newman, 2020):

1. Сервиси (дистрибуиране логичке целине) дистрибуираног монолита су *tightly coupled*² - У монолитној апликацији, функционалности (логичке целине) су међусобно зависне обзиром да деле заједничку базу кода на једном месту и покрећу се у оквиру једног истог процеса, заједно. На тај начин једна функционалност не може бити испоручена и подигнута на продукцији без да се

² *tightly coupled* – претходно је објашњен појам *loosely coupled*. Појам *tightly coupled* представља супротност датог појма у којем је комуникација и повезаности између компоненти и зависност између њих у највећој могућој мери.

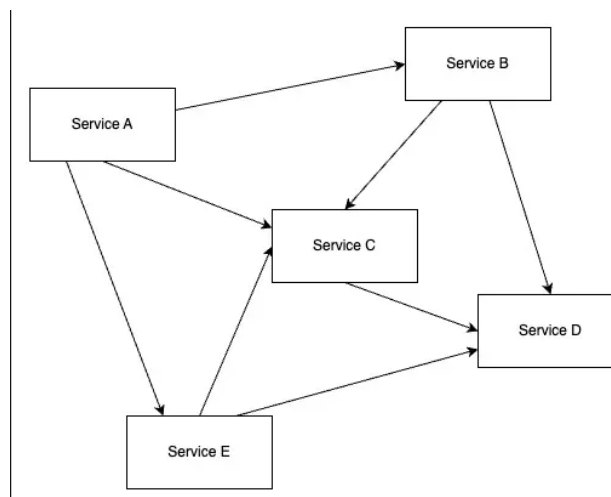
не изврше промене над функционалношћу од које зависи и која такође мора да буде испоручена и подигнута на продукцији. Проблем настаје када се такве функционалности одвоје у засебне сервисе које ће их имплементирати. Уколико узмемо пример монолита у којем постоји део *frontend*-а и *backend*-а, тада применом дистрибуираног монолита се праве два сервиса: један одговоран за *frontend* и други за *backend*. Насупрот њиховој међусобној комуникацији која је постојала у оквиру истог сегмента кода, на једном месту, сада ове компоненте комуницирају путем мреже (API-а). Конкретан проблем који настаје оваквом архитектуром јесте уколико један од сервиса падне, односно престане да функционише и ради. Рецимо, уколико падне сервис задужен за *backend* део, онда долази до проблема са сервисом *frontend* дела. Обзиром да су међусобно ови сервиси зависни, *frontend* сервис такође може да падне у зависности од степена зависности између ова два сервиса. Уколико *frontend* сервис може да ради без присуства другог, онда се доводи у питање и разлог његовог покретања на продукцији обзиром да никакву функционалност и одговоре не пружа корисницима, а оно што може проузроковати јесу само додатни трошкови одржавања, управљања и додатног програмирања.

2. Сервиси не скалирају једноставно – скалабилност представља једну од најзначајнијих одлика микросервисне архитектуре. Способност сервиса да скалирају брзо и ефикасно дозвољавају тимовима програмера олакшано управљање и контролу изненадних промена над сервисима, без оптерећења и забринутости о преоптерећености и потенцијалном паду целокупног система. Концепти хоризонталног (повећавање броја инстанци над којим ће сервис бити покренут) и вертикалног (промена конфигурације хардвера и других спецификација ресурса над којим је сервис подигнут) скалирања су од великог значаја за одржавање система са дистрибуираном архитектуром. Код дистрибуираног монолита, скалирање може представљати проблем. Разлог је тај што монолити захтевају подизање свих функционаности и целина заједно, што самим тим значи да са новом инстанцом на којој се систем подиже, подиже се и цела апликација заједно са свим њеним зависностима што доводи до тога да и остали сервиси буду поново подигнути обзиром да су зависни од сервиса који скалира. Иако по принципима микросервиса, сервиси

дистрибуираног монолита јесу донекле независни, за реализацију својих функционалности зависе од других сервиса и обзиром на принцип монолита да се морају сви заједно покретати, знатно се отежава да, апликација овог архитектуралног стила, буде ефикасно рукована и одржавана.

3. Сервиси међусобно превише комуницирају – дељење података између система са дистрибуираном архитектуром може бити изазовно са различитим проблемима. Обзиром на њихову комуникацију путем API позива преко мреже, падом саме мреже или недоступношћу једног од сервиса, могу настати различити проблеми и изазови који се тичу недоступности сервиса, података и сличног. Са микросервисном архитектуром је то лакше обзиром да сваки од микросервиса може функционисати и постојати независно од других без узроковања пада читавог система. Уколико сервиси превише комуницирају из потребе за подацима другх сервиса, онда таква комуникација може проузроковати спорију брзину враћања података као и проузроковати грешке приликом комуникације. Са дистрибуираним монолитом, због зависности између сервиса, свака грешка током комуникације или настанак одређеног периода чекања за одређеним податком (*timeout*) додатно проузрокује пад ефикасности и брзине рада читавог система, а такође и потенцијални пад система уколико потреба за одређеним подацима је хитна и неопходна.

На следећој слици је приказана илустрација зависност сервиса у дистрибуираној монолитној архитектури:



Слика 5. Илустрација зависности сервиса дистрибуираног монолита [11]

2.2. Дистрибуирана архитектура

Дистрибуирана архитектура подразумева систем са више подигнутих јединица који заједно раде како би извршили одређену бизнис функционалност. Предност овог архитектуралног стила долази из његових оперативних карактеристика где спадају скалабилност, толеранција, стабилност, агилност, перформанс система који имплементира такву архитектуру. Скалирањем софтерски систем се прилагођава оптерећењу повећањем или смањењем ресурса над којим се услуга извршава. Тиме, преусмеравањем захтева на друге инстанце исте услуге смањује се оптерећење система над једном инстанцом. Дистрибуирана архитектура је добро применљива за подршку одговора на пад система, односно садржи добар степен толеранције на пад самог система. Уколико један сервис падне, остали сервиси могу наставити са својим функционалностима или захтевима као да се ништа није ни десило. Са монолитном архитектуром то није могуће обзиром на чврсту повезаност и зависност функционалности унутар монолитне апликације. Агилност се односи на брзину којом се систем прилагођава промени. Односно, уколико дође до промене одређеног дела функционалности, само јединица, задужена за подизање и развијање тог дела функционалности, се мења и прилагођава променама без потребе за мењањем осталих јединица који комуницирају са датом. Јединице које имплементирају одређени део функционалности, које се подижу и функционишу самостално, се често називају сервисима и у наредним видовима дистрибуиране архитектуре појам сервиса се често појављује (Richards, 2015, стр. 7-8).

Међутим, поред наведених предности, недостатак који често спутава и умањује перформанс дистрибуиране архитектуре јесте мрежа помоћу које јединице имплементације комуницирају. Мреже падају, пуцају, прекидају услед различитих фактора као што су различити хакерски напади, падом ресурса који подижу мрежу и други. Самим тим, пад, прекид или пуцање мреже знатно отежава, ако не и недозвољава, комуникацију између јединица система. Обзиром да за реализацију целокупне функционалности је неопходна комуникација између целина које такву функционалност извршавају, оваква појава је честа међу различитим видовима дистрибуиране архитектуре (Richards, 2015, стр. 8).

У наставку је описано четири типова дистрибуиране архитектуре: сервисно-оријентисана, *event-driven*, *spaced-based* и микросервисна архитектура.

Сервисно-оријентисана архитектура

Сервисно-оријентисана архитектура (COA) представља стил архитектуре софтверског система која користи софтверске компоненте, сервисе, за креирање бизнис апликација. Сваки сервис садржи свој неки капацитет и ограничења које омогућавају да се, део функционалности коју тај сервис имплементира, реализује. Сервиси међусобно комуницирају путем различитих мрежа и платформи не би ли заједно реализовали потребе одређене бизнис функционалности. COA архитектура омогућава програмерима поновно коришћење функционалности једног сервиса током коришћења и имплементације других система и сервиса. На примеру функционалности аутентификације система, постоји један сервис који имплементира аутентификацију корисника на систем ради коришћења функционалности коју пружају други сервиси. Како код који имплементира аутентификацију не би био изнова поновљен и копиран у сваком од сервиса чију функционалност је неопходно заштити, сервиси којима је заштита неопходна шаљу захтев за аутентификацијом ка датом сервисом и тиме елиминишу потребу за имплементацијом аутентификације и понављању кода [12].

Не постоји детаљно упуство имплементације сервисно-оријентисане архитектуре. Међутим, најчешћи и најважнији принципи којим се тим програмера води приликом имплементације COA су следећи [12]:

- **Компатибилност** (енг. *Interoperability*) – софтверски систем може покретати било који сервис без обзира на програмски језик или платформу која је коришћена за потребе тог сервиса. Обзиром да сервиси нису међусобно зависни и не постоји директна интеракција, промене на једном сервисом неће утицати на друге компоненте које тај сервис користе.
- **Loose coupling** – претходно објашњен концепт *loosely coupled* је примењен и над овом архитектуром. COA сервиси међусобно не би требало да буду зависни при чему треба да се покрећу и имплементирају независно.
- **Апстракција** – клијент или други корисници сервиса у COA морају познавати логику функционалности и код који је задужен за такву логику. Самим тим, сваки сервис клијентима треба допремати са одређеном документацијом како би се разумео сам сервис и све што сервис пружа. Помоћу сервисних уговора (енг. *contracts*) се даје опис тачке путем које се приступа одређеној

функционалности. Односно, опис захтева, конкретне URL тачке на којој се функционалност налази, објекта који је неопходно послати и слично.

- **Грануларност** – сервиси би требало да имају одговарајући капацитети и опсег, идеално обухватајући једну бизнис функционалност по сервису.

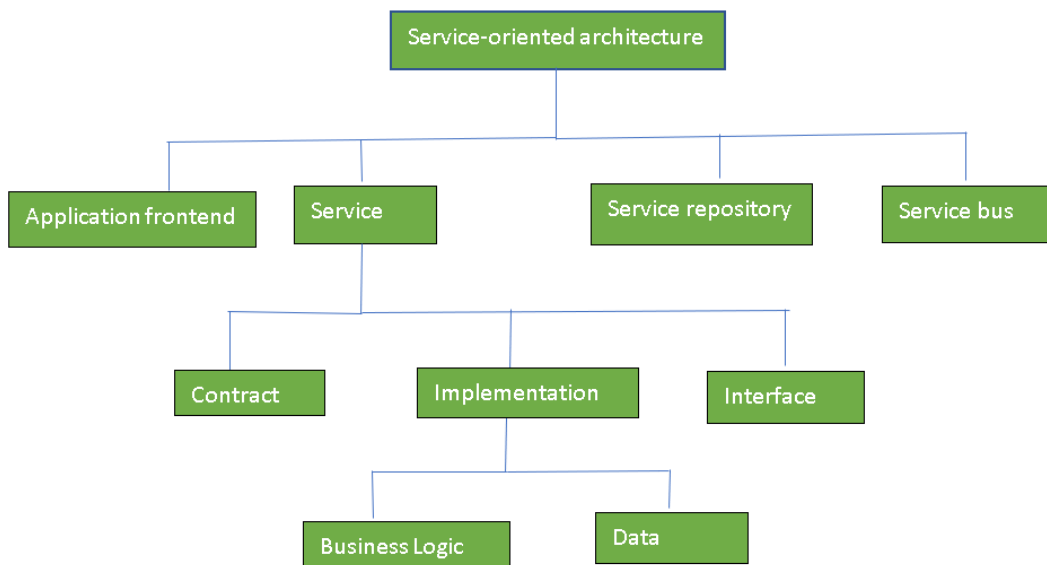
Постоје четири најважнијих компоненти сервисно-оријентисане архитектуре [12]:

1. **Сервис** - сервиси представљају блокове изградње, компоненте сервисно-оријентисане архитектуре. Могу бити доступни интерно, у оквиру организације, или путем интернета свима. Међутим, са тим, сваки сервис има три главне особине:
 - Сервисна имплементација – представља код који реализује логику функционалности.
 - Сервисни уговор – дефинише спецификацију сервиса и све њему приписане термине, ограничења и правила.
 - Сервисни интерфејс – у СОА, други сервиси или систем комуницирају са сервисом преко његовог интерфејса. Интерфејс сервиса дефинише на који начин се може позвати, извршити и комуницирати, односно размењивати податке, са жељеним сервисом. Тиме се смањују зависности између позиваоца сервиса и сервиса који је тражен. Тако на пример корисник, без знања о коду који имплементира сервис, може позивати тај сервис, узимати неопходне податке и реализовати тражену функционалност.
2. **Сервис провајдер** – креира, одржава и управља једним или више сервисом како би други могли да приступе и користе сервисе. Организације могу да креирају своје сервисе или да их купе од различитих *third-party* вендора.
3. **Сервис потрошач** (енг. *consumer*) – одговоран за слање затева ка сервис провајдеру за покретање одговарајућег сервиса. Потрошач може бити целокупан систем, сервис или апликација. Сервисним уговором се специфицирају правила које сервис провајдер и потрошач морају да прате када међусобно комуницирају како би на правилан начин користиле функционалности сервиса.
4. **Сервисни регистар** – сервисни репозиторијум, преставља директоријум свих доступних сервиса којима се може приступити путем мреже. Чува описну

документацију сервиса коју добија од сервис провајдера. Под описном документацијом подразумевају се информације које се тичу самог сервиса, комуникације са датим сервисом и слично. Сервис потрошачи могу лако наћи сервисе који им требају користећи се сервис регистром.

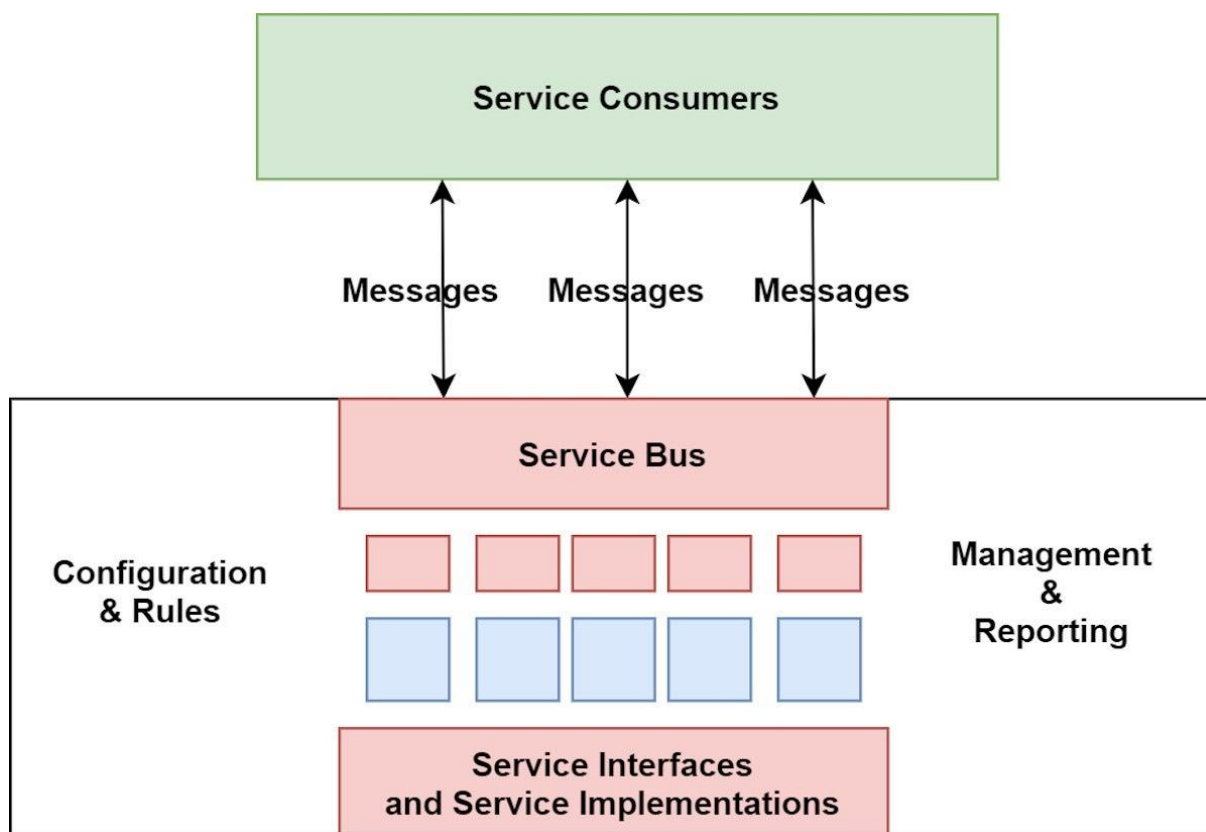
Међутим, поред четири главне компоненте, често у многим имплементацијама сервисно-оријентисане архитектуре јавља се и пета компонента. Компонента која потпомаже целокупном протоколу реализације и имплементације сервисно-оријентисане архитектуре – сервис „бусу“ или *enterprise service bus* (ESB). *Enterprise service bus* представља архитектурални патерн где једна централизована софтверска компонента извршава интеграцију између апликација. ESB извршава трансформације модела података, рукује конекцијом, рутира поруке, прави конверзију протокола комуникације и потенцијално може направити композицију долазећих захтева. Разлог зашто сервис бус није био набројан као део најважнијих компоненти сервисно-оријентисане архитектуре јесте због тога што COA се може изградити и без ESB-а. Међутим, изградити COA без сервис буса би било еквивалентно архитектури са гомилом неусклађених и некоординисаних сервиса. Свака апликација би морала директно да се конектује на тражени сервис и да одатле извршава неопходне трансформације података како би испуниле услове дефинисане интерфејсом сервиса. Овим би се креирали велики изазови одржавања такве архитектуре пре свега у конекцији са мрежи на којој се сви сервиси налазе. Начин како функционише јесте да сервиси, једноставном конекцијом, причају са ESB-ом и шаљу му неопходне податке, поруке и захтеве и препуштају ESB-у да трансформише протоколе и форматира податке, рутира поруке и слично, све у складу са дефиницијама и спецификацији интерфејса сервиса који је тражен. Оваквом интеграцијом између различитих апликација се знатно олакшава посао програмерима приликом имплементације. Са ESB-ом, посао интеграције апликација, прилагођавања протокола и сличног спада на ESB, при чему програмери мање времена троше на интеграцију, а знатно више на имплементацију, развијање и програмирање неопходне функционалности [13].

На следећој слици дат је детаљан приказ компоненти сервисно-оријентисане архитектуре:



Слика 6. Распоред компоненти сервисно-оријентисане архитектуре [14]

Дијаграм типичне сервисно-оријентисане архитектуре је приказан на следећој слици:



Слика 7. Дијаграм сервисно-оријентисане архитектуре [15]

Сервисно-оријентисана архитектура долази са својим бенефитима што је чини једном од популарнијих избора дистрибураних архитектуралних стилова. Као што је претходно споменуто, поновна употребљивост сервиса сервисно-оријентисане архитектуре чини ову архитектуру оптималнијим и ефективнијим избором. Свака функционалност сервиса се лако може интегрирати и користити у другим сервисима без претходне имплементације и увлачења кода потребног за реализацију такве функционалности. Изолованост и лако повезаност сервиса, омогућава се лако поновно коришћење тражених сервиса. Обзиром на карактеристику изолованости сервиса, сервисно-оријентисана архитектура се такође лако одржава обзиром да модификације једног сервиса неће утицати на перформансе других сервиса што процес подизања, испоруке и управљања сервиса далеко олакшава. Скалабилност, као главна предност дистрибуираних архитектура, такође је важна предност сервисно-оријентисане архитектуре. Могућност додавања додатних ресурса серверима над којим се сервиси покрећу, или проширење сервиса на више сервера, од великог је значаја за одржавање стабилности система на различите нападе, повећани број захтева и другог (Barney & Noelle, 2023).

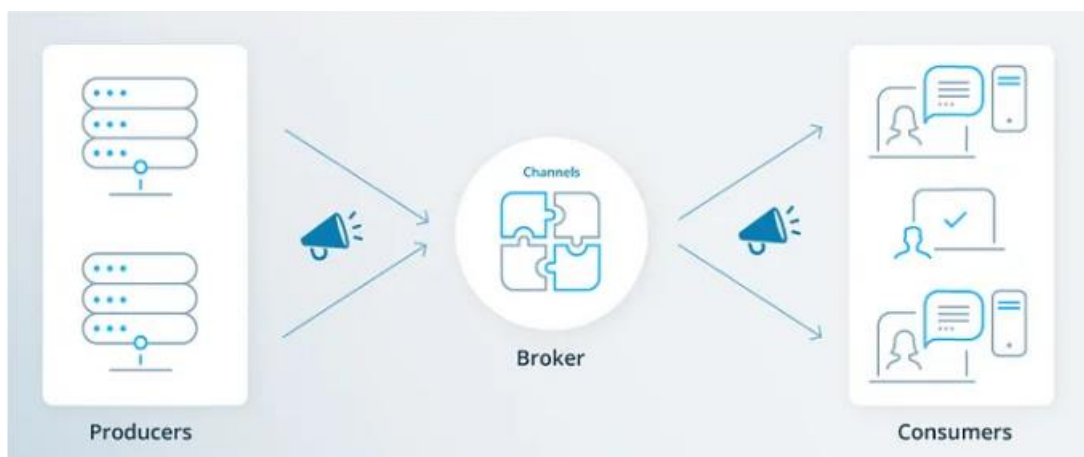
Поред бенефита, постоје лимити и недостаци које треба узети у обзир приликом имплементације сервисно-оријентисане архитектуре. Недостатак се јавља такође са једном од предности СОА, скалабилности. Наиме, скалабилност је лимитирана уколико сервиси морају да координишу са заједничким ресурсима. Уколико више сервиса деле један ресурс (базу података, други сервис...) канал ка датим ресурсом може бити преоптерећен што може проузроковати уско грло. Више различитих сервиса покушавају да приступају једној бази података што може довести до кашњења одређених података усед обраде више различитих трансакција. Поред лимитираности скалабилности, недостатак се може јавити и са менаџментом сервиса. Уколико сервиси међусобно размењују огроман број порука, праћење таквих порука може бити компликовано и захтевно. Претходно је поменут ESB са свим својим бенефитима које доноси својим увођењем у СОА. Међутим, СОА коришћењем ESB-а креира такозвани „*single point of failure*“. Овај концепт представља компоненту или део система чији квар може довести до престанка рада целокупног система. Уколико ESB падне, комуникација клијената са сервисима постаје немогућа што даље проузрокује компликације реализовања тражених функционалности (Barney & Noelle, 2023).

***Event-driven* архитектура**

Event-driven архитектура (EDA) представља софтверски дизајн патерн који омогућава организацијама да детектују догађаје (енг. *events*) или друга важна пословна дешавања (транзакције, посећивање сајта, куповна корпа и слично) и делује на њих у реалном времену или скоро реалном времену. Овим патерном се замењује традиционална захтев/одговор архитектура где сервис прво мора сачекати одговор на свој захтев како би могао наставити даље са извршавањем својих задатака. Ток EDA архитектуре покрећу догађаји и дизајниран је тако да одговара на њих или да изврши одређену акцију, транзакцију или слично као одговор на дати догађај. EDA се често помиње као имплементација асинхроне комуникације. Асинхрона комуникација означава да пошилац и прималац не морају да чекају једно друго како би прешли на извршавање других задатака. Целокупан рад система није завистан од јеног захтева, порука, које пошилац и прималац размењују и самим тим не постоји потреба за блокадом осталих задатака ради извршења једног. Класичан пример синхроног понашања и комуникације јесте телефонски разговор. Подносилац захтева реализује телефонски позив за одређеном акцијом где затим чека да особа, задужена за задатак, изврши одговорајућу радњу за дату акцију. Након пружаног одговора подносиоцу захтева, обе стране затварају комуникацију. Насупрот оваквом типу комуникације, постоји асинхрона комуникација која је карактеристична за EDA. Пример овакве комуникације би било дописивање порукама. Током писања порука, пошилац није у обавези да чека одмах одговор примаоца, већ такође може слати поруке колико год жели. Тачније, пошилац није ограничен чекањем одговора примаоца, већ је слободан да извршава своје задатке даље – шаље нове поруке [17].

Међутим, да би се разумело како EDA функционише, неопходно је дефинисати ша се под термином „догађај“ подразумева. Догађај се дефинише као промена стања неког кључног пословног система. Неко купи производ, пријави се за лет, аутобус касни и слично. Догађаји су приступи у сваком послу што значи да укључује све што је иницијално проузроковало да се порука креира, од производње, приказивања, детектовања или конзумирања. Сам догађај је одвојен од поруке. Док је догађај појава у току извршавања функционалности, порука представља обавештење које путује и преноси инициран догађај [17].

Компоненте EDA могу да укључе три дела: произвођача (енг. *producer*), потрошача и брокера. Брокер је опциони део обзиром да, уколико постоји један произвођач и потрошач који су у директној комуникацији, потреба за брокером нестаје и представља вишак. Најчеће, у многим пословним системима, постоје многобројни извори (произвођачи) који шаљу и окидају различите типове догађаја на које потрошачи морају да реагују. У таквим тренуцима је брокер важан како би преусмеравао догађаје на праве потрошаче и права места [17]. На следећој слици је приказан такав систем:



Слика 8. Event-driven архитектура [18]

За успешну и добру имплементацију EDA-е, постоји осам кључних концепта које треба разумети (Seetharamugn, 2023):

1. **Event broker** – представља пресретача захтева између различитих система који служи да рутира догађаје ка системима и назад. Све апликације се конектују прво на брокер где је затим брокер задужен за примање догађаја од пошиљалаца и за испоруку тих догађаја ка системима који су се преплатили и који су задужени за дате догађаје.
2. **Event portal** – Концепт портала омогућава дизајн, креирање, претраживање, дељење, визуализацију и управљање догађаја и EDA апликација. Портали догађаја (енг. *event portal*) могу бити примењене за три примарне групе публике:
 - Програмери архитекте могу користити портале да дефинишу, прегледају или воде дискусију о догађајима, дефиницијама података и релацијама апликација.

- Програмери могу користити портале за истраживање, разумевање и поновно коришћење догађаја кроз различите апликације, делове пословања или између спољних организација.
 - Научници података (енг. *data scientists*) могу да користе portale за разумевање података који се крећу кроз EDA.
3. **Topics** – мета подаци закачени за сваки од догађаја, чија је сврха опис догађаја за који су закачени, јесу теме (енг. *topic*). Тема представља *string*, текст који описује шта је у догађају. Пошиљачи приликом слања догађаја треба да знају ка којој теми шаљу догађај где затим брокер преузима одговорност слања и испоруке догађаја систему којем треба. Уколико корисници апликација имају интерес за неки догађај, могу се претплатити на тему која је релевантна датим догађајима.
 4. **Event Mesh** – креиран и омогућен путем интернет мреже за брокере који су путем интернета и повезани. Представља слој инфраструктуре за дистрибуцију догађаја ка издвојеним апликацијама, *cloud* сервисима или уређајима без обзира где се тражене апликације, сервиси или уређаји налазе у свету. Самим тим, *event mesh* представља мрежу међусобно повезаних брокера који размењују теме, информације или поруке како би на крају доспеле до претплатника на дате догађаје.
 5. **Одложено извршење** – Кључ EDA архитектуре јесте да једном када се догађај објави и пошаље, пошиљалац на одговор не чека. Како функционише ток EDA архитектуре јесте да брокер узима догађај и држи га за себе док не види да ли потрошач има интереса да прихвати и реализује догађај. Из тог разлога, може проћи доста времена између објављивања и слања догађаја до добијања одговарајућег одговора. Односно, слањем и објављивањем догађаја, догађај се неће нужно извршити одмах, већ његово извршење може бити одложено.
 6. **Евентуална конзистентност** – надовезујући се на концепт одложеног извршења, где идеја о томе да приликом иницирања догађаја се не зна нужно када ће се тај догађај извршити, настаје концепт о евентуалној конзистентности. Наиме, уколико се не зна да ли се догађај извршава или да ли ће бити извршен, сам пошиљалац догађаја не чека на тај догађај већ наставља даље. Због тога, пошиљалац догађаја не може са сигурношћу да тврди да подаци обрађени у догађају су у складу са свим што треба да се догоди приликом извршења

догађаја. На примеру базе података, уколико је инициран догађај који има задатак уписивање или измену одређених података, пошиљалац не може са сигурношћу да каже да ће подаци бити уписани и измењени у бази у том тренутку, већ може тврдити да ће евентуално ти подаци бити регулисани када се догађај буде извршио.

7. **Кореографија** – концепти одложеног извршења и евентуалне конзистентности воде ка концепту кореографије. Како би се пратила секвенца акција коју сервиси преузимају приликом реализације догађаја, постоји опција увођења једног највишег, главног – *master* сервиса. *Master* сервис би био задужен за праћење свих других сервиса и за предузимање акције у случају грешака. Овим приступом, односно оркестрацијом, се пружа једна тачка са које се може референцирати приликом праћења грешака или проблема али такође може представљати и ткз. „*single point of failure*“ и проузроковати уско грло. Са EDA архитектуром сервиси су нужни да разумеју долазећи догађај који такође може генерисати нове догађаје. Из тог разлога индивидуални сервиси морају ускладити рад над својим задацима са радом других сервиса са којима заједно праве координисани одговор који се шаље назад пошиљаоцу. Постоје комплексни догађаји који захтевају рад више различитих сервиса, због чега су концепти кореографије и усклађивања рада тих сервиса од великог значаја.
8. **Command Query Responsibility Segregation (CQRS)** – честа пракса скалирања сервиса јесте одвајање сервиса задужених за извршење команди, од сервиса задужених за извршавање и примања упита. Сервиси за примање и извршавање упита затим могу бити одговорни за упит уписивања, ажурирања, брисања и сличног. Међутим, са EDA архитектуром поједностављен је принцип због постојања тема. Тема би требала да садржи специфичну реч, на основу којих се креирају нове инстанце сервиса задуженог за упите, чији једини задатак би био да слуша теме са одговарајућим упитним глаголом (реч).

Из претходних принципа, правила, концепата и компоненти, могу се издвојити одређене предности које овај архитектурални стил чине примамљивим. Једна од главних предности EDA архитектуре јесте способност одговора на информације у реалном времену, брзина и једноставност приликом додавања нових сервиса ради значајнијег побољшања пословних процеса и корисничког доживљаја. Обзиром на

принцип асинхроне комуникације, све се дешава што пре и не чека се на одговоре. Односно, приликом инцирања догађаја, пошиљалац догађаја наставља даље ка извршавању својих активности што чини ову архитектуру ефективнијом приликом реализације више различитих, истовремених процеса. Обзиром да спада у групу дистрибуираних архитектуралних стилова, особине агилности и скалабилности сервиса важе и за овај архитектурални стил. Због способности да раде са подацима и догађајима у реалном времену, честу примену EDA архитектура уважава код случајева интеграције апликација, преноса и дељења података кроз апликације, конекције IoT уређаја ради анализе података и сличних [17].

***Space-based* архитектура**

На растуће потребе тржишта за бржим, скалабилнијим, еластичнијим и агилнијим софтверским системима, као одговор настаје архитектура, базирана на концепту „*tuple space*“, названа *space-based* архитектура (SBA) (Selin, 2023).

Tuple space представља имплементацију модела асоцијативне меморије за паралелно/дистрибуирано рачунање. Обзиром да је у питању асоцијативна меморија, нуди библиотеку *tuple*-ова којима се може приступити. Сваки од *tuple*-а садржи нула или више аргумената и кључева путем којих се приступа садржају. Колекција *tuple*-ова подржава основне операције додавања, брисања, ажурирања и сличних операција над *tuple*-ом. Колекција *tuple*-ова се чува и одржава путем мреже више *tuple space* сервера. Свеукупно, представља простор дељене меморије где се сви подаци чувају и којима могу приступити све јединице обраде (енг. *processing units*) (Rouse, 2012).

У својој основи, SBA је дизајн софтверске архитектуре усмерен на постизање линеарне скалабилности за високоперформантне апликације и системе. Начин на који ова архитектура постиже скалабилност јесте дистрибуцијом процеса обраде и складиштења на више сервера, самим тим стварајући тзв. „ништа заједничко“ (енг. *shared nothing*) систем. Системом „ништа заједничко“ сваки сервер ради независно и не дели ресурсе са другим серверима чиме рад једног сервера није условљен и зависан радом другог. Тиме, елиминишу се проблеми попут уског грла, повећава се ефективност и поузданост, јер квар једног сервера не утиче на рад другог. Претходно је објашњен концепт *tuple space* одакле и потиче термин *space-based* архитектуре, односно од принципа дељеног обика меморије где сви подаци бораве и доступни су свакој јединици обраде. За разумевање механизма SBA неопходно је разумети његове три основне компоненте (Selin, 2023):

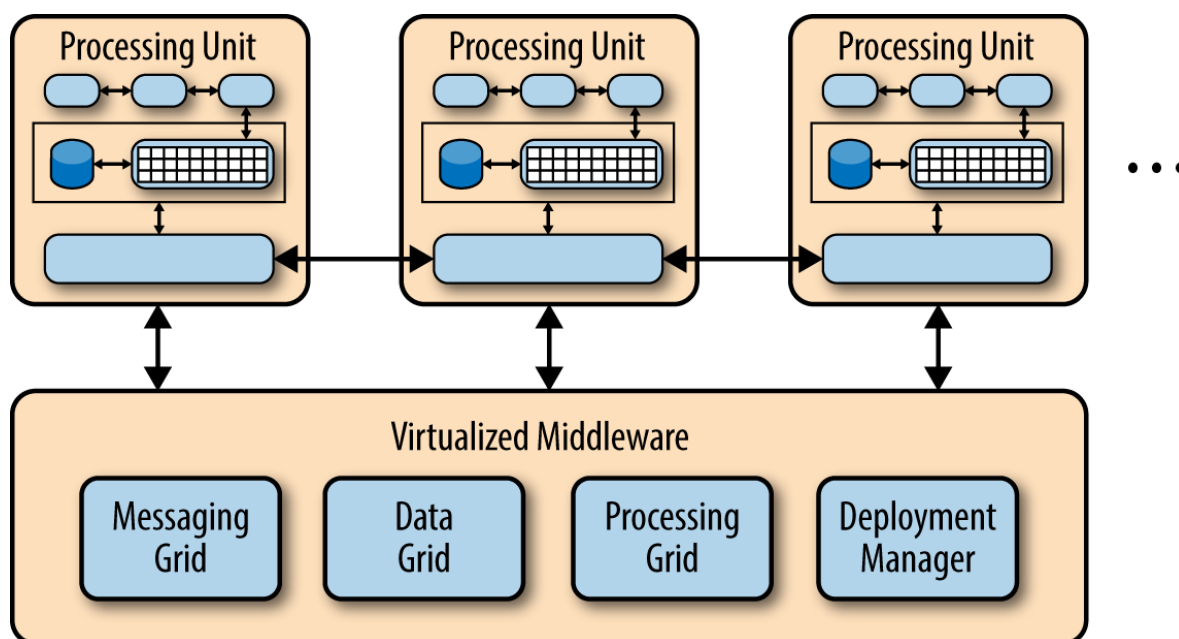
1. ***Јединице обраде*** – у SBA, јединице обраде су основне компоненте, градивни блокови читавог система. Свака јединица обраде је самостална и обухвата неку своју пословну логику и податке неопходне за обављање одређених операција. У суштини, свака јединица обраде представља мини систем за себе. Посматрањем јединице као мини система омогућава се боља флексибилност и модуларност обзиром да се на тај начин свака јединица може засебно

тестирати, имлентирати и развијати. Такође, јединица обраде је дизајнирана тако да се реплицира, односно садржи способност да се може копирати и покренути на више различитих сервера. Способношћу реплицирања се омогућава ефикасно балансирање оптерећења рада између јединица. Уколико се појави велики број захтева ка систему, рад се може равномерно расподелити између више јединица или уколико дође до одређеног кvara јединице, може доћи до преузимања рада једне јединице од јединице која је у кvarу.

2. **Простор** (енг. *Space*) – представља дистрибуирану, дељену мрежу у којој се налазе сви подаци. Представља централно место где се подаци чувају, убацују, ажурирају, преузимају, бришу и слично. Обзиром да све јединице обраде имају приступ простору, представља кључну компоненту за обезбеђивање да систем настави да функционише добро чак и уколико нека од јединица обраде падне и не успева.
3. **Рутер** – представља кључну улогу у оркестрацији укупног функционисања система. Задатак му је да усмерава захтеве клијената ка одговарајућим јединицама обраде, обезбеђује да се оптерећење рада равномерно расподели и осигурава да се сваки захтев обради у року.

Већина апликација које користе принципе и патерне SBA архитектуре јесу стандарне веб апликације које добијају захте од веб претраживача и извршавају одређену акцију. Такође, садрже одређену базу над којом уписују податке и слично. Разлог зашто је ова архитектура добра за веб апликације јесте због проблема које решава, а тичу се проблема скалабилности, истовременог извршавања и пренатрпаности рада као и проблем потенцијалног уског грла. Као што је претходно напоменуто, јединице обраде имају спообност да динамички буду покренуте и угашене, распоређене тако да не постоји оптерећење рада над системом. Због концепта *touple space*-а случај једне, централне базе података и комуникације свих јединица обраде над њом не постоји. Са SBA-а, централна база података се избацује и уводе се реплицирани меморијски простори који су распоређени и придружени свим активним јединицама обраде. Обзиром да нема једне, централне базе података, проблем уског грла је решен јер сада свака јединица обраде не приступа једној бази података, већ приступају реплицираном меморијском простору без бриге од пренатрпаности захтева ка њој од других јединица обраде (Richards, 2015, стр. 55-57).

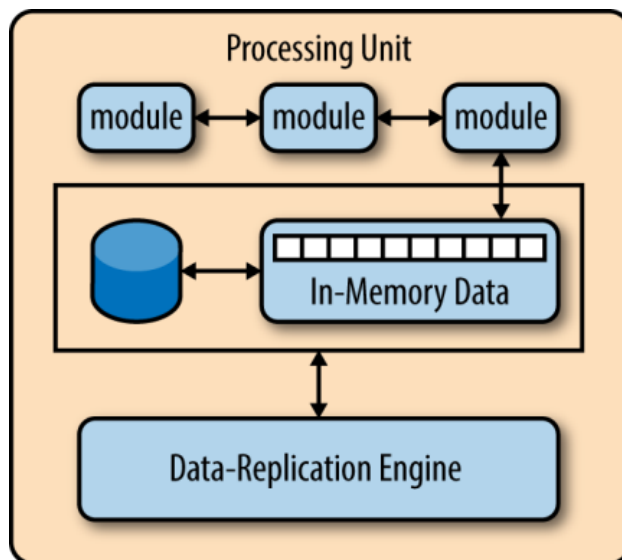
На следећој слици приказан SBA патерн са још једном компонентом која претходно није споменута– *virtualized middleware*.



Слика 9. Space based архитектурни патерн [21]

Две основне компоненте SBA патерна јесу: јединице обраде и *virtualized middleware*. Важно је напоменути да је претходна слика једног од могућих патерна SBA архитектуре који се може користити у многобројним ситуацијама и проблемима, при чему један од таквих служи и за решавање проблема једне, централизоване базе података која је претходно споменута. Компонента јединице обраде садржи апликационе компоненте. Под апликационим компонентама подразумевају се веб компоненте, код пословне логике и слично. Као што је и претходно речено, јединице обраде су управо и задужене за реализацију проблема, захтева и тражње тржишта. У зависности од величине апликације, може постојати једна јединица обраде на којој ће се налазити читава апликација или, уколико је апликација већа, постојати више јединица обраде од којих ће свака бити задужена за неки део пословне логике апликације. Јединица обраде обично садржи модуле апликације заједно са унутрашњим меморијском простором података (енг. *in-memory data grid*). Такође, садржи и компоненту механизма репликације (енг. *replication engine*) коју користи друга компонента, *virtualized middleware*, за пренос промене података коју направи једна јединица обраде на друге активне јединице обраде. Тиме се осигурава да сви делови система имају најновије податке и омогућава доследност и конзистентност у раду система (Richards, 2015, стр.

56-57). На следећој слици је дат приказ компоненте јединице обраде SBA патерна:



Слика 10. Компонента јединице обраде SBA [21]

Поред компоненте јединице обраде, друга важна компонента јесте *virtualized middleware*. Представља компоненту која се бави одржавањем и комуникацијом. У себи садржи компоненте које су задужене за контролисање различитих аспеката синхронизације података и обраде захтева. *Virtualized middleware* јесте контролер целе архитектуре који управља захтевима, сесијама, репликацијом података, дистрибуцијом обраде захтева и подизањем јединица обраде. Са претходне слике SBA патерна уочавају се четири главне компоненте *virtualized middleware*-а. Компонента за размену порука (енг. **Messaging Grid**) управља информацијама о улазним захтевима и сесијама. У тренутку када захтев дође до *virtualized middleware* компоненте, тада компонента за размену порука одређује које активне компоненте обраде су доступне да приме захтев и прослеђује захтев једној од тих јединица обраде. Компонента података (енг. **Data grid**) представља можда најважнију и пресудну компоненту овог патерна. Компонента података је у константној комуникацији са механизмом репликације јединице обраде како би ажурирала податке између јединица обраде приликом ажурирања података. Обзиром да компонента за размену порука може да прослеђује захтеве ка било којој јединици обраде, врло је важно да су подаци конзистентни у сваком меморијском простору свих јединица обраде. Реплицирање података између јединица обраде се обрађује паралелно, асинхроно, због чега сам процес реплицирања може трајати веома кратко, чак неколико микросекунди. Компонента за обраду (енг. **Processing grid**) јесте опциона компонента унутар *virtualized middleware* компоненте која управља

дистрибуираном обрадом захтева када постоји више јединица обраде, при чему свака рукује делом апликације. Уколико стигне захтев који подразумева координацију више различитих врста јединица обраде (на пример јединица за обраду наруџбина и јединица за обраду корисника), компонента за обраду је та која посредује и оркестрира захтев између таквих јединица обраде. Компонента за менаџмент и имплементацију (енг. ***Deployment-manager***) управља динамичким покретањем и гашењем јединица обраде на основу оптерећења које имају. Ова компонента континуирано прати време одзива и оптерећење корисника, и у зависности од тога поркеће нове јединице обраде када оптерећење расте, а када оптерећење опада тада гаси одређене јединице обраде. Представља кључну компоненту за постизање скалабилности унутар система (Richards, 2015, стр. 58-60).

Микросервисна архитектура

Микросервиси су архитектурални и организацијски приступ развоју софверских система где је софтвер изграђен од малих, независних сервиса који комуницирају преко јасно дефинисаних API-а. Софтверске сервисе поседују мали, самостални тимови задужене за њихову имплементацију и развој [22].

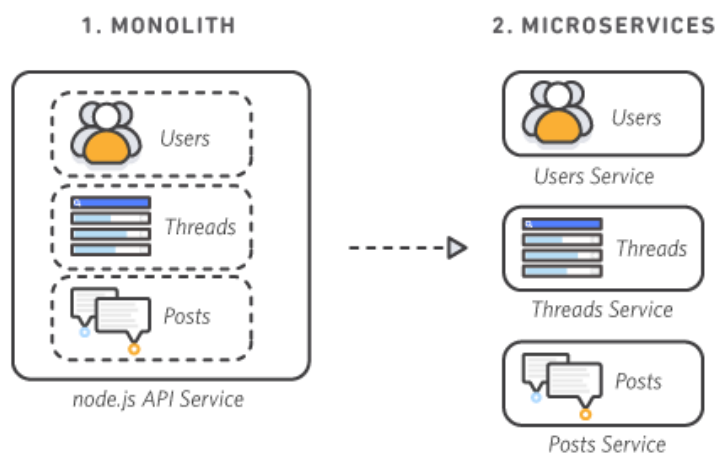
Према Њуману (2019, стр. 1), микросервиси представљају независне, имплементирајуће сервисе који су моделирани око домена пословања. Такође, представљају тип сервисно-оријентисане архитектуре, са чврстим ставовима о томе како треба поставити границе сервиса при чему је могућност независне имплементације кључно. За разумевање микросервиса, начина њиховог функционисања и имплементације, неопходно је покривати неколико главних принципа микросервиса, односно микросервисне архитектуре (Њуман, 2019, стр. 2-6):

- **Могућност независне имплементације и покретања** – идеја иза могућности независне имплементације и покретања јесте да се могу извршити промене над микросервисима и исте измене подигнути на продукцију без утицаја и промене над другим микросервисима. Самим тим, промене се брже испоручују на продукцију јер промена једино утиче на микросервис који обухвата дати пословни домен. Међутим, да би овај концепт функционисао, микросервиси морају бити „*loosely coupled*“, односно зависности између микросервиса мора бити минимална како би целокупан процес независне имплементације, подизања и покретања на продукцији био могућ.
- **Моделирани око пословног домена** – вршење измена кроз целокупан домен пословања може бити скупоцено и узети доста времена. Микросервисну архитектуру је неопходно пројектовати тако да, са новим променама, што мање микросервиса буде обухваћено променом. Уколико је микросервисни систем пројектован тако да промена једног домена утиче на два микросервиса и затим сам процес покретања и подизања се мора оркестрирати између два микросервиса, знантно би једноставније, јефтиније и ефективније било уколико би микросервисна архитектура и микросервиси били пројектовани тако да промена дела домена утиче само на микросервис који је задужен за тај домен.

Из датог разлога, врло је важно пројектовати микросервисе тако да сваки обухвати неки свој мали домен пословања како би се потенцијални проблеми, грешке или промене лакше имплементирале и идентификовале.

- **Поседовање сопственог домена података** – сваки микросервис треба да поседује свој домен података над којим врши захтеване операције ради реализације функционалности пословног домена. Самим тим, концепт заједничке, дељене, базе података између више различитих микросервиса је замењем постојањем посебног домена података (базе података) за сваки од микросервиса. Уколико су микросервису, за извршење одређеног захтева, неопходни подаци који се налазе у оквиру домена података другог микросервиса, микросервис једноставно, API позивом, шаље захтев ка другом микросервису који такве податке поседује. Поседовањем сопствених домена података се даје могућност микросервисима да одлуче који подаци могу бити дељени а који не. Претходно је напоменуто да микросервиси јесу мали делови једног великог пословног домена. Постојањем заједничке базе података се независно функционисање, имплементација и покретање знатно отежава јер микросервис се везује за одређени домен података који се њега не тиче и није неопходан домену пословања који обухвата.

На следећој слици је дат приказ разлагања монолитне архитектуре у микросервисну архитектуру:



Слика 11. Раздвајање монолитне апликација у микросервисе [22]

На основу претходних принципа и карактеристика могу се извући различити бенефити које микросервисна архитектура пружа софтверским системима (Њуман, 2015):

- **Хетерогеност технологија** (енг. *Technology Heterogeneity*) - представља коришћење различитих технологија за развој сваког микросервиса. Са софтверским системом који се састоји од више различитих сервиса, потребе сваког сервиса се могу имплементирати коришћењем различитих технологија. Тако апликација једног сервиса може бити писана у *Java* програмском језику, друга у *JavaScript* и тако редом. Тиме је тимовима програмера дозвољено да користе праве алате за потребе сваког сервиса, а не да су условљени коришћењем само једног истог скупа алата за потребе различитих функционалности. Карактеристике независности, самосталног покретања и одржавања сервиса без утицаја на друге, утичу на то да је бенефит коришћења различитих технологија међу микросервисима могућ. Такође, овај бенефит отвара и простор за лични напредак и усавршавање у погледу знања о новим технологијама. Уколико постоји интерес ка увођењу нове технологије, са системом који садржи више микросервиса програмери имају више различитих места где могу да испробају и тестирају технологију од интереса одабиром микросервиса са најнижим ризиком и утицајем на читав систем. Са монолитним апликацијама то не би био случај обзиром да је целокупан монолитни систем изграђен од једног стриктног скупа технологија и алата и увођењем новог може доћи до потенцијалних проблема и грешака у раду читавог ситета.
- **Отпорност** (енг. *Resilience*) – кључан концепт приликом пројектовања отпорности система јесте преграда (енг. *bulkhead*). Уколико један део система откаже, али тај квар се не шири даље, треба систем пројектовати тако да се дати проблем може изоловати а остатак система наставити са радом. У контексту микросервиса, њихове границе представљају преграде које их раздвајају да проблем ван не утиче на њих. Са монолитним апликацијама то није случај јер падом једног дела система, сви остали делови стају са радом. Постоји могућност да са монолитном архитектуром се могу покренути више различитих машина на којој ће се покретати читав монолитна апликација и на тај начин смањити шансу за падом система и обезбедити доступност на продукцији. Међутим, са микросервисима се изграђује систем који изолује проблем и спречава читав пад система тако што може да изолује, деградира функционалност и микросервисе погођене проблемом.

- **Скалирање** – концепт скалирања се код свих, претходно побројаних, дистрибуираних архитектура јавља као предност и разлог избора дистрибуиране архитектуре као вид реализације пословања. Такав случај је и са микросервисном архитектуром. Са монолитним апликацијама, приликом потребе за скалирањем једног дела функционалности, скалира се читав систем. Микросервиси дају могућност скалирања само оних микросервиса који у датом тренутку имају потребу за скалирањем услед пораста или пада захтева, као и преоптерећености радом или мањка рада. Тако скалирање може бити и повећање броја ресурса, као и смањивање све у зависности од ситуације у којој се систем налази.
- **Лако подизање и имплементација сервиса** – са монолитном архитектуром, промена над једној линијом кода захтева подизање и поновно покретање читаве апликације како би промена, која се имплементирала, била доступна на продукцији. Поред проблема што је само подизање промене споро и неефективно, такође имплементација и подизање читавог система због одређене мале измене носи ризик од грешака и проблема. Обзиром да за имплементацију и подизање нове промене често је неопходно угасити и затворити целу апликацију, прекида се самим тим и контакт са корисницима апликације. Уколико су промене мање и једноставније, оне се не пуштају на продукцији одмах, већ се гомилају до следеће верзије која ће бити подигнута на продукцији. То значи да нова верзија која се пушта на продукцију може садржати велики број промена одједном. Што је већа разлика између верзија, веће су и шансе да је нешто погрешно имплементирано. Са микросервисима, промена над кодом једног сервиса не утиче на друге сервисе и самим тим не захтева њихово поновно покретање и подизање. Промењени сервис се самостално имплементира, подиже и покреће на продукцији изоловано од остатка система. Самим тим је код промене и сама промена знатно брже доступна на продукцији.
- **Усклађивање тимова у организацији** – микросервиси поред тога што утичу на саме функционалности, код, инфраструктуру система и сличног, такође остављају утицај и на организацију тимова у складу са архитектуром система која се имплементира. За сваки микросервис је предодређен један тим који има власништво над датим микросервисом. Такав тим је задужен за његову имплементацију, подизање, тестирање, покретање и слично.

- **Композитност** (енг. *Composability*) - битан концепт дистрибураних архитектура јесте поновна употребљивост и коришћење, за различите сврхе, делова функционалности пословања. Са микросервисима, даје се могућност коришћења микросервиса за различите потребе у зависности од тражње и захтева. Уколико функционалност која је тражена застарела, лако се функционалност датог сервиса може дорадити и изменити у складу са потребама и захтевима тржишта. Такође, микросервис се може уништити, односно избацити из система уколико тражња за датим сервисом нестане и једино што је преостало од дате функционалности и сервиса јесу трошкови одржавања и покретања на инфраструктури.

Како постоје предности, тако постоје мане и проблеми које микросервисна архитектура може донети систему. Узрок најчешћих проблема и мана јесте комуникација, односно мрежа помоћу које микросервиси међусобно комуницирају. Комуникација између микросервиса путем мреже може да касни, односно одговор на захтев одређеног уређаја може да касни. Због кашњења одговора, проблем који се јавља јесте брзина враћања одговора корисницима на захтеве који шаљу. Проблем кашњења може да варира што додатно може да закомпликује ствари и учини понашање система непредвидивим. Такође, мреже падају, што проузрокује да одређени подаци, пакети и информације буду изгубљене у процесу слања. Монолитни системи су по питању комуникације ефикаснији и сигурнији из разлога што не користе мрежу да би делови њихових функционалности комуницирали међусобно. Све се налази на једном месту. Иако споменуто као предност, коришћење различитих и нових технологија може представљати проблем уколико се користи погрешно. Погрешним руковањем и имплементацијом нове технологије могу се направити додатни непотребни трошкови организацији (Њуман, 2019, стр. 6-7).

Након описа карактеристичних, најчешће примењиваних и спомињаних архитектуралних стилова у развоју софтвера, у наставку фокус је на два архитектурална стила и њихову трансформацију: монолитну и микросервисну архитектуру.

3. Проблеми, изазови и начини миграције у систем са микросервисном архитектуром

Архитектуралним стиловама дат је увид у различите алтернативе којим бизниси приступају приликом пројектовања архитектуре софтверског система и њиховог пословања. Стављено је до знања значај плана приликом упуштања у реализацију изабраног стила архитектуре софтверског система. Припремање детаљног плана и програма, успостављање структуре и дефинисање тимова, одабир алата и технологија и друге, представљају неопходне процедуре које компаније примењују како би изабрани стил архитектуре софтверског система био на најбољи могући начин реализован. Међутим, са сваким архитектуралним стилем и процесом реализације долазе потенцијални проблеми и изазови који утичу на, како сам софтверски систем, тако и на пословање самог бизниса. Током описа разлике кључних појмова архитектуре софтверског система, акценат је стављен на појмове архитектуралних и софтверских патерна. Разлог истицања ових појмова јесте због њиховог значаја приликом имплементације софтверског система. Проблеми, недостаци и грешке су неизбежне приликом реализације софтверског система. Патерни помажу тако што пружају решења за управљање и сузбијање проблема на које софтверски систем наилази. На тај начин архитектурални патерни решавају проблеме реализације архитектуре софтверског система, док софтверски патерни помажу приликом недостатака који се дешавају током реализације функционалности и логике софтверског система.

Циљ мастер рада је приказати такав процес реализације једног конкретног архитектуралног стила – микросервисне архитектуре. Полазећи од стандарне, опште и уобичајене архитектуре софтверског система (монолитне архитектуре), у мастер раду се даје опис процеса трансформације монолитне архитектуре у систем микросервисне архитектуре. У наставку је дат увид у неопходне ствари које треба разматрати приликом имплементације микросервисне архитектуре, проблеме и недостатке који се јављају са микросервисном архитектуром као и различити видови миграционих патерна који служе за конкретну трансформацију монолитне апликације у систем са микросервисном архитектуром.

Током приче о имплементацији микросервиса, у мастер раду се често као подсетник наводи важност упостављања доброг плана и јасних циљева који се желе постићи микросервисном архитектуром. Идеја мастер рада јесте приказати трансформацију монолитних апликација у систем са микросервисима, са свим предностима и недостацима који долазе са таквом трансформацијом. Међутим, мастер радом није циљ да се микросервисна архитектура успостави као најбоља и најидеалнија опција имплементације архитектуре софтверског система. Микросервиси не требају да буду нужно циљ пословања компаније. Одлука о присвајању микросервисне архитектуре треба да буде детаљно испланирана, пажљиво протумачена, рационално покренута и донешена. Присвајање микросервисне архитектуре не означава нужно „победу“ пословања и испуњавање визије компаније. Целокупан процес размишљања о увођењу микросервисне архитектуре би требао започети онда када софтверски систем, са својом тренутном архитектуралном сликом, није у могућности да реализује и постигне циљеве и захтеве пословања. Присвајање микросервисне архитектуре из разлога што друге компаније своје пословање тако реализују је замка у коју се лако може упасти и тако проузроковати проблеме. Проблем који може настати током имплементације микросервисне архитектуре, без претходно добро успостављеног плана и циља, јесте да запослени који раде на таквој трансформацији у одређеном тренутку забораве почетан разлог саме трансформације. Разлог такве ситуације јесте да бенефит од имплементације микросервисне архитектуре се не може видети одмах, већ мора проћи одређени период да би се могао приметити напредак (Њуман, 2019, стр. 33-34). Током приче о монолитним архитектурама, било је речи о модуларној монолитној архитектури и напоменуто је да прелазак са такве монолитне структуре на архитектуру микросервиса не мора да означава нужно оптималнији, бољи и ефективнији избор. У одређеним ситуацијама, током реализације новог захтева тржишта, јефтиније може да буде одржавање постојеће архитектуре него имплементација нове - микросервисне.

Према Њуману (2019, стр 35), следећа питања могу да буду довољна да пониште одлуку о усвајању микросервисне архитектуре и убеди организацију, уколико немају прецизан одговор, да одустану:

- Шта желите постићи микросервисном архитектуром?

Одговор на ово питање би требало да буде детаљан списак резултата и циљева који су усклађени са оним што пословање треба да постигне, притом да описују корист крајњим корисницима система.

- Да ли сте разматрали алтернативе коришћењу микросервиса?

Одговор би садржао списак алтернатива које су потенцијалне опције за имплементацију функционалности коју микросервиси треба да имплементирају и разлози зашто је на крају донесена одлука да опција буде микросервисна архитектура.

- Како ћете знати да ли трансформација функционише?

Компанија мора пратити процес и утврдити како ће знати да ли иде у правом смеру ка реализацији жељеног циља и постизању тражених резултата.

Утврђивањем плана и програма се такође требају адресирати различити проблеми и изазови који долазе усвајањем микросервисне архитектуре. Проблеми и изазови се могу наћи у различитим стадијумима имплементације микросервисне архитектуре софтверског система, од проблема који се тичу алата, дефинисања домена сервиса, писања програмског кода и сличног, све до изазова организације тимова, распоређивања улога и обавеза међу члановима тима. Тиме, сви проблеми и изазови који се јављају током трансформације монолитне апликације у систем са микросервисном архитектуром се могу сврстати у две групе – техничких и организационих изазова (Kalske, Mäkitalo & Mikkonen, 2018).

3.1. Изазови и проблеми миграције

Технички изазови

Међу најважнијим техничким изазовима приликом имплементације микросервисне архитектуре тичу се поделе домена пословања на различите микросервисе. Обзиром да процес трансформације почиње од монолитне апликације, организација је већ упозната са функционалностима које апликација поседује и логиком која их реализује

(Kalske, Mäkitalo & Mikkonen, 2018). Као важан концепт, приликом дефинисања граница микросервиса и расподеле функционалности који ће микросервиси имати, јавља се концепт *domain driven* дизајна (DDD) (Њуман, 2019, стр. 56). Претходно, приликом дефинисања доменско партиционисаних архитектура споменуте су карактеристике овог концепта дизајна. *Domain driven* дизајн јесте приступ развоју софтверских система који је базиран на програмирању модела домена који поседује довољно знања о процесима и функционалностима пословања и рукује и има потпуно власништво над њим (Fowler, 2020). Одлична почетна тачка приликом креирања микросервиса јесте раздвајање и дефинисање пословања путем ограничених контекста (енг. *bounded contexts*). Сваки ограничени контекст представља један део домена пословања бизниса са добро дефинисаним границама и правилима које се тичу датог домена. Сваки од ограничених контекста представља такође потенцијалну јединицу декомпозиције на више мање различитих делова који ће имплементирати део датог домена. Међутим, целокупан процес разлагања домена пословања може бити изазован и проблематичан. Узрок је граница до које се домен пословања разбија, односно до колико ситних детаља се домен пословања треба и може разбити. Да систем не би непотребно био уситњен на мање делове који могу функционисати заједно, тимови програмера константо дискутују архитектуру система, раде поновно разлагање и груписање домена и тиме покушавају да обликују продуктиван и ефективан систем са микросервисима. Поновно разлагање, груписање, модификовање домена су стандардни процеси којима се тимови програмера користе када су у питању архитектуре вођене доменима пословања. Тиме, велика је вероватноћа да систем неће први пут имати све домене (заједно са микросервисима) прецизно дефинисане (Њуман, 2019, стр. 56-57). Приликом разбијања монолитне апликације у микросервисе важно је да пројектовањем микросервиса се не утиче на рад друих домена функционалности апликације. Због тога је претходни рад на монолитној апликацији важан јер тим програмера је упознат са функционалностима и њиховом међусобном повезаношћу. Можда најбољи приступ ка креирању микросервиса и разбијању монолита јесте да се са разбијањем крене од најједноставнијих и најочигледнијих домена. Након што организација стекне довољно знања и искуства о микросервисима, може кренути са креирањем детаљнијих микросервиса и са разбијањем далеко сложенијих функционалности и процеса. Са све већим бројем микросервиса, терет комуникације путем мреже постаје већи. Обзиром да микрсервиси међусобно

комуницирају путем API захтева, односно слањем HTTP позива, вероватноћа за кашњењем одговора на сваки послат захтев постаје већи због великог бој микросервиса који међусобно комуницирају. Дељењем сваког делића функционалности, микросервиси постају превише детаљни тако да за реализацију било какве потребе, захтева или функционалности, неопходно је послати позив ка другом, такође детаљном, микросервису (Kalske, Mäkitalo & Mikkonen, 2018). Претходно описан проблем је још један разлог зашто је поновно дефинисање, разлагање и дискутовање о архитектури софтверског система важно јер могу постојати одређени микросервиси који могу бити спојени или се могу додатно разложити.

Поред дефинисања домена и граница микросервиса, такође велики проблем јесте и интеграција између различитих микросервиса (Kalske, Mäkitalo & Mikkonen, 2018). Једна од главних предности микросервисне архитектуре јесте то што омогућава коришћење разлитих алата и технологија приликом имплементације различитих микросервиса. Међутим, са технолошки неутралном микросервисном архитектуром долазе и одређени проблеми који могу настати уколико алати и технологије који се користе нису добро испланирани и на прави начин коришћени. Са сваким алатом и технологијом које се користе, постоје пар ствари које треба промислити пре њиховог коришћења. Увођењем нове технологије се требају избећи, у што већој мери, измене на више различитих страна. Уколико микросервис, новом технологијом, изврши операцију додавања новог поља за ажурирање и слање података, утицај (измене) на страни потрошача, који користи функционалности микросервиса, не би требао да постоји. Микросервина архитектура је пројектована тако да микросервиси међусобно комуницирају путем мреже, слањем API позива. Веома је важно осигурати да API-ји коришћени за комуникацију између микросервиса буду технолошки неутрални. Технологије које захтевају имплементацију специфичних технологија за комуникацију треба избегавати из разлога што, специфицирањем конкретне технологије за комуникацију, се може ограничити одабир технологија за имплементацију других микросервиса. Овим се супроставља предност микросервиса да су технолошки неутрални и да дају могућност за усавршавање знања и вештина кроз коришћење нових различитих технологија. Микросервисе треба пројектовати тако да су потрошачима интуитивни и лаки за коришћење. Испрограмирати и дефинисати микросервис на најбољи могући начин не значи ништа уколико његово коришћење од

стране потрошача кошта и узима доста времена (Њуман, 2015). Такође, микросервисе је неопходно пројектовати тако да са сваком новом променом над микросервисом, потрошачи не буду подложни такође драстичним променама (Kalske, Mäkitalo & Mikkonen, 2018). Скривеност детаља и имплементације микросервиса је кључна ставка имплементације и писања програмског кода, како код микросервисне архитектуре, тако и код других видова софтверских система и апликација. Међутим, са концептом скривене имплементације долази и повећање зависности између потрошача и микросервиса. Односно, са променом над микросервисом, потрошач се мора обавестити да, за коришћење извршене промене, је неопходно извршити одређене промене над кодом који користи функционалности микросервиса. Због концепта енкапсулације, скривености детаља и имплементације, треба избегавати технологије које по својој природи захтевају нарушавање концепта енкапсулације (Њуман, 2015).

Потенцијална опасност приликом комуникације између микросервиса јесте њихова реакција на пад микросервиса. Иако директно не утиче на њихов рад и развој, због особина независности и изолованости, приликом реализације функционалности која захтева комуникацију са палим микросервисом, неопходно је приказати стање, разлог или одређену поруку зашто је немогуће извршити тражени захтев. Тиме, микросервиси требају бити дизајнирани тако да буду толерантни на пад било којег микросервиса са којим комуницирају. У организацијама које одржавају софтверски систем са великим бројем микросервиса и функционалности, неизбежно је да ће микросервиси у једном тренутку бити под великим теретом који може проузроковати њихов пад и прекид у раду. Због тога, треба постојати одређени механизам који рукује падом микросервиса и одржава да цео ток функционалности и рада софтверског система буде конзистентан и непрекидан. Такав механизам се може реализовати путем *circuit breaker* патерна (Kalske, Mäkitalo & Mikkonen, 2018). *Circuit breaker* патерна представља дизајн патерн који се користи као пресретац захтева и одговора да би процесуирао да ли тражени сервис ради (Subhashana, 2021). Патерн рукује грешкама и неуспелим извршавањима тако што пружа резервну опцију која враћа подразумеван одговор уместо да чека на одговор од сервиса. Прати грешке и када број грешака дође до одређеног прага (границе) (енг. *threshold*), тада захтеви неће више бити усмерени ка сервису и неће бити извршени, већ ће патерн уместо тога вратити одређену грешку (Kalske, Mäkitalo & Mikkonen, 2018).

Микросервисна архитектура пружа могућност коришћења разлитих, засебних база података за сваки микросервис којем је неопходна. Међутим, са пружањем слободе управљања и коришћења више различитих база података, долазе и одређени изазови. Више различитих база података значи и теже одржавање и управљање таквим базама обзиром да могу бити различитих типова и слично. Трансакције над базама података је такође теже руковати обзиром на време када се трансакција извршава код сваког микросервиса и њихову неусклађеност. Са тим долази и неконзистентност података између база података уколико се заједнички тип и врста података јавља у више различитих база података које међусобно морају бити усаглашене. Због тога, као решење оваквог проблема јавља се концепт „евентуалне конзистентности“. Евентуална конзистентност значи да промене извршене над базом података једног микросервиса, неће бити одмах присутне у базама података других, већ после неког времена ће доћи до конзистентности података. Овим, постоји период времена где одређени микросервиси неће имати приступ промењеним подацима које је један од микросервиса изменио својом трансакцијом. Такви микросервиси, који неће имати одмах приступ промењеним подацима, морају бити спремни да врате одређену поруку корисницима у случају недоступности таквих података (Kalske, Mäkitalo & Mikkonen, 2018).

Организациони изазови

Поред наведених техничких изазова, током процеса трансформације монолитне апликације у систем са микросервисном архитектуром, организација се суочава са још једном групом изазова – организационим изазовима.

Међу главним организационим изазовима јесте структура организације током имплементације микросервисне архитектуре. У нади за изградњом доброг софтверског система, организација мора да усагласи своју структуру са структуром софтверског система који се имплементира (Kalske, Mäkitalo & Mikkonen, 2018). Током историје, тимови у ИТ организацијама су били распоређени и структурисани око посла који чланови тимова обухватају. Обзиром да је први и стандардни тип архитектуре софтверског система била монолитна архитектура, тимови су направљени на основу

посла који сваки слој апликације обухвата. Тако су постојали тимови за програмирање апликација, управљање и одржавање базе података и слично. Самим тим, *Java* програмер је радио са другим *Java* програмером у тиму за програмирање, тестер је радио са другим тестером у тиму за тестирање (*quality assurance* (QA) тим). Тако би цео ток имплементације функционалности и захтева изгледао овако: Од пословног аналитичара (енг. *business analyst*) се добијају захтеви које је прикупио у разговору са потрошачима. Захтеви се прослеђују тиму програмера који се баве програмирањем и имплементацијом траженог захтева. Након завршене имплементације, направљене промене се шаљу ка тиму за тестирање који се баве различитим видовима тестирања над имплементираним променама. Уколико се пронађе грешка у имплементацији, шаље се назад тиму за програмирање који решава пријављене грешке. Онда када сви тестови прођу, тада се прослеђује тиму за операције који дате промене пуштају на продукцију (Њуман, 2019, стр. 62-63).

Међутим, са новијим архитектуралним стиловима и притупима оваква структура тима се мења. Тестери се сада интегришу са тимовима програмера где постају заједно део процеса имплементације и развојка функционалности софтверског система, а тимови са тестерима постају прошлост. Такође, посао тимова за операције сада постаје такође део тима за програмирање где програмери добијају одговорност над процесом испоруке завршног производа. Многи тимови сада садрже чланове који обаљају послове тима операције, *DevOps* програмере. Овим променама у структури тимова, тимови су постали доста независнији и аутономнији него пре. Преузели су одговорност целокупне испоруке производа, од имплементације до подизања. Обзиром на постојање поделе домена пословања са микросервисном архитектуром, нови структурирани тимови су преумсерили фокус на опсег пословања, обухватајући један опсег и над њим преузели потпуну одговорност и власништво. Сада, фокус тимова више није на техничкој страни и опису посла који обухватају, већ на домен пословања који ће реализовати и имплементирати (Њуман, 2019, стр. 63). Са прошлом структуром тимова, тимови су били специјализовани у области коју су обухватали (нпр. програмирање), међутим за испоруку функционалности и промена зависили су од рада и времена других тимова. Тако тимови за програмирање би завршили део везан за имплементацију, међутим морали би да чекају тимове за тестирање и операције за даљу испоруку и проверу. Када су тимови организовани према домену пословања

(микросервисима), тимови садрже целокупну одговорност над испоруком функционалности. Не чекају друге тимове, већ цео процес тестирања, имплементације и испоруке се обавља унутар једног тима, где сви заједно константно раде да производ буде испоручен на време и у складу са захтевима. Такође, овакви тимови не морају да чекају на одобрење и дозволу других. Обзиром да су у потпуности одговорни и имају потпуно власништво над микросервисом, сами одлучују када ће доћи до имплементације и испоруке. Самим тим што је тим одговоран за микросервис, одговорности над грешкама микросервиса је такође на истом том тиму. Пребацивање кривице са једног на други тим више не постоји зато што је један тим одговоран за целокупан ток развоја датог микросервиса. Овим се повећава одговорност и брзина решавања проблема обзиром да се зна где је проблем настао и ко има довољно знања да дати проблем реши (Kalske, Mäkitalo & Mikkonen, 2018).

3.2. Патерни миграције

Претходно у мастер раду истражени су и објашњени различити концепти и појмови који су неопходни за разумевање и примену како би процес трансформације монолитне апликације у систем са микросервисном архитектуром био успешан. Како дефинисати микросервисе и поделити домене пословања по дефинисаним микросервисима, као и шта је неопходно узети у обзир пре почетка трансформације, представљају главне појмове и концепте који су претходно били обрађени. Такође, у раду су истакнути чести проблеми и изазови који прате усвајање микросервисне архитектуре, као и начини за њихово превазилажење и решавање. Наставкак мастер рада је оријентисан на конкретне процесе и методе миграције који помажу при усвајању микросервисне архитектуре.

Пре почетка самих миграција и утврђивања места на којим миграције треба да се изврше, неопходно је установити и водити се одређеним смерницама приликом реализације миграционих патерна, односно током разбијања монолитне апликације. Кроз мастер рад микросервисима се често придружује појам *loosely coupling*-а, односно принцип у којем микросервиси међусобно нису зависни, понашају и опстају самостално у софтверском систему. Међутим, нови појам, и те како важан приликом обликовања микросервиса и разбијања монолитне апликације јесте висока кохезија (енг. *high cohesion*). Приликом пројектовања софтверског система, жељена пројекција система би била да релевантно понашање (функционалности) система буду што ближе заједно, док нерелевантно што даље. Разлог оваквог начина пројектовања јесте да, уколико дође до промене једног понашања система (функционалности), имплементација дате промене понашања би требало да се изврши на једном месту тако да се иста та промена може испоручити на продукцију у што краћем временском периоду. Када за испоруку идентификоване промене је неопходно променити понашање на више различитих места, процес испоруке и имплементације је далеко спорији и тежи. Због наведених разлога микросервисе треба пројектовати тако да су међусобно *loosely coupled* и исто тако високо кохезивни, односно монолитну апликацију треба разбијати на такве делове који ће омогућити да микросервиси поседују наведене особине. Врло важна смерница приликом пројектовања микросервисне архитектуре и разбијања монолитне апликације је да приступ

разбијању буде инкременталан. Инкременталним приступом се свака функционалност монолита посматра засебно, при чему се сваки део монолитне апликације постепено одваја и пројектује путем микросервиса. Овим приступом се омогућава да се сваки део посматра индивидуално, што поједностављује процес трансформације и смањује ризике од појављивања одређених грешака и проблема (Њуман, 2015). Наведене смернице су круцијалне приликом пројектовања микросервиса током процеса разбијања монолитних апликација. Међутим, пре самог процеса разбијања, неопходно је утврдити разлоге због којих долази као и места над којим је процес разбијања неопходан.

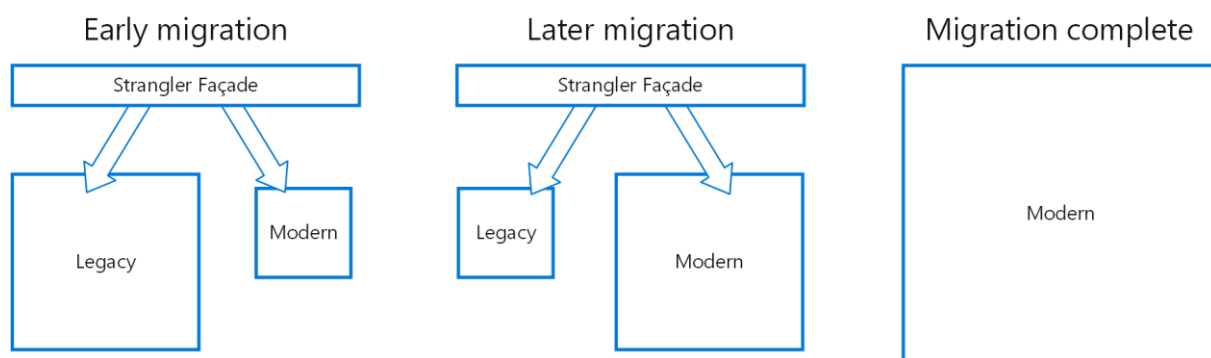
Приликом разбијања монолитне апликације, без обзира на разлог, треба се водити приступом да се одвајањем одређеног дела кода и прављењем микросервиса извучи највећа могућа корист. Раздвајање монолитне апликације ради само креирања микросервисне архитектуре без конкретног циља или користи треба избегавати. Разлози трансформације и пројектовања микросервиса могу бити различити. Уколико организација предвиђа велике промене које захтевају инкременталне промене над једним делом функционалности, одвајањем те функционалности у микросервис се знатно поједностављује поступак имплементације и испоруке такве промене на продукцији. У зависности од структуре тимова организације, исплативије је уколико се код подели тако да је тим задужен за испоруку функционалности која им је предодређена, не бавећи се функционалностима које им нису у опису посла. Такође добар разлог разбијања може бити и безбедност осетљивих информација функционалности. Софтверски системи банака садрже финансијске и личне податке корисника. Издвајањем сервиса који се баве регулацијом, управљањем и одржавањем података и прављењем засебне безбедносне шеме, оваква функционалност се штити од других, незаштићених, функционалности које је могу угрозити. Имплементација нове технологије може бити такође чест и исплатив разлог увођења микросервиса. Наиме, тим задужен за одређену функционалност има жељу за коришћењем нове технологије која ће поспешити перформансе и рад функционалност. Због тога, издвајањем дела кода функционалности у микросервис, жељена технологија се може тестирати над таквим микросервисом и утврдити како ради и доприноси раду и резултатима софтверског ситема (Њуман, 2015).

У наставку мастер рада су описани различити миграциони патерни и њихова примена у многобројним ситуацијама приликом усвајања микросервисне архитектуре. Пре описа миграционих патерна, прво је потребно размотрити да ли је за потребе реализације миграције планирана измена постојећег монолита, уколико постоји таква могућност. Са могућношћу промене постојећег монолита долази флексибилност у одабиру миграционих патерна. Међутим, у одређеним ситуацијама могу постојати одређена ограничења која ће лимитирати опсег могућих патерна. Таква ситуација се може јавити када део софтверског система, који је у власништу компаније са треће стране (добављача), нема доступан изворни код или је развијен коришћењем технологије за коју компанија нема довољно знања, вештина и искуства. Ако се приступи измени постојећег изворног кода, могу постојати различити фактори који ће ометати такву намеру. Стање тренутног монолита може бити толико лоше да трошкови промене могу бити веома високи. Због тога би боља опција била да се процес миграције и трансформација започне изнова. Друга потенцијална ситуација јесте да се над монолитом ради од стране других програмера при чему би миграција утицала на њихов рад. Уколико оваква ограничења се не односе на монолит и постоји могућност измене постојећег кода, када се започне процес миграције функционалности у микросервисе није увек најјасније шта да се ради са постојећим кодом. Нејасноће углавном настају око питања да ли миграну функционалност треба постићи копирањем већ постојећег кода или имплементирањем, односно писањем новог кода. Уколико је постојећи код монолита јасан и прецизно дефинисан, копирањем кода се знатно уштеди време процеса миграције. Кључна ствар приликом копирања јесте да копирани код који се испоручи путем микросервиса не треба уклањати из монолита. Чувањем мигриране функционалности на монолиту даје могућност повратка и преусмеравања захтева ка тој функционалности у случају грешака и проблема на микросервису или покретања обе функционалности паралелно. Такође, то не ограничава брисање функционалности са монолитне апликације уколико је рад функционалности на микросервису задовољавајући. Проблем са копирањем постојећег кода лежи у чињеници да код монолита није пројектован око домена пословања. Претходно је објашњена повезаност микросервиса са *domain driven* дизајном софтверског система, са нагласком да сваки микросервис треба бити обликован на основу ограниченог контекста. Због тога, процес чистог копирања кода може представљати проблем, јер је код обликован на основу слоја (слојевита архитектура) у

ком се налази. Из тог разлога, приступ поновне имплементације би био исплативији и ефективнији. Међутим, на основу свега претходног, можда најбољи приступ миграцији и разбијању монолита би било инкрементално раздвајање функционалности монолита. Инкременталним одвајањем мањих делова функционалности и њиховом поновном имплементацијом смањује се ризик од грешака и повећава брзина испоруке производа. Као резултат, испорука производа ће бити чешћа, обзиром на постојање више мањих делова функционалности, и свака испоручена функционалност ће брже долазити до потрошача спремна за коришћење (Њуман, 2019, стр. 76-78).

Патерн – *Strangler Fig Application*

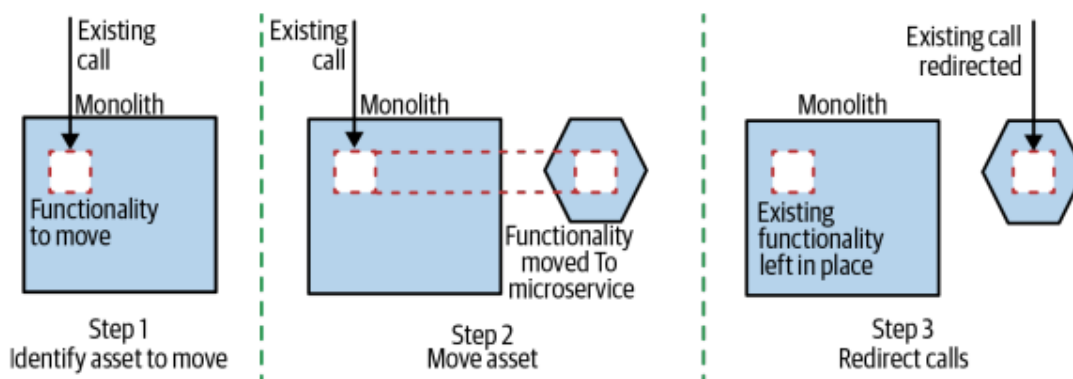
Техника која се често користи приликом поновног писања и програмирања система јесте *strangler fig application* (Њуман, 2019, стр. 79). Путовањем у Аустралију, Мартин Фовлер (енг. Martin Fowler) је први приметио и дефинисао овај патерн, посматрајући понашање врсте биљака под називом – давител смокве (енг. *strangler figs*). Давитељи смокве свој животни циклус започињу високо, на крошњама дрвећа где се њихово семе одлаже. Постепено се ове биљке спуштају све док се не укорене на тлу земље. Током времена, давител смокве наставља да расте формирајући лепе и фасцинантне облике, убијајући тиме дрво домаћина. Начин живота давител смокве је инспирисало Фовлера за опис начина поновног преписивања и изградње система. Наиме, овај метод преписивања система подразумева постепено креирање и развој новог око старог система. Временом, нови систем постаје јачи и стабилнији, преузимајући све функционалности старог система, чиме стари систем постаје непотребан и тиме гаси се. Најважнији разлог за разматрање примене *strangler fig application* патерна јесте смањени ризик од настанка грешака. Постепеним развојем новог система, корисницима се обезбеђује континуирана вредност и фреквентна испорука софтвера у деловима. Овај приступ омогућава пажљивије праћење целокупног процеса развоја новог система, јер постоји могућност истраживања и праћења свих, па чак и најмањих, делова система (Фовлер, 2004). *Strangler fig* патерн се често користи приликом миграције монолитне апликације у нови систем са такође монолитном архитектуром. Међутим, може се користити и за мигрирање ка микросервисима. (Њуман, 2019, стр. 79). Целокупан процес транзиције монлитних архитектура је дат на следећој слици:



Слика 12. *Strangler fig* патерн миграције на примеру монолитних апликација [29]

Међутим, обзиром да је циљ мастер рада трансформација монолита у систем са микросервисном архитектуром, приказаће се начин рада *strangler fig* патерна са микросервисима. Као што је спомињано раније, оваква транзиција може подразумевати копирање кода или поновну имплементацију и програмирање тражене функционалности. Према Њуману (2019, стр. 79-80), имплементација *strangler fig* патерна путем микросервиса се заснива на три корака:

1. Идентификација делова система који се желе мигрирати. Неопходно је узети у обзир све факторе који могу утицати на миграцију и направити детаљан план и програм. Разумети које се ствари ризикују а које стичу реализацијом идентификованих делова система.
2. Имплементација идентификованих делова система у микросервисе.
3. Када је имплементација микросервиса готова и спремна за коришћење, може се приступити преусмеравању свих позива са старог, монолитног система на нови микросервис. Цео процес транзиције функционалности на микросервисе није потпун док се позиви не преусмере на нове микросервисе, чак иако су микросервиси пуштени и подигнути на продукцији. Овај приступ програмерима пружа већу слободу и време за имплементацију функционалности, чиме се нови микросервис чини стабилнијим и боље функционалним. Слобода програмера приликом имплементације је кључна из разлога што могу тестирати различите ствари и утврдити промене без притиска да корисници не добију погрешну имплементацију. Оног тренутка када је имплементација коначно готова, приступа се поступку преусмеравања позива на готову имплементацију функционалности, односно на микросервису.



Слика 13. Процес имплементације *Strangler fig* application патерна [24]

Поред особине инкременталне миграције и развијања функционалности у нов систем (микросервисе), веома битна ставка *strangler fig* патерна јесте и могућност лаког повратка на стару верзију система, односно поништавања промене уколико је потребно. Грешке се дешавају, да ли током програмирања или само испоруке, због чега је кључно да постоји могућност повратка на стару верзију система уколико дође до изненадних проблема и грешака (Њуман, 2019, стр. 80).

Са *strangler fig* патерном дозвољава се померање функционалности на нове микросервисе без потребе за мењањем кода постојећег система. Претходно су споменути недостаци који се јављају када миграциони патерни условљавају одређену промену кода постојећег система како би функционисали. Један од недостатака јесте био када промене постојећег система утичу на рад других који раде над датим системом. Код *strangler fig* патерна не постоји такав проблем обзиром да се читава миграција може извршити без промене постојећег система (Њуман, 2019, стр. 81).

Приликом преусмеравања позива, врло је важно разматрати и природу позива који се шаљу постојећем систему. Протоколи се међусобно разликују и због тога је важно утврдити начин како ће се позиви преусмерити са старог, монолитног система, на сада нови микросервисни систем обзиром да сваки од протокола има своје неке стандарде и процедуре. На примеру HTTP захтева, најбољи начин преусмеравања позива са монолита на нове микросервисе јесте коришћењем HTTP proxy-а. Proxy служи као посредник између система и пошиљаоца захтева (корисник, веб апликација) који има задатак да обрађује захтеве и прослеђује их на тражену дестинацију. Приликом

пројектовања RESTful апликација које се ослањају на принципе HTTP протокола, као додатан корак имплементације *strangler fig* патерна се може додати креирање и увођење HTTP proxy-а. Такав корак се уводи пре почетка имплементације микросервиса и преусмеравања позива. Код појединих система постоји могућност да проху већ постоји што онда поједностављује принцип јер се корак увођења проху-а не мора имплементирати. Начин како ће се проху имплементирати остаје на тиму програмера и саме организације да одлуче (Њуман, 2019, стр. 83-86).

Патерн – *UI Composition*

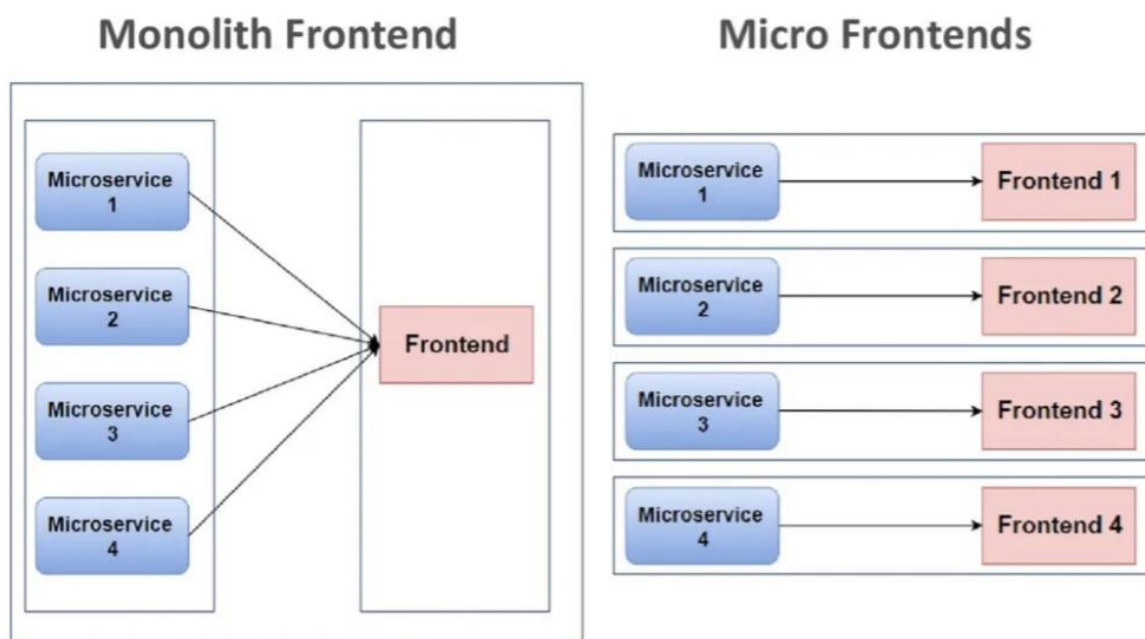
Када се говори о микросервисима најчешће се повезују са серверском страном, односно са *backend* апликацијама. Међутим, постоје патерни миграције који се могу применити и на UI, односно *frontend* апликације. Клијентска страна постаје све оптерећенија и преузима све више одговорности, што концепт „мршаваг клијента“ чини све ређим, док концепт „дебелог клијента“ постаје све чешћи, означавајући апликацију са превеликим бројем одговорности и функционалности на једном месту. Уместо проналажења начина да се растерете апликације на *frontend*-у, све више оквира за развој *frontend* апликација се пројектује како би апликације учиниле бржим и динамичнијим. Новији оквири поспешују рад и перформансе апликација, али питање целокупне одговорности и расподеле на различите сервисе како би растеретили софтверски систем остаје нерешено (Witkowski, 2018). Због тога, постоје различити начини и приступи изградње *frontend* апликација, монолитног типа, у микросервисни систем.

Један од начина јесте извлачењем једне целокупне странице и прављењем апликације од те странице, тако да корисници добију потпуно другачији преглед и осећај када посете страницу са траженом функционалношћу. Самим тим, извлачењем странице, URL странице ће се променити са новим издањем, што условљава промену URL-а код других старих страница који позивају нову издвојену страницу. Издавање читаве странице и њене функционалности може бити напоран и спор начин миграције за софтверске системе који желе бржу и динамичнију испоруку решења корисничких захтева. Због тога, уместо да се имплементира целокупна нова страница, користи се приступ са виџетима, где се имплементира један део странице као независни виџет који

се затим позива и учитава на страници. Дељење виџета према подели рада на серверској страни данас је све ређе због напредовања интернет претраживача, који сада могу да управљају и извршавају слагање комплетне странице. Коришћење виџета има предност у томе што ако један виџета не успе да се прочита (грешка на страни сервера), остали виџети се и даље могу прочитати и функционисати. На тај начин, целокупној страници ће недостајати тај један виџет, односно једна функционалност коју обухвата (Њуман, 2019, стр. 99-100).

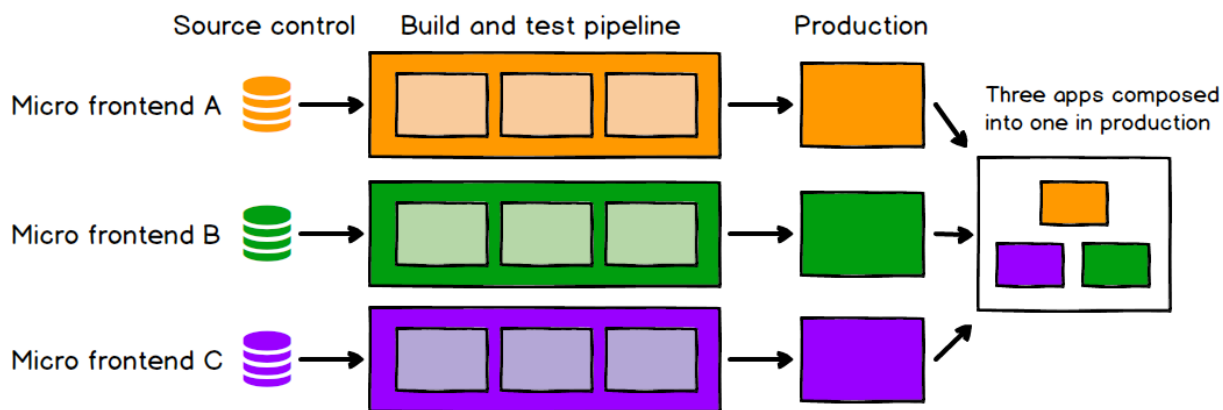
Претходно су приказани начини миграције на микросервисну архитектуру на примеру веб апликација које су сачињене од више веб страница. Међутим, интернет претраживачи и различити оквири за развој веб апликација су се временом развијали где многи кориснички интерфејси данас користе посебан вид веб апликација – *single page* апликација. *Single page* апликација представља веб апликацију која динамички мења и преписује садржај на једној, јединој, веб страници са новим подацима које добија од веб сервера. Уместо да интернет претраживач учитава више различитих веб страна, са *single page* апликацијама се само учитава захтевани садржај који се приказује на једној страни (Lawson, 2024). Са *single page* апликацијама долази и нови начин миграција на систем са микросервисном архитектуром. Обзиром да се целокупан кориснички интерфејс испоручује путем једне веб странице, претходни начин миграције извлачењем веб страница није у оптицају. Међутим, одређена форма приступа са креирањем виџета може бити једна од алтернатива. Такав приступ водио је ка креирању новог правца имплементације микросервисне архитектуре на *frontend*-у под називом - *Micro Frontends* (Њуман, 2019, стр. 103-104). Према Кам Џаксону (енг. *Cam Jackson*) (2019) концепт ***Micro Frontends*** представља „архитектурални стил у којем независне, испоручиве, *frontend* апликације се састављају у већи целокупни систем“. Бенефити коришћења микро *frontend*-ова јесу исти као и са коришћењем микросервиса а то су:

- Мање, независне и више кохезивне базе кода
- Скалабилност организације са више аутономних тимова
- Могућност инкременталне надоградње, ажурирања или преписивања делова *frontend*-а



Слика 14. Приказ разлике монолитног и микро *frontend* типа апликације [30]

Инкременталним ажурирањем и надоградњом се тимовима даје могућност разлагања старе апликације део по део, притом остављајући простор за испоруку нових функционалности корисницима. На тај начин, када постоји одређена грешка, проблем који се треба решити или нова функционалност која се треба додати, сваки од тражених микро *frontend*-ова се може независно ажурирати без притиска да се целокупан систем мора зауставити и све заједно ажурирати и покренути. Сваки микро *frontend* има своју изоловану базу кода која је заслужна за имплементацију тражене функционалности. Тиме, целокупан изворни код монолита се декомпунује у мале независне базе кодова који се могу изоловано посматрати и имплементирати. За разлику од монолитног типа *frontend* апликације која се мора испоручити заједно у једном тренутку, са микро *frontend*-овима се сваки испоручује независно, без утицаја од стране других микро *frontend*-ова, у неком свом тренутку (Џаксон, 2019). Изолована испорука микро *frontend*-а је приказана на следећој слици:



Слика 15. Независна испорука микро *frontend*-а [32]

Са изолованим функционалностима спакованим у микро *frontend*-ове, настају аутономни тимови задужени за њихово управљање, одржавање и имплементацију. Са монолитним апликација, тимови су били задужени за слојеве које монолитна *frontend* апликација има. На пример, постојао је тим задужен за стилизовање, дефинисање форми, имплементацију валидације форме и долазећих/одлазећих података и слично. Таква подела тимова се назива хоризонтална подела, са хоризонталним прецесањем апликације и дељењем у зависности од слојева који се добију пресецањем. Са микро *frontend*-овима, долази до вертикалне поделе, односно поделе по функционалности коју сваки микро *fronted* обухвата (Џаксон, 2019).

Патерн – *Branch by Abstraction*

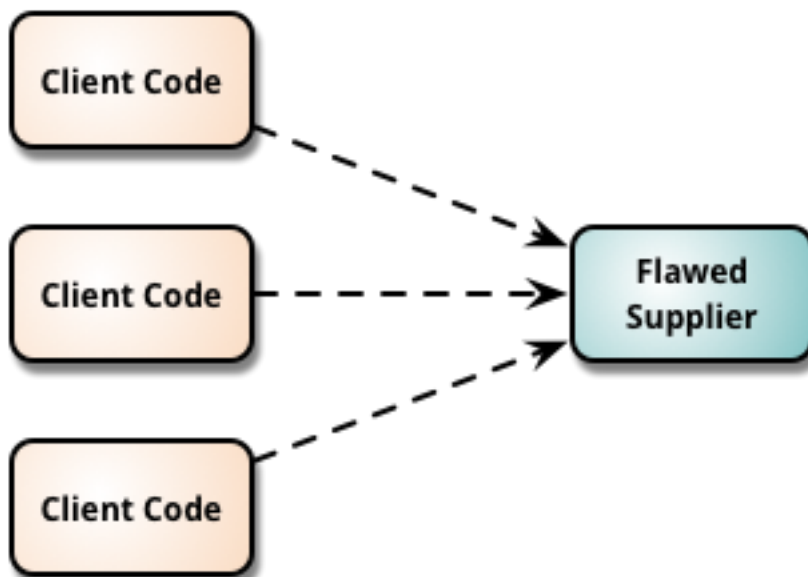
Да би *strangler fig application* патерн био успешан, неопходно је да функционалност која се мигрира и извлачи буде доступна за позиве како би се ти позиви могли пресрести ван монолита. На тај начин, функционалност се може лако мигрирати у нов микросервис без мењања постојећег кода, јер једина промена која се извршава јесте преусмеравање позива ка новом микросервису. Међутим, уколико се тражена функционалност, која се жели мигрирати, налази дубље у систему и није доступна путем позива, онда коришћење *strangler fig application* патерна не би било могуће. У том случају, мигрирање такве функционалности ће захтевати и измене постојећег система и кода, што додатно отежава примену овог патерна. Мењање постојећег кода утиче на друге програмере који раде над истом сегменту кода, што додатно може

закомпликовати ствари и постати место потенцијалних проблема и грешака. Са једне стране, постоји јака жеља за инкременталним приступом и променама функционалности или сегмента кода, али са друге стране се не жели утицати на рад и продуктивност других програмера. Као решење овог проблема, програмери често приступају изради додатне гране кода над којом могу имплементирати своје промене. Нова грана настаје на основу изворног кода, односно гране која садржи базу изворног кода, и након завршене имплементације, промене се спајају са изворном базом кода, спајањем нове гране са основном граном. Међутим, проблем са овим приступом је тај што спајањем две гране могу настати конфликти у коду који се морају усагласити између тимова програмера, што значајно продужује поступак испоруке измене. Такође, основна грана не престаје са имплементацијом и развијањем, што нову грану, која је настала од основне, чини застарелом јер поседује старију верзију изворног кода од које је настала. Самим тим, што дуже нова грана постоји и одржава се, то су већи проблеми који долазе са њом. Због наведених проблема, програмери желе инкременталну изградњу и измене базе кода, уз минималан утицај на рад других програмера. **Branch by abstraction** патерн може се користити за такве потребе. Овај приступ функционише тако што се мења постојећа база кода како би омогућила да имплементације безбедно коегзистирају у истој верзији кода, без проузроковања било каквог прекида или ремећења рада других програмера (Њуман, 2019, стр. 104-105).

Према Њуману (2019, стр. 105), *Branch by abstraction* се састоји од пет корака:

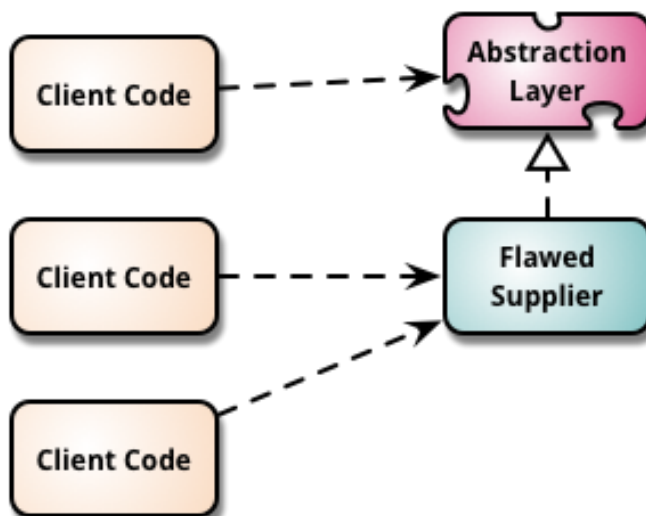
1. Креирање апстракције за функционалност која се жели заменити
2. Преусмеравање клијента са постојеће функционалности на нову креирану апстракцију
3. Креирање нове имплементације апстракције са прерађеном функционалношћу. Са микросервисима, нова имплементација позива нови креирани микросервис.
4. Замена имплементације код апстракције тако да користи нову имплементацију
5. Чишћење апстракције и брисање старе имплементације

Пре почетка реализације патерна, почетна слика софтверског ситема који се мења имплементацијом патерна изгледала би следеће:



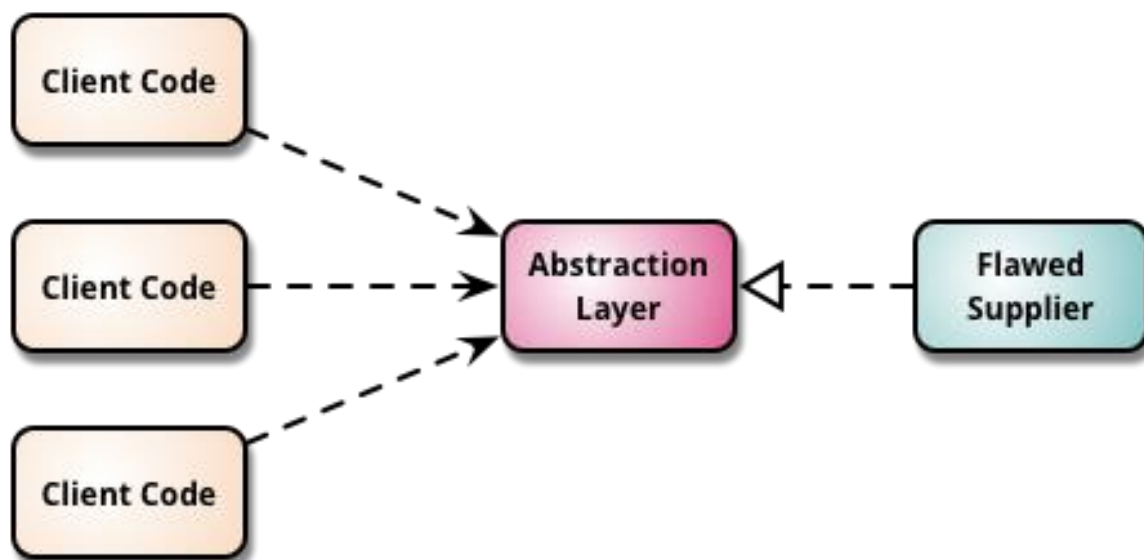
Слика 16. Софтверски систем са функционалношћу која се мења и кодом који га позива [33]

Креирање апстракције, која представља интеракцију између сегмента кода који се мења и позиваоца датог кода, представља први корак овог патерна. Кључна ставка првог корака јесте наћи кодове који представљају поменућу интеракцију и од тога направити апстракцију (Њуман, 2019, стр. 105-106). На следећој слици дат је приказ креиране апстракције и где се треба налазити у односу на претходно приказани систем:



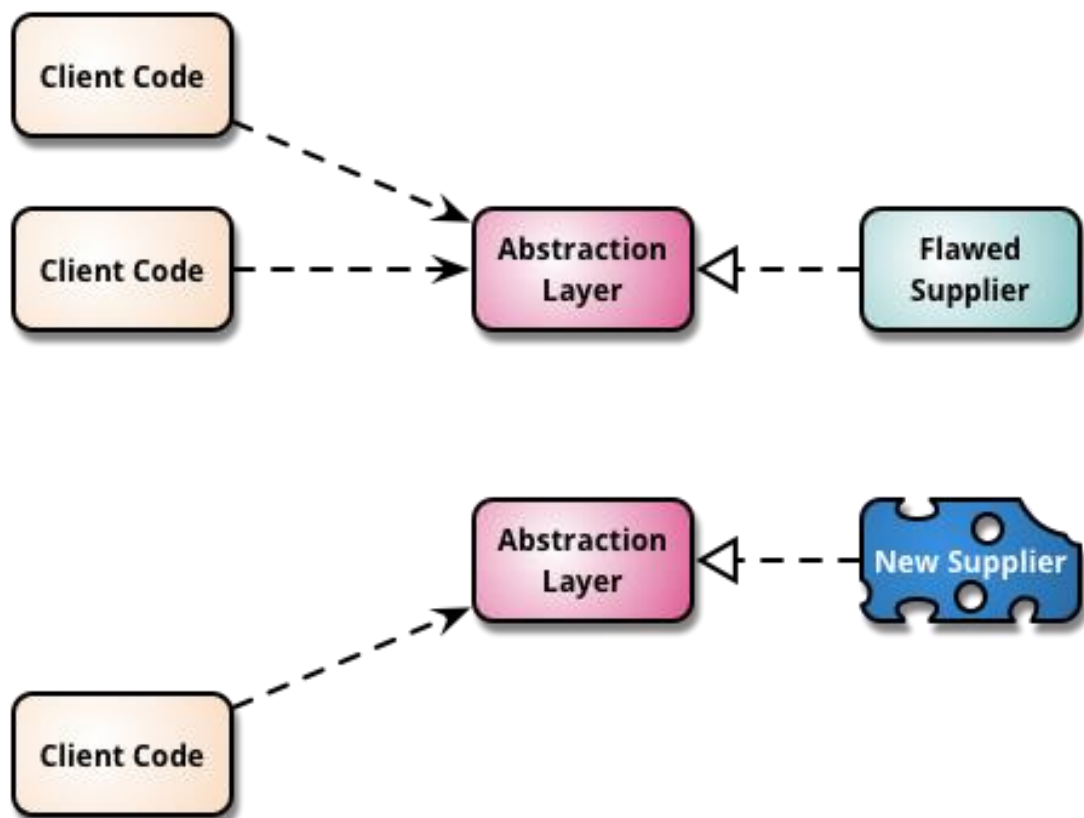
Слика 17. Први корак – креирање апстракције [33]

Са креираном апстракцијом на месту, приступа се преусмеравању сваког клијент кода ка новој креираној апстракцији (Фовлер, 2014). На следећој слици је приказан други корак патерна, односно преусмеравање позиваоца функционалности ка новој креираној апстракцији:



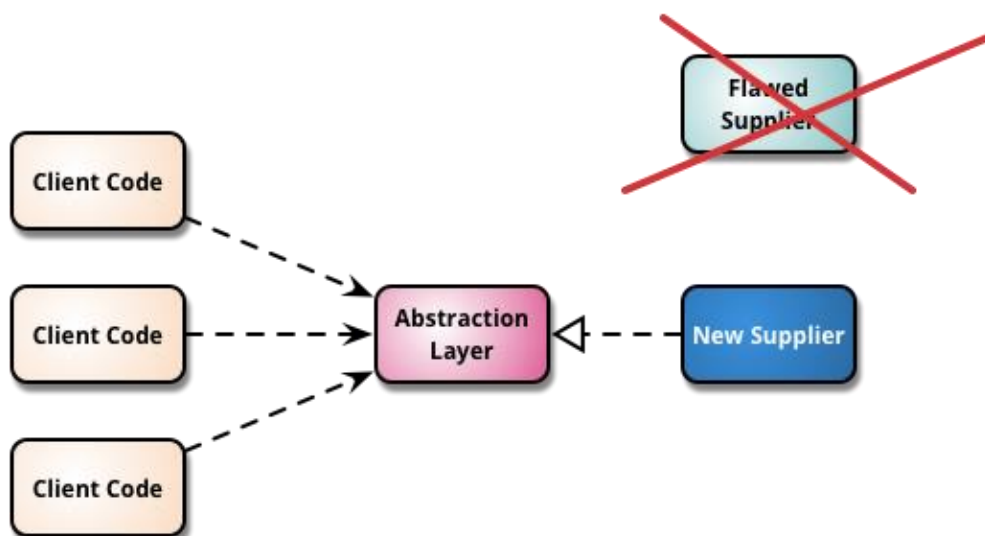
Слика 18. Други корак – преусмеравање клијент кода на нову апстракцију [33]

Након постављања апстракције и преусмеравања свих клијентских кодова функционалности која се мења, приступа се трећем кораку реализације патерна – креирању нове имплементације. Битна ставка приликом креирања нове имплементације јесте да сада постоје две имплементације. Једна, стара, постојећа имплементација и нова, од којих је само једна имплементација активна у систему. Стара имплементација је активна докле год је креирање нове имплементације у развоју. Оног тренутка када је тим програмера задовољан новом имплементацијом, тада се стара имплементација полако гаси и уклања. Током процеса реализације трећег корака патерна, тим програмера може подигнути и испоручити на продукцији нову имплементацију. Као и код *strangler fig* патерна, подизање и испорука нове имплементације не означава нужно да ће се она тад и користити и да ће бити доступна за позиве (Њуман, 2019, стр. 106-107). Приказ креиране нове имплементације, у односу на претходно дефинисан софтверски систем, се налази на следећој слици:



Слика 19. Трећи корак – креирање нове имплементације [33]

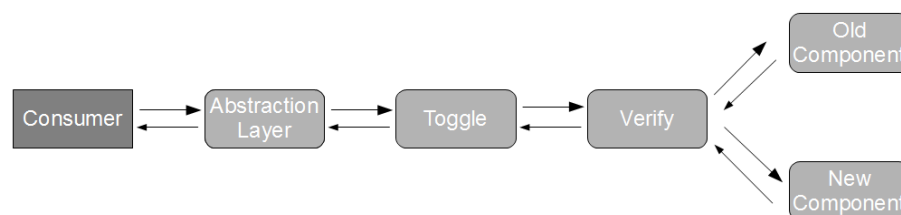
Оног тренутка када је тим програмера сигуран у рад нове имплементације, процес преусмеравања апстракције са старе имплементације на нову може да започне (Фовлер, 2014). На следећој слици је приказана апстракција која сада у потпуности користи нову имплементацију, док се стара гаси:



Слика 20. Четврти корак – преусмеравање на нову имплементацију [33]

Сада када сви клијенти користе функционалности нове имплементације, може се започети са петим, последњим кораком – чишћења и брисања старе имплементације. Такође, са брисањем старе имплементације, апстракција се може уклонити обзиром да њена улога посредника током креирања нове имплементације више није значајна. Постоји могућност да апстракција поспешује и побољшава саму базу кода што даје разлог да је тимови задрже ради бољег и прегледнијег кода. Уколико апстракција представља обичан интерфејс који има веома мали утицај на базу кода, његовим чувањем се ништа не постиже (Њуман, 2019, стр. 110-111).

Могућост враћања назад, на коришћење старе имплементације, уколико нова имплементација не даје тражене резултате или проузрокује одређене проблеме, је кључна. Из тог разлога, може се креирати одређени механизам који ће се бавити преусмеравањем позива на нову или стару имплементацију у зависности од ситуације (Њуман, 2019, стр. 112). Стив Смит (енг. *Steve Smith*) (2013) уводи варијацију *branch by abstraction* патерна тако што, између апстракције и имплементације, додаје два нова посредника. Један од посреника представља прекидач који има задатак да пребаци извршавање захтева на стару имплементацију у случају грешака, на одређено време. Међутим, проблем са постојањем само једног овог посредника је тај што, иако апстракциони слој захтева исто понашање, прекидач не може гарантовати да ће имплементација бити иста. Зато Смит (2013) предлаже увођење новог посредника, са којим ће прекидач комуницирати, који обезбеђује додатну верификацију. Апстракција комуницира са прекидачем који се усмерава ка новом верификационом посреднику. Верификациона имплементација затим позива обе имплементације са истим улазима и тако брзо открива грешке упоређујући им резултате које добија од обе имплементације. Овим начином се смањује време истраживања некомпатибилности старе и нове имплементације, као и време налажења насталог проблема и грешке. На следећој слици је приказана архитектура софтверског система насталог *branch by abstraction* патерном са два додатна посредника:

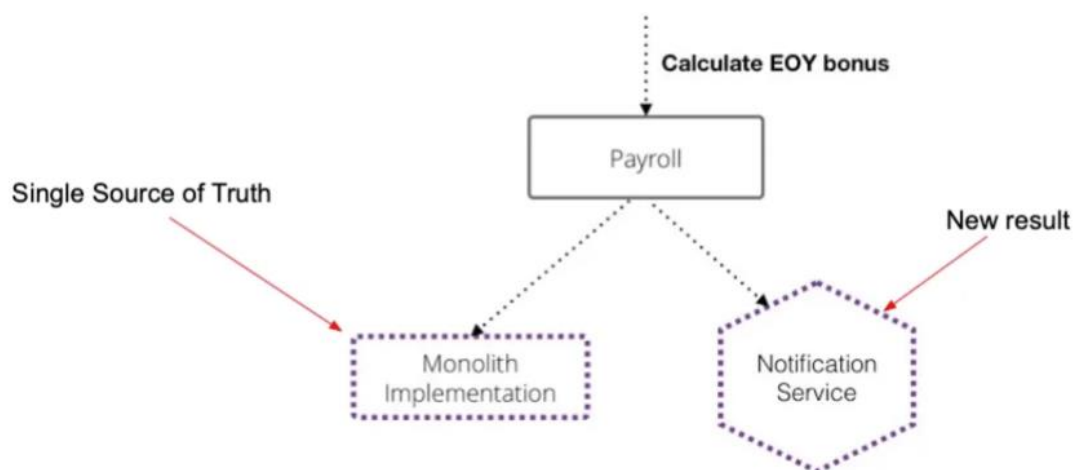


Слика 21. Verify Branch by Abstraction патерн [34]

Патерн – Парарелно извршавање (енг. *Parallel Run*)

Оба патерна, *strangler fig application* и *branch by abstraction*, дозвољавају да стара и нова имплементација функционалности коегзистирају на продукцији у исто време. Углавном, обе ове технике дозвољавају корисницима да извршавају или стару имплементацију која се налази у монолиту или нову имплементацију која се налази у микросервису. Такође, оба патерна садрже одговарајуће механизме који, у случају проблема или грешака насталих у новој имплементацији, могу лако вратити стару имплементацију за извршавање функционалности. Са патерном паралелног извршавања, уместо позивања једне од имплементација (старе или нове), покренуте су и позивају се обе у исто време, дозвољавајући корисницима да упоређују резултат добијен овим имплементацијама. Иако се обе имплементације позивају, једна се сматра валидном и изворном тачних резултата и одговора. Типично, стара имплементација је та која се тако сматра, односно представља „*source of truth*“, док се не утврди да је нова имплементација валидна и да даје тачне и жељене резултате. Техника паралелног извршавања се, поред поређења одговора више имплементација, може користити и за праћење других оперативних параметра као што су брзина извршавања, прекиди у раду имплементације и сличних (Њуман, 2019, стр. 113).

На следећој слици приказан је пример патерна паралелног извршавања:



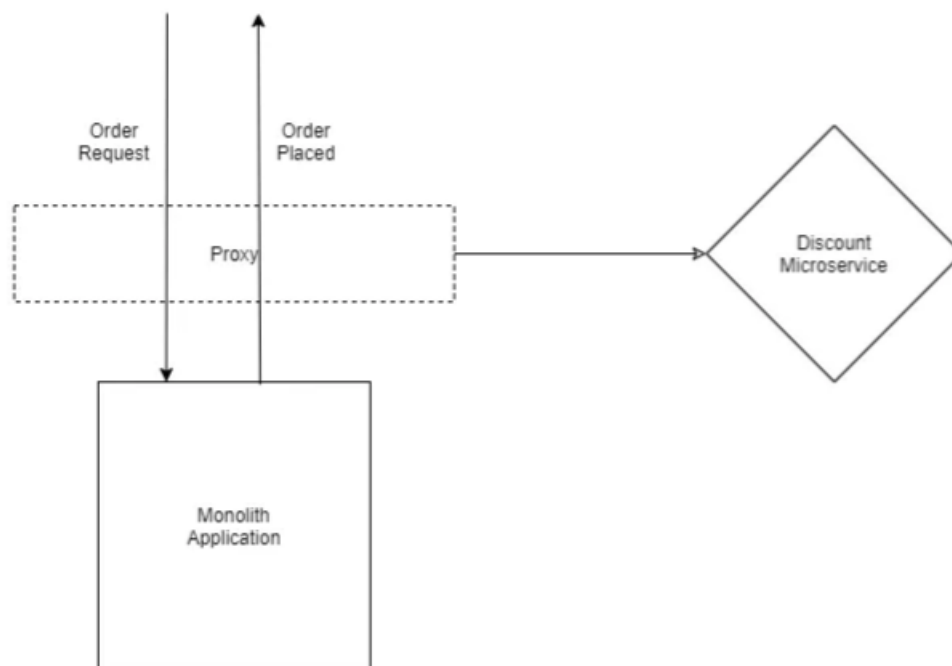
Слика 22. Патерн паралелног извршавања [35]

Парарелним извршавањем две имплементације добијају се резултати које је неопходно упоредити на неки начин како би се утврдила њихова једнакост, односно еквивалентност нове и старе имплементације. Поред једнакости добијених резултата и њихових одговора, врло је важно испратити и оперативне карактеристике имплементације које се тичу брзине одговора, времена слања одговора назад, као и потенцијалних препрека при реализацији и слања одговора назад. Постоје различите технике верификације имплементације које су задужене да претходни критеријуми и услови буду, у што већој мери, испуњени. Једна од техника јесте коришћење *GitHub* библиотеке, *GitHub Scientist*. Ова Ruby библиотека дозвољава да се имплементације покрећу једна поред друге, прикупљајући праве информације од нове имплементације како би помогла у разумевању да ли нова имплементација ради како треба (Њуман, 2019, стр. 117).

Патерн – *Decorating Collaborator*

Постоје ситуације у којим се одређено понашање жели изградити и подићи али без могућности промене самог монолита у којем се такво понашање налази. *Decorating Collaborator* патерн представља решење овакве ситуације. Наиме, овај патерн додаје нову функционалност одређеном делу система без да је тај део система свестан нове функционалности. Помоћу *decorating collaborator*-а, монолитна апликација се пројектује тако да директно шаље захтеве креираном микросервису, без промене самог монолита. За разлику од *strangler fig application* патерна, који чека позив споља да би га преусмерио ка новом микросервису, *decorating collaborator* патерн допушта да позив нормално дође до монолита и настави своју путању. Затим, на основу резултата позива, екстерни микросервис може бити позван путем различитих механизма колаборације (Њуман, 2019, стр. 118).

На следећој слици је приказан пример реализације *decorating collaborator* патерна:



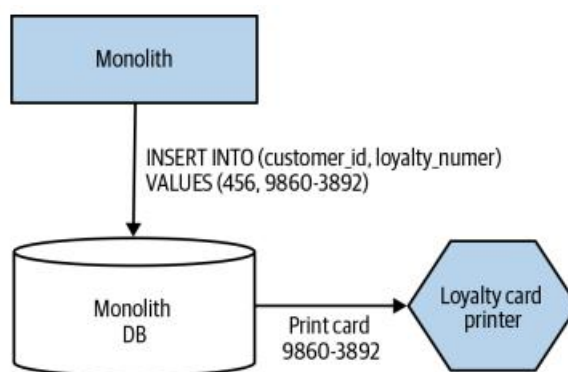
Слика 23. Пример *decorating collaborator* патерна [36]

Са *strangler fig application* патерном, уведен проху је био једноставнији. Његов задатак је био преусмеравање захтева микросервису који имплементира тражену функционалност која је некада била на монолиту. Сада, код *decorating collaborator* патерна, проху захтева сложенију логику и имплементацију. Логика треба бити испрограмирана тако да, на основу одређених услова, позива креирани микросервис и потом враћа одговарајуће одговоре кориснику. Потенцијални изазов такође лежи и у захтевима који доспевају до проху-а и монолита. Сваки захтев треба садржати довољно информација на основу којих се може закључити да ли је неопходан и могућ позив ка новом микросервису (Њуман, 2019, стр. 119).

Према Њуману (2019, стр. 120), овај патерн се може најбоље искористити у ситуацијама када је могуће тражену информацију извући из долазећег захтева или одговора од монолита. Имплементација функционалности постаје комплекснија уколико је већи број информација неопходан како би се захтев ка микросервису реализовао. Због тога, уколико долазећи захтев и одговор од монолита не садрже информације неопходне за прављење микросервиса, онда се треба добро промислити о коришћењу овог патерна.

Патерн – *Change Data Capture*

Уместо покушаја пресретања и покретања акција на основу захтева који се шаљу монолиту, са патерном *change data capture* се реагује на промене у бази података са којом монолит комуницира. Међутим, услов да би овај патерн функционисао јесте да нови имплементирани систем мора бити повезан са базом података монолита. Начин рада овог патерна је такав да реагује на тражену интеракцију између монолита и базе података. У тренутку када се деси тражена интеракција (транзакција), позива се нови микросервис који ће имплементирати одређену логику и функционалност која се тражи на основу дате интеракције са базом података (Њуман, 2019, стр. 120-121). На следећој слици се налази приказ *change data capture* патерна на примеру штампања карте на основу уноса одређених података корисника:



Слика 24. Пример коришћења *change data capture* патерна [24]

Постоје различите технике које се могу користити за имплементацију *change data capture* патерна. Њуман (2019, стр. 121-124) идентификује и описује три такве технике:

- **Тригери базе података** – већина релационих база података дозвољава креирање сопствених тригера који ће извршити одређено понашање у тренутку промене података. Начин на који се тригери имплементирају и користе зависи од система за управљање базом података, међутим за сваки од њих важи да је инсталиран унутар саме базе података. Проблем који може настати са системима који користе тригере јесте када им се читав систем заснива на њима. У једном тренутку се не може препознати како систем заправо ради, што може

довести до потешкоћа у одржавању и управљању тригера.

- **Читање *log*-ова трансакција** – у већини база података постоје *log*-ови свих трансакција. *Log* представља одређени дневник који исписује све податке и омогућава праћење извршења различитих ствари, од трансакција па све до рада читавог микросервиса и слично. Најчешће, *log* јесте фајл у којем се исписују сви подаци трансакција. Креирање патерна на основу *log*-ова означава да се интеракција са новим микросервисом обавља на основу читања *log*-ова. *Log* трансакција приказује само промене које су извршене над подацима, тако да на тај начин микросервис се не оптерећује стварима које су заправо промењене трансакцијама. За разлику од тригера базе података, препознавање промена и читање трансакција се реализује ван базе података, што смањује зависност микросервиса од базе података монолита.
- **Редовна копија података** – можда и најједноставнији приступ имплементацији патерна јесте креирање програма или скрипте који по редовном распореду скенира базу података у потрази за подацима који су промењени и копира те податке на одређену дестинацију. Главни проблем који настаје јесте препознати који су подаци промењени од последњег скенирања. Одређене базе података дозвољавају преглед мета података помоћу којих се може видети који делови базе података су се променили. Такође, други начин може бити да се табелама додају колоне које се односе на време и датум када је одређени рекорд био уписан или мењан у бази података.

Change data capture патерн је генерално користан патерн у различитим ситуацијама, али посебно се истиче када се ради о реплицирању података. Што се тиче микросервиса, најбољи случај за коришћење овог патерна настаје када је неопходно да систем реагује на одређену промену података, али такву реакцију не могу предвидети пресретањем захтева ван монолита или променом већ постојећег кода (Њуман, 2019, стр. 124).

3.3. Подела домена података

Претходним описом патерна миграција приказани су различити начини како се функционалност монолита може издвојити у микросервис. Међутим, проблем и изазов који су изостављени приликом описа патерна миграције јесте везан за домен података који извучена функционалност обухвата. Током описа микросервиса наглашено је да микросервиси функционишу најбоље када сваки микросервис има посебну базу података за себе, са којом комуницира и за коју је у потпуности одговоран и има потпуно власништво. Проблем настаје приликом извлачења функционалности монолита која је везана за одређени домен података којем такође и остале функционалности имају приступ и које га потенцијално користе. Због тога, приликом миграција функционалности у микросервисе доводи и до разбијања базе података монолита, на тај начин да не ремети рад монолита, а поспешује и унапређује функционисање микросервиса (Њуман, 2019, стр. 125). У наставку мастер рада приказани су различити патерни разбијања базе података монолита, као и сви изазови и проблеми који долазе са тим. Међутим, пре описа патерна разбијања базе података, неопходно је описати изазове и проблеме који се јављају са разбијањем базе података монолита. Проблеми и изазови се јављају током сепарације базе података, где такође је битно објаснити начине сепарације, односно разлику између физичког одвајања базе података и логичке сепарације и слично.

Пре почетка приче о различитим приступима разбијању базе података, неопходно је објаснити разлику између логичке и физичке сепарације. Прва помисао током приче о разбијању базе података јесте на његову логичку сепарацију, односно на сепарацију према функционалностима које обухвата. Један од начина логичке сепарације јесте да у оквиру једне конекције система за управљање базом података, постоји више различитих шема базе података, од којих свака је задужена за одређену функционалност монолита са којом комуницира. У тренутку када се шеме базе података одвоје у две различите конекције система за управљање базом података или у две различите врсте система за управљање базом података, тада се ради о физичкој сепарацији базе података. Тиме, логичком сепарацијом се омогућава једноставно издвајање функционалности и испуњавање принципа енкапсулације, односно скривања информација, док се физичком сепарацијом постиже боља отпорност

система (отпорност на грешке, отпорност на различите нападе и одржавање стабилности ситема). Такође, физичка сепарација помаже приликом решавања конфликта око приступа ресурсима који утичу на то да потенцијални проблеми уског грла и кашњења одговора буду у што мањој мери присутни (Њуман, 2019, стр. 158-159).

Како би прича о патернима раздвајања базе података била комплетна, прво је неопходно разматрати различите приступе разбијању базе података, односно секвенце корака приликом разбијања базе података. Са издвајањем дела базе података, издваја се и код релевантан за њега и од тога се формира микросервис. Процес формирања микросервисне архитектуре није комплетан докле год функционалност није издвојена у микросервис и домен података који она обухвата није радзвојен од базе података монолита. Међутим, чест проблем који се јавља код тимова програмера јесте одлука о томе шта ће се прво разбити. Због тога, разликују се три приступа разбијања базе података – прво разбијање кода па онда базе података, прво разбијање базе података па онда кода и разбијање кода и базе података истовремено (Њуман, 2019, стр. 160-161).

Разбијањем базе података прво, база података се дели у две шеме са издвојеним подацима релевантних и битних за функционалност која ће се извучити. Најтежи и најзначајнији изазов приликом дељења базе података јесте нарушавање референцијалног интегритета података. У наставку мастер рада, у делу патерна разбијања базе података, је описан патерн којим се решава питање проблема референцијалног интегритета података, али представља важан изазов и проблем који може утицати на перформансе и функционисање апликација. Такође, извршавање одређених захтева ка бази је знанто спорије него са свим подацима на једном месту. Разлог споријег враћања одговора је у подацима који се сада, са подељеним базама, налазе на два различита места. Сада, када монолитна апликација шаље захтев за SELECT упитом, која треба да врати одређене податке, упит се мора спровести над две шеме базе података, где се затим мора користити JOIN операција која спаја све податке враћене из обе шеме базе података који се затим, спојени, враћају моноитној апликацији. Постоје одређени патерни који се користе када се одлучи прво за разбијање базе података. Један од патерна јесте прављење посебних репозиторијума за сваки од ограничених контекста (функционалности монолита које се издвајају).

Уместо да постоји један слој репозиторијума који ће бити задужен за перзистенцију свих података сваког ограниченог контекста, увођењем посебних репозиторијума од којих сваки је задужен за један свој домен података у зависности од ограниченог контекста који обухвата, смањује се оптерећење и одговорност једног слоја репозиторијума. Поделом репозиторијума се добија бољи утисак и преглед који део кода је задужен за који део домена података што доприноси даљем процесу разбијања базе података. Овај патерн је јако ефективан у ситуацијама када је неопходно изменити монолит да би се боље разумело како га поделити. Такође, поделом репозиторијума се, поред прегледа домена података у бази података и како их треба поделити, даје и увид у ограничене контексте који се налазе у монолиту. Други патерн који се може користити јесте креирање шема базе података по ограниченом контексту. Главна идеја микросервиса јесте да сваки од њих рукује сопственим подацима и за њих буде у потпуности одговоран. Пре декомпозиције и издвајања кода функционалности, раније се може започети раздвајањем базе података по шемама у зависности од ограничених контекста. Овим се олакшава сам процес издвајања функционалности у микросервисе, обзиром да домен података којим он рукује га већ чека, спреман и издвојен од базе података монолита. Међутим, коришћење овог патерна је исплативо само онда када је намера трансформација монолита у микросервисну архитектуру. Уколико одлука остане при монолитној архитектури, онда постојање више база података је сувишно, обзиром да је додатан напор неопходан како би се више база података одржавало и руковало (Њуман, 2019, стр. 161-165).

Разбијањем кода прво, односно издвајањем функционалности у микросервисе, се боље стиче увид у то који подаци су неопходни новом микросервису за функционисање. Онда, када се утврди домен података неопходан микросервису, прелази се на процес разлагања базе података монолита. Такође, разбијањем кода прво, добија се бенефит ранијег добијања сегмента кода који се може самостало испоручити и подинути. Међутим, честа лоша пракса тимова програмера јесте да након извлачења функционалности у микросервисе, не започну процес раздвајања података из базе монолита. Врло битна ствар приликом одлуке за разбијањем кода прво јесте да се буде истрајан у намери да, након креирања микросервиса, се такође и неопходни домен података извуче и надовеже микросервису за коришћење. Као и са претходним приступом разбијања, и за овај приступ постоје одређени патерни који се

користе. Један од таквих патерна може бити креирање API-а у монолиту који ће бити задужен за комуникацију са креираним микросервисом и враћати му податке који су му неопходни. Уместо да директно приступа подацима у монолиту, нови микросервис ће слати захтеве ка дефинисаном API-у путем којег ће микросервис приступити траженим подацима. Овим патерном се не мора приступити разлагању података из базе одмах, већ се база података чува онаква каква јесте и тиме се концепт енкапсулације и даље задржава обзиром да је монолитна апликација та која је одговорна за скривање информације од спољашњих корисника. На овај начин се микросервис изолује од монолита, и уколико се приступи чувању података у бази монолита (иако није препоручиво), овим патерном је то најидеалније постићи обзиром да се најбитније карактеристике микросервиса и даље поштују, сачувајући независност и самосталност микросервиса. Најбоље је користити овај патерн у ситуацијама када је код, одговоран за руковање и одржавање тражених података, и даље у оквиру монолита. Битан концепт микросервиса јесте енкапсулација података којим она приступа. Обзиром да се то у монолиту постиже, оваквим приступом је могуће делимично испунити главне карактеристике микросервиса. Међутим, важна особина микросервиса јесте и потпуна одговорност и власништво над подацима што се овим патерном не испуњава јер је монолит тај који је и даље одговоран за податке у бази података. Због тога, овај патерн треба да буде привремено решење пре него што се приступи потпуном раздвајању података од базе података монолита и његовом придруживању микросервисима. Претходни патерн је добар приступ како микросервиси могу приступити подацима монолита и тако задрже своју изолованост и самосталност и испоштују концепте енкапсулације. Међутим, ситуација која може бити проблематична јесте када функционалност, коју микросервис имплементира, ради испуњавања захтева има потребу за уписивањем у базу података. На пример уколико функционалност тражи уписивање нових података у табелу, база података монолита се проширује што се коси са тиме да подаци и шема базе података буде у надлежности монолита и функционалности које монолит има. Наредним патерном се дати проблем решава креирањем нове шеме само за потребе микросервиса. Микросервис остаје у комуникацији са шемом базе података монолита, али такође добија своју шему у којој може уписивати жељене податке. Такође, чест проблем нарушавања референцијалног интегритета се јавља и у овом патерну који се може решити али више речи о таквим патернима је дата у наставку. Разлог нарушавања референцијалног интегритета је тај

што сада постоје две исте табеле али у различитим шемама базе података при чему у једној шеми, табела може бити повезана са више различитих табела што процес извлачења и додавања табеле у другој шеми, у којој не постоје такве табеле, може бити изазован. Као што је и претходно речено, овај патерн је најбоље користити када додавањем нове функционалности у микросервис се захтева уписивање одређених података. Обзиром да подаци који се уписују нису неопходни монолиту и представљају вишак, њиховим додавањем у базу података монолита се само проузрокују додатни трошкови одржавања података (Њуман, 2019, стр. 165-169).

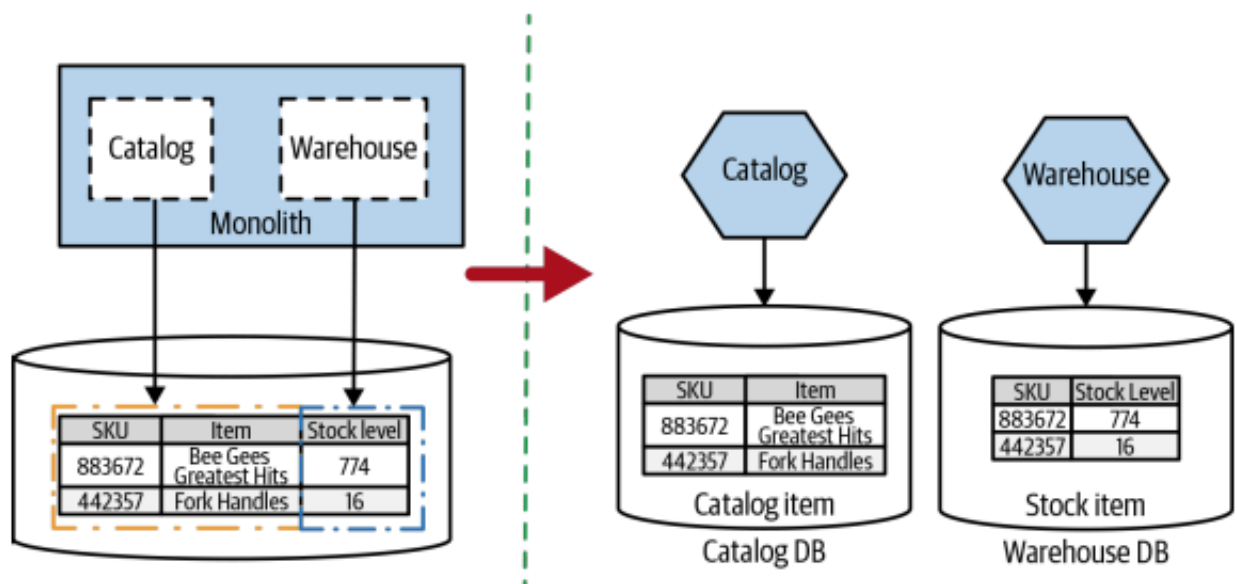
Разбијање кода и базе података заједно, у исто време, је могуће, међутим резултат таквог разбијања није могуће видети одмах, већ после одређеног времена након што се успостави све и пусти да ради. Овакав процес разбијања је један велики корак који подразумева доста малих корака који се морају предузети заједно како би све функционисало и било на месту. Због ограниченог прегледа развоја, овај приступ треба избегавати обзиром да пркоси такође и принципима микросервиса – инкременталном поступку развијања, имплементације и испоручивања (Њуман, 2019, стр. 170).

Описана су три приступа којима се може користити приликом процеса разбијања монолитног система. Међутим, одабир самог приступа зависи од различитих фактора и ситуација. Према Њуману (2019, стр. 170), уколико је могуће променити монолит, и тим програмера је забринут око потенцијалног утицаја који може имати на перформансе и конзистентност података, најбољи приступ би био разбити прво базу података. У осталим ситуацијама, разбијање кода прво је бољи приступ, обзиром да може помоћи приликом разумевања домена података и којем ограниченом контексту припадају. Важно је напоменути да одабир приступа варира без обзира на ситуацију, и због тога је неопходно дискутовати са тимом програмера и у договору донети одлуку која ће бити најбоља за организацију и софтверски систем који се развија.

Приступима разбијања базе података приказала се сепарација шеме базе података на једном вишем нивоу прегледа целокупног разбијања софтверског система. Такође, установљена је и разлика између логичке и физичке сепарације базе података која је важна за целокупан процес трансформације монолитне апликације у систем са микросервисном архитектуром. Међутим, претходним се нису покрили комплекснији и детаљнији изазови који долазе разлагањем базе података. У наставку су дати патерни разлагања базе података који дају детаљнији преглед и решења проблема и изазова који се јављају у дубини самих шема база података.

Патерн – Подела табеле

У одређеним ситуацијама, нови микросервиси ће сваки захтевати део података (колона) једне табеле и тиме проузроковати поделу табела и пребацивања делова на друге микросервисе (Њуман, 2019, стр. 171). На следећој слици се може видети како из монолита, чије функционалности шаљу захтеве ка једној шеми базе података, се добија микросервисна архитектура са две базе податка:



Слика 25. Патерн поделе табеле базе података [24]

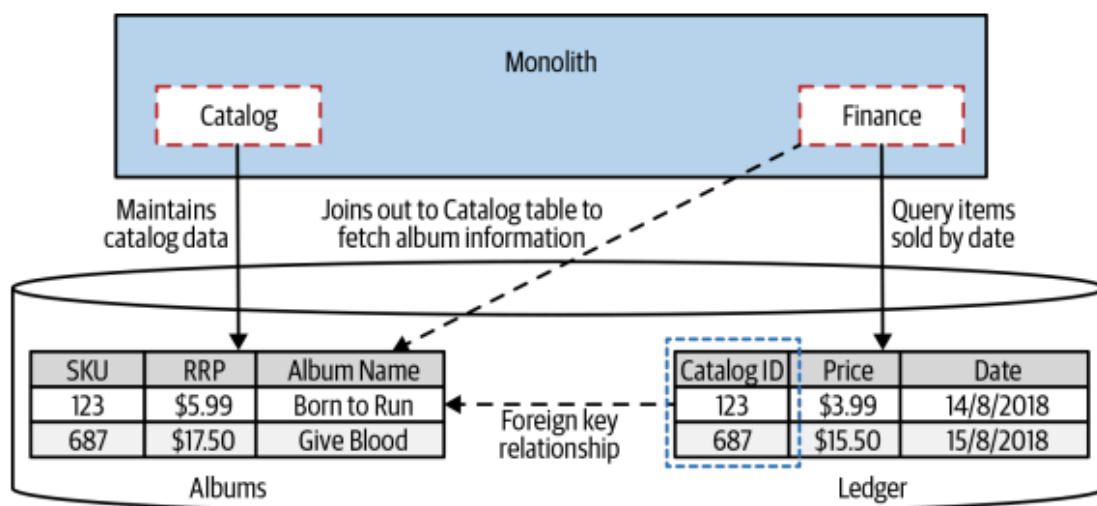
Патерн поделе табеле је интуитиван и јасан. Функционалности приступају различитим колонама и за сваку функционалност која се издваја у микросервисима неопходно је одвојити и домен података којима приступају. Обзиром да подаци којима приступају се налазе у истој табели, поделом табеле на две које садрже потребне колоне свакој од функционалности које се придружују креираним микросервисима. Такође, иако су у две различите шеме, обе табеле морају да садрже примарни кључ који је идентификатор дате табеле. Међутим, патерн поделе табеле не функционише у тренуцима када обе функционалности приступају истој колони једне табеле. Решење које се примењује у таквим ситуацијама јесте да се колона припише табели (шеми базе података) оног микросервиса чији ограничени контекст је више задужен или би требао у потпуности да буде задужен за ту колону. Други микросервис, у тренуцима када треба да приступи датој колони, шаље API захтев микросервису који је задужен за заједничку колону и он извршава тражене операције над колоном. Недостатак овог решења код микросервиса је тај што микросервис губи сигурност коју добија трансакцијом своје базе података, већ се ослања на функционисање базе података другог микросервиса (Њуман, 2019, стр. 172-173).

Патерн – Пребацивање везе између табела, са ограничењима референцијалног интегритета, у код

Патерн поделе табеле решава проблем када две функционалности приступају подацима које се налазе у оквиру једне табеле. Међутим, патерн поделе табеле је немогућ у ситуацијама када је неопходно да функционалност приступа подацима две табеле које су међусобно повезане спољним кључевима, односно између њих постоје одређена ограничења референцијалног интегритета. Проблем се јавља приликом раздвајања функционалности у микросервисе који комуницирају са две повезане табеле. Питање расподеле табела и придруживања табела микросервисима постаје теже, обзиром да за реализацију таквог процеса се нарушава референцијални интегритет базе података. Кроз пример две функционалности, финансија и каталога албума, даје се преглед проблема који се јавља у ситуацијама издвајања функционалности у микросервисе која комуницира са две или више повезаних табела. Функционалност финансија се бави продајом, односно ценом, датумом куповине свих албума који се налазе у бази података, док се каталог албума бави чувањем свих тих

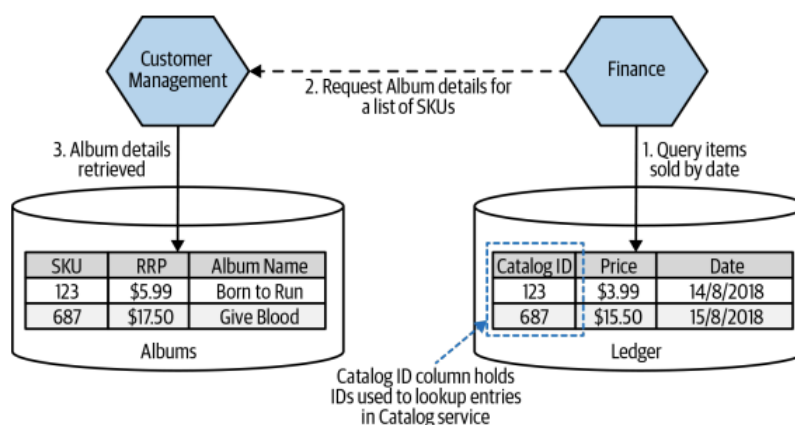
албума, тј. назива албума, његове цене и слично. Функционалност финансија као један од захтева може имати приказ свих неопходних података једног албума. Међутим, за добијање свих података неопходно је проћи кроз обе табеле које су међусобно спојене кључевима. Коришћењем операције JOIN се лако путем једног упита долази до свих неопходних података једног албума када је монолитна апликација са једном базом података у питању. Са микросервисном архитектуром, издвајањем функционалности, процес реализације таквог примера се компликује. Обзиром на концепт ограничених контекста микросервиса, сваки микросервис представља једну малу целину домена пословања са свим својим подацима које обухвата. На основу тога, микросервисна архитектура се пројектује тако да функционалност финансија постоји као засебан микросервис са базом података која у себи једино садржи табелу финансијских података албума. Са друге стране, функционалност каталога албума се такође издваја у посебан микросервис са својом базом података и табелом са подацима о албумима. Захтев прикупљања свих података је сада, путем једног упита са JOIN операцијом, немогуће остварити. Као решење, јавља се патерн померања везе, које табеле међусобно имају, у код помоћу којег ће се подаци спајати (Њуман, 2015).

На следећој слици приказана је монолитни систем систем претходно описане ситуације који се претвара у микросервисну архитектуру:



Слика 26. Монолитни систем са табелама повезаних кључевима пре примене патерна померања везе у код [24]

Прва ствар, током имплементације решења оваквог проблема, јесте уклонити везу функционалности финансија са табелом албума. Табела албума у потпуности припада функционалности каталога и над њом има потпуно власништво. Најлакши и најбржи начин како се може регулисати приступ подацима албума од спољашњих функционалности јесте креирањем API-а над функционалношћу каталога. Слањем API позива, функционалност финансија лако може добити неопходне податке албум табеле. Тим програмера се мора помирити са чињеницом да овакав систем остварује два позива ка бази података како би формирао неопходни одговор са свим подацима. Са већим бројем позива долазе и ризици од потенцијалног кашњења, спорије брзине извршавања система и сличног. Врло је важно успоставити добре и лоше стране увођења микросервисне архитектуре и утврдити колико је исплативо имплементирати је. На примеру ових функционалности, спорији одзив система због слања два позива ка базама података не би био проблем обзиром да генерисање извештаја о албумима би се обављало једном месечно. Онда када се утврди да мана потенцијалног споријег одзива система не утиче толико на бенефите који се добијају микросервисном архитектуром, одлука о покретању миграције на микросервисну архитектуру делује исправно и могуће. Такође, патерном се губи и важан концепт референцијалног интегритета међу табелама. Ограничења и правила постављени референцијалним интегритетом се сада дефинишу у оквиру микросервиса, а не као пре у издвојеним базама података. Таква ограничења и правила се дефинишу различитим валидацијама кроз читав микросервис или се креирају одређене акције над подацима који се налазе у микросервису. У зависности од технологија које микросервис користи, дефиниција оваквих валидација ограничења и правила не мора нужно постојати (Њуман, 2015). Приказ микросервисне архитектуре након примене патерна се налази на следећој слици:



Слика 27. Микросервисни систем са одвојеним табелама након примене патерна померања везе у код [24]

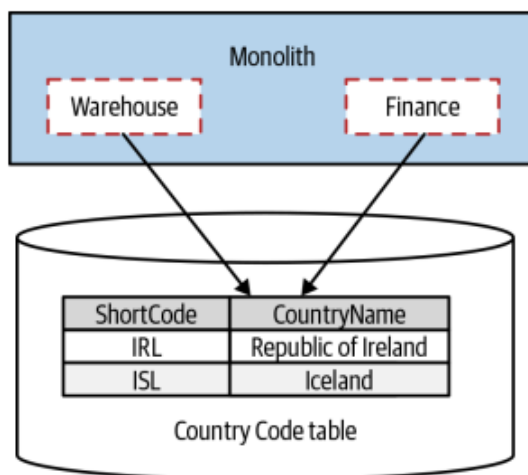
Генерални проблем који се јавља приликом разбијања и раздвајања базе података јесте везан за конзистентност података. Референцијални интегритет у бази података игра кључну улогу у одржавању конзистентности базе података. Наиме, брисање једног рекорда у табели албума неће бити могуће уколико постоји одређени рекорд у табели финансија који је спољним кључем везан за њега. Такође, ажурирање спољног кључа одређеног рекорда на вредност рекорда који не постоји је немогуће управо због референцијалног интегритета. Издвајањем функционалности у микросервисе и додавањем њихових табела, брисање рекорда у табели финансија неће бити спречено и ограничено обзиром да не постоји референцијални интегритет ка табели албума. Табела албума се сада налази у новој бази података, бази података која је придружена каталог микросервису. Међутим, постоје неколико трикова којима се може служити како би се конзистентност података, између база података микросервиса, одржала. Први је везан за брисање рекорда из базе података. Наиме, пре брисања рекорда једне табеле, у другој табели (другог ограниченог контекста) треба проверити да ли рекорд, који се брише, се користи. То се може постигнути лако, позивом провере ка API-у микросервиса који је задужен за базу података друге табеле. Проблем који може настати са овим приступом јесте да се у току провере коришћења рекорда, унесе рекорд који ће заправо користити рекорд који се брише. Микросервис добија обавештење да се рекорд не користи и брише га, док у том тренутку се заправо ствара зависност са датим рекордом, новим чувањем у базу података. Како би се спречио настанак овакве ситуације, неопходно је спречити било какво чување рекорда који у себи садржи зависност ка рекорду који се брише. Постоје различите технике закључавања ресурса које се могу користити за овакве потребе. Такође, решење оваквих проблема је једноставније када се ради о архитектури са максимално два микросервиса. Међутим, код система са великим бројем микросервиса, слањем позива ка сваком микросервису ради провере ресурса је захтеван, скуп процес и одузима превише времена. Боља опција може бити омогућити функционалности да рукује чињеницом да информација коју тражи од друге функционалности можда не постоји тамо. Статусни код 410 (*Gone*), као део одговора који се враћа функционалности, представља и најреалнији одговор у ситуацијама недоступности или непостојања ресурса код других база података. Наиме, 410 *Gone* јесте статусни код који означава да затражени ресурс је био доступан али више није. За разлику од статусног

кода 404 *Not Found*, који означава да ресурс није пронађен, 410 статусни код је прецизнији обзиром да даје до знања позиваоцу да је тражени ресурс постојао, али да сада више не постоји. Поред тога што добија проверу доступности ресурса, функционалност добија знање о претходном стању ресурса, односно да је такав ресурс постојао и да је због одређене ситуације обрисан или једноставно онемогућен за коришћење. Међутим, најједноставнија опција осигуравања конзистентности приликом брисања података јесте конфигурисати систем тако да не постоји могућност брисања података. Тиме, једини начин брисања који се дозвољава јесте ткз. „меко брисање“ (*soft delete*). Меко брисање се извршава променом стања одређеног рекорда у стање које означава његову недоступност. Свака табела поседује одређену колону која означава стање у којем се рекорд налази тако да неким брисањем се извршава операција ажурирања дате колоне у статус недоступности (Њуман, 2019, стр. 176-177).

Приликом реализације оваквог патерна, врло је важно узети у обзир процену ситуације коришћења патерна. Наиме, уколико се раздвајају две табеле које очигледно припадају различитим контекстима а међусобно су повезане, овакав патерн представља реалну и исплативу опцију. Међутим, врло лако се долази у проблем када се патерн имплементира над табелама које су међусобно повезане кључевима али очигледно спадају у оквиру истог ограниченог контекста. Коришћење патерна у таквим ситуацијама не даје резултате и само може проузроковати додатне проблеме (Њуман, 2019, стр. 178).

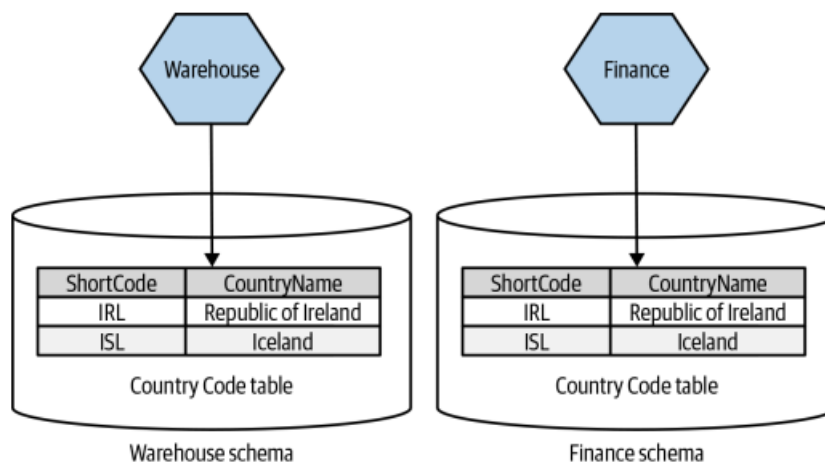
Патерн – Дељење референцираних статичких података

Референцирани статички подаци у табелама могу креирати неке занимљиве изазове приликом миграције у микросервисну архитектуру. Статички подаци, попут информација о државама (назив, код и слично), представљају изворе података многим функционалностима које се не мењају током релацијаје захтева, већ их различите функционаности дохватају ради реализације сопствених потреба. Обзиром да такве податке могу дохватати различите функционалности, јавља се проблем распоређивања таквих података приликом миграције монолитног система у систем са микросервисном архитектуром (Њуман, 2015). На следећој слици приказан је монолитни систем и комуникација са таквим, статичким, подацима:



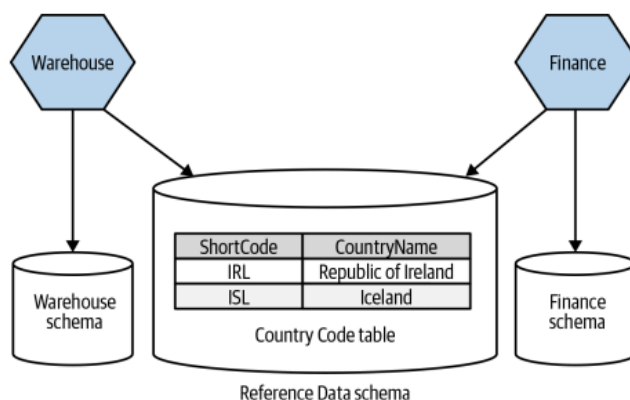
Слика 28. Монолитни систем са дељеним референцираним статичким подацима [24]

Постоје неколико видова патерна дељења референцираних статичких података. Један од видова јесте дуплирање табеле за сваку од функционалности која је користи. Тиме, свакој издвојеној функционалности, будућем микросервису, се додаје табела у бази података коју имају. Један од недостатака оваквог вида јесте исти као и са претходним патерном. Проблем конзистентности података. Иако је претходно речено да се овакви подаци најчешће користе за дохватање, подаци су динамични и мењају се па и тако овакви подаци се могу ажурирати, додати нови и слично. Додавањем у једној табели једне базе података проузрокује и додавање код других база података управо због конзистентности која мора постојати међу базама података микросервиса (Њуман, 2015). На следећој слици приказан је систем након реализације оваквог патерна дуплирања података:



Слика 29. Микросервисни систем са дуплираним референцираним статичким подацима [24]

Уколико се тежи постојању само једног извора проверених и тачних информација, без ризика од некозистентних података међ различитим базама података, креирањем једне заједничке шеме базе података којој ће сви микросервиси приступати делује као најисплативија опција. Наиме, микросервиси и даље имају своју базу података али статички подаци на које референцирају ће се налазити у једној заједничкој бази података. Међутим, иако решава проблем конзистентности података, други изазови се јављају овим видом патерна. Ризици везани за промене које свака од функционалности може извршити су донекле ублажени природом самих података, обзиром да се овакви подаци најчешће користе у сврхе дохватања и читања. Принцип микросервиса који се крши оваквим приступом јесте принцип потпуног власништва микросервиса над подацима које користи. Обзиром да користе заједнички извор података, статички подаци су дељени и нису у потпуном власништву ниједног микросервиса (Њуман, 2019, стр. 181). Приказ микросервиса са дељеном шемом базе података којој приступају се налази на следећој слици:



Слика 30. Микросервисни систем са дељеном шемом базе података [24]

Трећи начин јесте искључити коришћење базе података у потпуности. Наиме, референцирани статички подаци би се чували у коду на одређени начин. Може се остварити чувањем података у одређеном фајлу и уврстити га у оквиру сервиса, креирањем енумариције или слично (Њуман, 2015). Проблем конзистентности се може јавити и овим начином обзиром да се оваква нумерација или фајл са подацима мора копирати у сваком микросервису и онда променом једног енума или фајла се морају такође мењати и остали у другим микросервисима. Како би се спречила дупликација података и појава проблема конзистентности, јавља се опција копирања података у заједничку библиотеку којој микросервиси могу приступити. Међутим, овакав приступ такође долази са манама. Наиме, главна особина микросервиса јесте да су то независне целине које се могу испоручити у било ком тренутку без утицаја других микросервиса. Заједничка дељена библиотека се супроставља овом златном правилу микросервиса у ситуацијама када се библиотека, због одређених фактора, мора ажурирати. Како би сви микросервиси покупили нове, ажуриране, податке, сви микросервиси ће морати да се заједно поново подижу на продукцији у тренутку кад ажурирана библиотека буде доступна. Због тога, уколико се користи принцип постојања енумерације или одређеног фајла за сваки микросервис, тимови програмера се морају помирити са чињеницом да неће сви микросервиси у истом тренутку имати најновије податке (Њуман, 2019, стр. 183).

Последњи начин јесте можда екстремнији и представља поступак креирања сервиса у којем ће се налазити референцирани статички подаци. Међутим, овај приступ се често третира као претереним послом обзиром да се, за тако једноставне и просте податке, прави потпуно нови сервис са којим долази целокупна процедура његове имплементације, испоруке и подизања на продукцији (Њуман, 2015). У зависности од процене процеса прављења оваквог сервиса, тимови се могу одлучити за усвајање овакве тактике. Уколико процес није захтеван и одузима мало времена, може представљати разумну опцију обзиром да се принципи микросервиса овим поступком не нарушавају. Са друге стране, ако се утврди да процес може трајати дуго и знато потрошити ресурсе и повећати трошкове, упуштање у процес креирања микросервиса за статичке податке може бити превише. Нови микросервис означава и проширење инфраструктуре и повећану комуникацију на мрежи. Тиме се утиче на кашњење одговора, јер се додаје још један сервис са којим други микросервиси комуницирају а

који је неопходан за остварење њихових функционалности. (Њуман, 2019, стр. 184-185).

Овим поглављем описани су разилити проблеми и изазови који се јављају поступком трансформације у микросервисну архитектуру. Такође, обрађени су и начини миграције, како самих функционалности, тако и домена података који обухватају. На основу овог поглавља, у наставку мастер рада приступа се конкретној имплементацији система са микросервисном архитектуром, од почетног дефинисаног монолитног система.

4. Приступ трансформације монолитне апликације у програмски систем са микросервисном архитектуром

Након описаних патерна миграције, мастер рад обухвата и конкретну примену знања стеченог у претходним областима мастер рада. У наставку је представљен пример архитектуре монолитног система над којем се имплементирају изабрани патерни миграције, односно опис функционалности које обухвата и који су кључни приликом процеса раздвајања монолита. Опис и приказана архитектура представљају кључне факторе који утичу на одабир и реализацију миграције. Поред тога, овим поглављем мастер рада пружа се увид у кораке примене патерна миграције и њихове резултате. Објашњава се поступак реализације као и резултати конкретне реализације изабраног патерна миграције. Коначни исход представља комплетну архитектуру микросервисног система насталог миграцијом изабраног монолитног система.

4.1. Вербални опис

Пре почетка миграције функционалности монолита у микросервисни систем, неопходно је детаљно разумети постојеће функционалности монолита. Вербални опис представља кључан корак како би се функционалности на адекватан начин разумеле и издвојиле у засебне извршне целине.

У наставку је дат софтверски систем за праћење полагања студената. Циљ софтверског система јесте да омогући платформу на којој професори могу да креирају тестове које студенти полагају. Са сваким креираним тестом, професор резервише полагање за дати тест где такође има могућност пријављивања нових студената на дато полагање. Платформа пружа опцију праћења резултата које студенти остваре на тестовима као и штампање тестова, скидањем фајлова PDF формата, за потребе уручивања студентима на датум резервисаног полагања у просторијама факултета. Са друге стране, студенти се пријављују на платформу како би могли да прате резултате које су остварили на полагањима које професор, након прегледаних тестова, уписује такође путем платформе.

Приступ софтверском систему је могућ једино путем пријављивања на систем кроз регистроване налоге. Сваки студент има могућност самосталне регистрације на систем, као и опцију креирања налога од стране професора. Уколико студент одабере опцију креирања налога од стране професора, налог се креира са креденцијалима студентовог мејла и лозинке његовог броја индекса. Са тим, приликом сваког пријављивања студента на систем, на платформи се приказује порука у којој се наглашава да лозинка коју студент тренутно користи је слаба и да је послат мејл са даљим поступцима измене постојеће лозинке. За разлику од студената, професори своје налоге већ имају у систему, ускладиштене у бази података, са већ постојећим креденцијалима датим за коришћење.

На основу претходног се закључује да постоје два актора софтверског система: студент и професор. Софтверски систем ова два актора разврстава у улоге корисника (енг. *"User"*) и администратора (енг. *"Admin"*). Улоге се разликују по пермисијама које имају за приступ различитим функционалностима. На тај начин, корисник (студента) има приступ прегледу података својих полагања као и пријављивање на одређено полагање. Са друге стране администратори садрже, поред тих, приступ свим осталим функционaлностима које се тичу креирања полагања, студената, тестова и њихових питања и одговора, брисања, ажурирања и других.

4.2. Архитектура монолита

Претходно су наведене различите интерпретације монолитне апликације, али заједничка карактеристика свих је то да су дизајниране тако да целокупна пословна логика и домени података буду обједињени на једном месту, у оквиру исте апликације. На следећој слици је представљена архитектура монолитног софтверског система, укључујући начин на који комуницира са корисничким интерфејсом и базом података.



Слика 31. Општа архитектура монолитног система

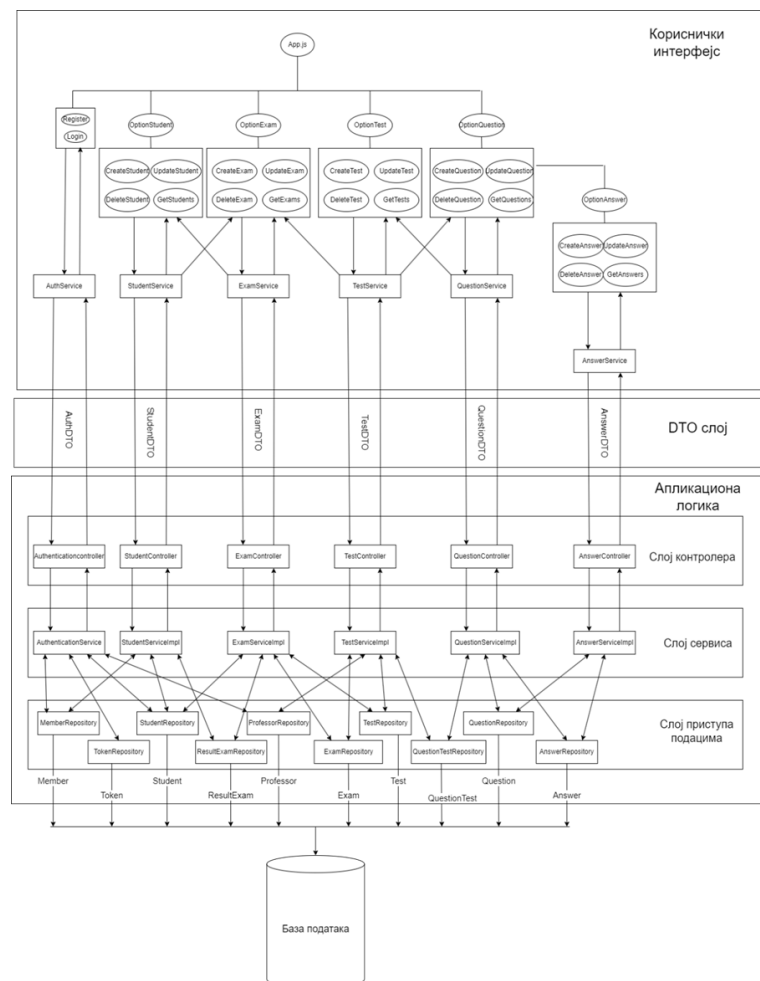
Претходна архитектура илуструје ток комуникације корисничког интерфејса и апликације једног софтверског система. Неопходним подацима кориснички интерфејс приступа звањем изложених тачака приступа монолитне апликације, при чему свака тачка садржи логику потребну за обраду захтева. Кориснички интерфејс нема директан приступ бази података, већ приступ подацима остварује искључиво преко монолитне апликације. На слици су такође приказани „DTO” (енг. *Data Transfer Object*) и објекти домена, који играју важну улогу у преносу података и мапирању доменских података софтверског система.

Комуникација корисничког интерфејса са сервер апликацијом подразумева размену различитих типова података неопходних за извршавање захтеваних процеса. Међутим, обзиром да представљају две одвојене целине, подаци са којима раде се могу разликовати по типу, вредности или намени. У одређеним ситуацијама, корисничком интерфејсу је потребно више информација које сервер апликација мора да припреми и проследи. Са друге стране, кориснички интерфејс понекад шаље вишак података који нису неопходни монолиту за чување и читање из базе података. Из наведених разлога, чест изазов је припремити тачно један објекат који би служио за враћање података корисничком интерфејсу, чување у бази података и за интерне потребе монолитне апликације. Такође, важно је спречити изложеност објекта базе података другим факторима и компонентама система, обзиром да су такви објекти одговорни за складиштење осетљивих података који су спремни за упис или читање података из базе података. Решење ових проблема представљају „објекти преноса података“, DTO објекти. DTO објекти јесу једноставни објекти са методама за приступ и уписа података у променљиве које садрже, односно представљају пакете информација у комуникацији са корисничким интерфејсом. Поред података неопходних за рад са базом података, могу садржати и друге податке које могу бити од значаја апликацији са којом се комуницира. Међутим, DTO објекат не садржи директну везу са објектима који се користе за комуникацију са базом података. Такви објекти, који се користе за везу са базом података, се називају „објекти домена“ (енг. *Domain object*). Увођењем DTO и објекта домена, кориснички интерфејс се изољује од објекта који имају директан контакт са базом података, чиме се чува интегритет и сигурност базе података. Иако подаци који су потребни бази података долазе са стране корисничког интерфејса путем DTO објекта, неопходно је успоставити механизам података који ће од DTO објекта направити објекат домена са свим неопходним подацима. Реализација оваког трансфера података је могуће извршити на различите начине путем кастомизованих класа и његових дефинисаних метода, коришћењем библиотека треће стране (енг. *third-party libraires*) који у себи садрже одређену логику мапирања DTO објекта у неопходне објекте домена и сличних начина (Фовлер, 2012).

На основу претходно дефинисане опште архитектуре монолитног софтверског система, креиран је конкретан пример софтверског система креирања тестова за полагање студената. Вербални опис дефинише седам опсега, односно функционалности који овакав софтверски систем обухвата:

1. Питања и одговоре које тест садржи
2. Тестове
3. Полагања
4. Студенте
5. Професоре
6. Аутентификацију
7. Слање мејлова студентима

Следећа слика приказује детаљнију архитектуру монолитног софтверског система конкретног примера, од које започиње процес разлагања монолитне архитектуре у програмски систем са микросервисном архитектуром:



Слика 32. Детаљна слојевита архитектура конкретног примера монолита

Врста монолитне архитектуре, која је одабрана за процес трансформације у систем са микросервисном архитектуром јесте слојевита монолитна архитектура. Сваки слој има своју улогу у функционисању целокупног система и представља засебну целину унутар монолита. Слој контролера служи за пријем и преусмеравање података захтева ка одговарајућој логици, која се налази у слоју сервиса. Слој сервиса је задужен за извршавање конкретне логике система. Затим, слој сервиса даље комуницира са базом података путем слоја за приступ подацима који једини садржи директан контакт са базом података монолита. У оквиру сваког слоја налазе се мале, засебне целине, одговорне за испоруку специфичних функционалности система монолита. На конкретном примеру, тест контролер комуницира са тест сервисом, који обавља сву логику везану за функционалност тестова. Приступ подацима у бази података тест сервис остварује преко тест репозиторијума, који припада слоју за приступ подацима. Иако слојевита структура нуди одређене предности организације и структурираности, кључна мана јесте међузависност ових функционалности. Немогуће је приступити функционалностима полагања уколико тест функционалности нису доступне, управо због тога јер се све функционалности налазе на једном месту и међусобно су умрежене у једну целину.

Са претходно дефинисаном монолитном архитектуром и разлогом њеног растављања, прелази се у фазу миграције монолитног система у програмски систем са микросервисном архитектуром. У наставку мастер рада приказани су различити патерни миграције који су се користили за разлагање функционалности оваквог монолитног система, домена података као и базе података коју она поседује.

4.3. Поступак миграције

Са дефинисаним примером монолитне архитектуре започиње се процес трансформације у програмски систем са микросеверном архитектуром. У наставку мастер рада приказује се поступак трансформације кроз примере патерна миграције, дефинисаних у претходном теоријском делу мастер рада. Обзиром да је целокупна трансформација конкретног примера извршена постепено, у наставку је приказан сваки корак трансформације, кроз патерн миграције који се користио у сваком од специфичних корака.

Пре почетка трансформације, неопходно је утврдити путању процеса раздвајања неопходних функционалности. Претходно су утврђена три начина почетка трансформације монолитне архитектуре према Њуману (2019, стр. 160-161): разбијањем кода прво, разбијањем базе података прво и разбијањем кода и базе података истовремено. За потребе имплементације конкретног примера узет је први приступ, разбијањем монолитног кода прво. Разлог одабира оваквог приступа јесте могућност утврђивања припадности домена података свакој од функционалности која је одвојена разлагањем програмског кода. Монолитна апликација је направљена тако да домени података нису засебни за сваку функционалност, већ се провлаче кроз различите функционалности и нивое апликације. Због тога, како би се боље утврдиле границе домена података и њихова распоређеност, постепеним разбијањем кода проактивно се такве границе одређују и припајају датим функционалностима.

За првобитно извлачење функционалности из монолита и формирање микросервиса користио се „*Strangler fig application*“ патерн миграције. Разлог његовог коришћења јесте управо могућност употребе монолитног система, од стране корисника, током процеса трансформације у нови микросервисни систем (Њуман, 2019, стр. 81). Обзиром да се софтверски систем праћења полагања може активно користити од стране особља факултета, неопходно је одржати такав систем активним током разлагања и оптимизације архитектуре софтверског система. У наставку, приказан је пример извлачења конкретне функционалности монолитног софтверског система.

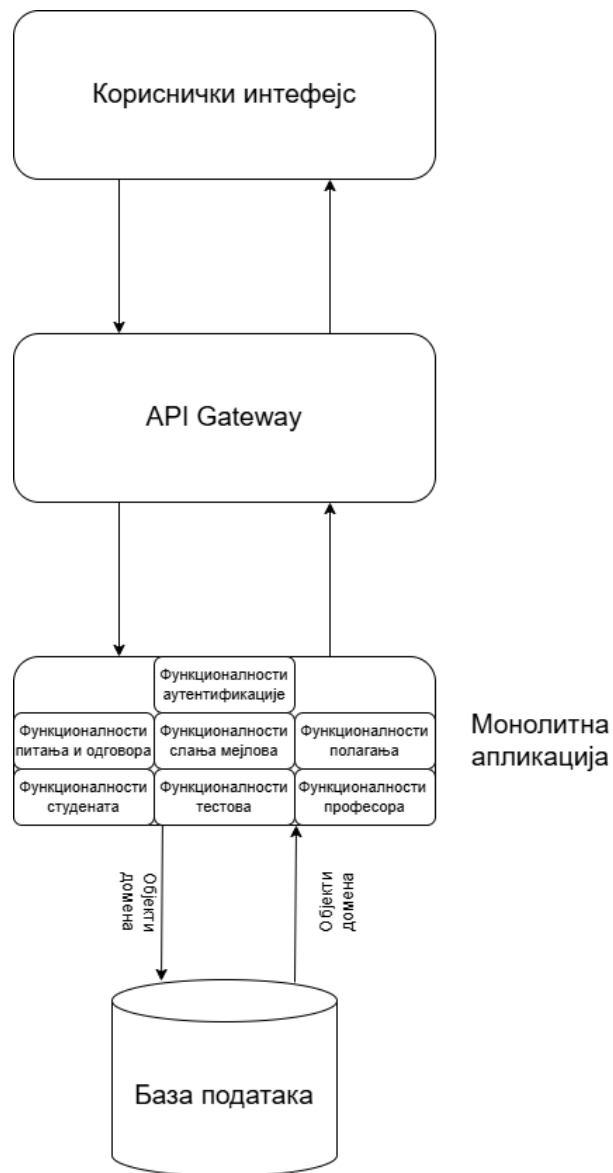
Разлагање „*Strangler Fig Application*“ патерном миграције

Циљ „*Strangler Fig Application*“ патерна миграције јесте креирати нов ситем који ће моћи да коегзистира са старим системом, не утичући на његово извршавање и перформансе. Такав систем, у одређеном тренутку треба у потпуности да замени стари систем и тиме га угаси, при чему ће сви захтеви тада бити преусмерени ка новом систему за извршавање (Њуман, 2019, стр. 79).

Претходно је споменуто да разлог највеће употребе овог патерна јесте доступност старог система корисницима на коришћење у току процеса извлачења дате функционалности из ситема, и коегзистенција изворног кода система са новим који ће бити креиран. Таква могућност да би била доступна, неопходно је креирати одређеног посредника који одлучује и преусмерава захтеве ка специфичним системима. На тај начин, се путем посредника, може установити у ком тренутку ће захтеви бити преусмерени ка новом или старом систему за извршавање (Њуман, 2019, стр. 83-86).

На основу претходне слике архитектуре монолита, примећује се да апликација корисничког интерфејса директно комуницира са софтверским системом монолита. Пре почетка миграције, уводи се нова целина која преузима улогу посредника између корисничког интерфејса и софтверског система – проху. Проху сервер је представљен путем новог микросервиса који се назива „*API Gateway*“ и преузима улогу преусмеравања захтева ка жељеним системима за извршавање функционалности. Такође, путем оваквог микросервиса се додаје могућност излагања новог система који је у процесу имплементације, како би се добио увид у стање његовог процеса имплементације.

На следећој слици приказана је архитектура монолитног софтверског система са новим посредником (*API Gateway*-ом) између корисничког интерфејса и монолитне апликације.



Слика 33. Proxy посредник између корисничког интерфејса и монолитне апликације

Нова архитектура монолитног система са посредником сада представља почетну тачку и основну архитектуру којом се воде сви типови миграције на конкретном примеру мастер рада. На основу овакве архитектуре, креће се са миграцијом жељене функционалности у нови микросервис и поставља се темељ програмског система са микросервисном архитектуром.

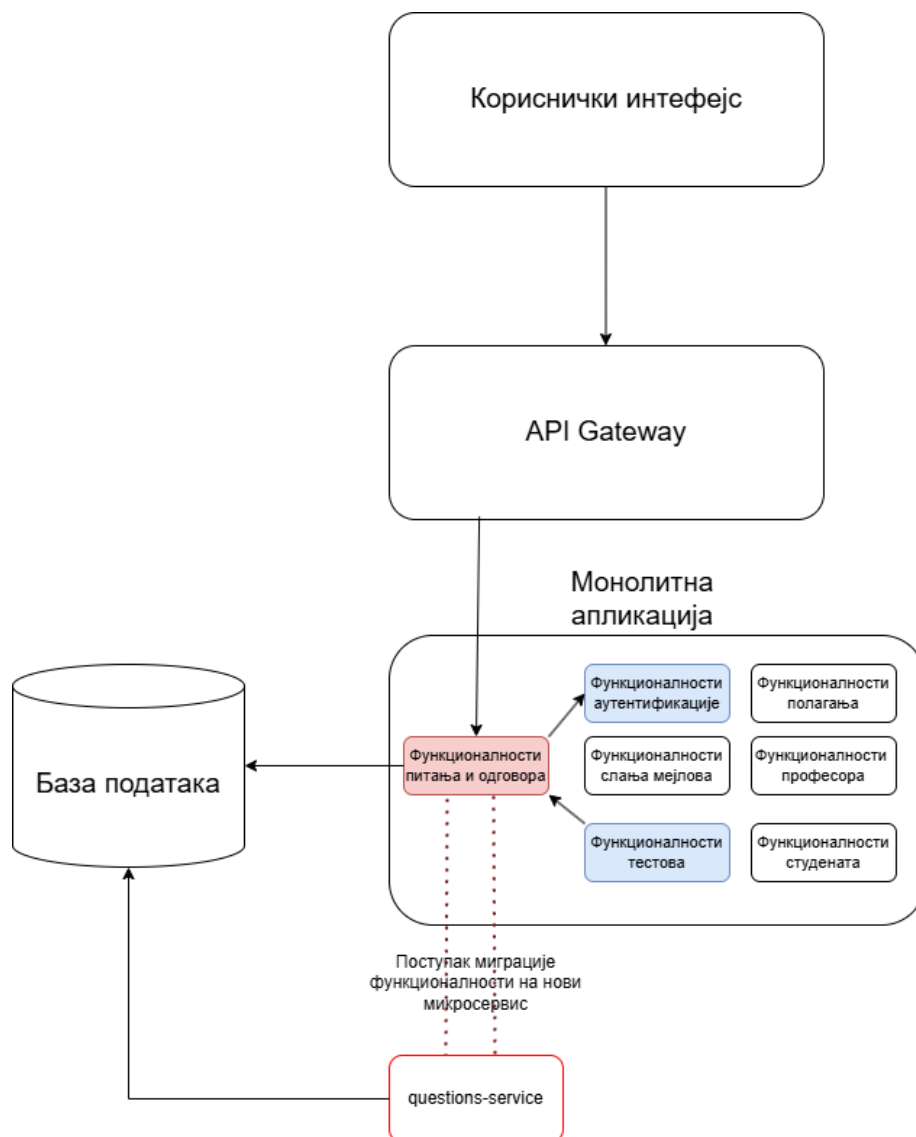
Миграција монолитне функционалности започиње одабиром функционалности која ће бити прва издвојена у микросервис. Главни изазов оваквог процеса јесте сам избор те функционалности, односно одређивање почетне тачке. Према Њуману (2015), разбијање монолитне у микросервисну архитектуру треба да буде инкрементално, при чему се за почетак бирају функционалности које имају мање зависности од осталих функционалности. Приликом процеса разбијања монолита важно је осигурати да свака издвојена функционалност пружа максималну могућу корисност и има јасан смисао у оквиру целокупног система. Са тим у вези, функционалности које су најизолованије и чија логика најмање зависи од рада осталих функционалности, представљају добре кандидате за почетну функционалност миграције обзиром да захтевају мање промена код осталих функционалности са којима комуницирају.

Битно је напоменути да се може кренути од било које функционалности приликом трансформације у микросервисну архитектуру. Разлог зашто је издвојен проблем одабира јесте у времену и напору које ће разлагање одузети приликом трансформације. Како би процес трансформације био бржи, прецизнији и лакши, неопходно је кренути од функционалности које у мањој мери зависе од других. Таква функционалност је функционалност питања и одговора која је најизолованија тачка у читавом монолитном системом. Њена једина повезаност са осталим функционалностима и процесима јесте путем тест функционалности. Због тога, као прва функционалност одабрана је функционалност питања и одговора на чијем примеру се приказује *„Strangler Fig Application”* патерн. На сличан начин, постепено се и остале функционалности, истим патерном, мигрирају у нове микросервисне системе. У мастер раду се приказује само поступак овог патерна миграције на примеру функционалности питања и одговора.

Након првог корака утврђивања функционалност која се мигрира, следећи корак јесте постепено премештање такве функционалности у нову целину, тј. микросервис. Процес трансформације подразумева креирање структуре пројекта над којом се микросервис покреће и дефинисање базе кода коју микросервис користи за извршавање тражене функционалности. Чест изазов приликом мигрирања функционалности јесте база кода, односно да ли са новим микросервисом долази и новији, свежији изворни код или се код са старог система копира на нови. Према Њуману (2019, стр. 76-77), одлука о томе

да ли ће код бити прекопиран или бити писан изнова зависи од ситуације. Обзиром да циљ миграције конкретног примера мастер рада није оптимизација и писање нове функционалности, већ пребацивање постојеће функционалности, копирањем кода се значајно скраћује време и умањује напор потрошен на имплементацију логике микросервиса.

На следећој слици приказан је почетак миграције функционалности питања и одговора у нови микросервис „*questions-service*” :

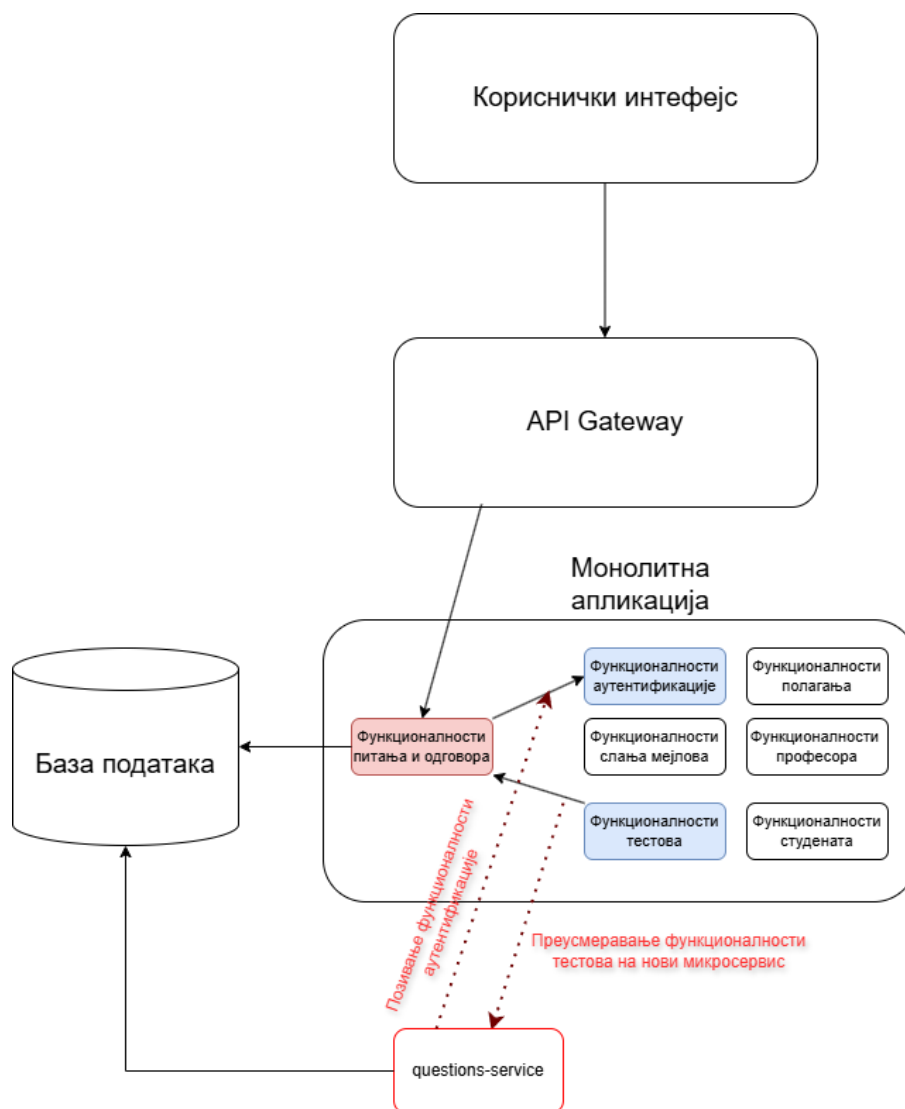


Слика 34. Миграција функционалности питања и одговора на нов микросервис

Нови микросервис, „*questions-service*“, сада садржи целокупну логику функционалности питања и одговора. Међутим, са слике се примећује да је функционалност питања и одговора у оквиру монолита у комуникацији са функционалности тестова и аутентификације. Наиме, сваки захтев који треба доћи до функционалности питања и одговора, преусмерава се ка функционалности аутентификације који је задужен за проверу креденцијала и пермисија које корисник, који приступа жељеној функционалности, садржи. Одређене функционалности су обезбеђене да се користе само од стране администратора (професора), док су друге доступне и администраторима и корисницима (студентима). Такође, једини вид комуникације функционалности питања и одговора са осталим функционалностима монолита јесте путем функционалности теста. Тест садржи питање и одговоре и самим тим је у потпуности креиран и спреман да буде присвојен одређеном полагању који ће студенти полагати.

Са претходне слике нови микросервис „*questions-service*“ ни на који начин не комуницира са поменутих функционалностима. Како би микросервис обављао у потпуности све функционалности питања и одговора, неопходно је га повезати са таквим функционалностима и тиме га спремити за коришћење и потпуну транзицију са монолитне функционалности у микросервисну.

На следећој слици је приказано умрежавање новог микросервиса са функционалностима са којима је повезан:

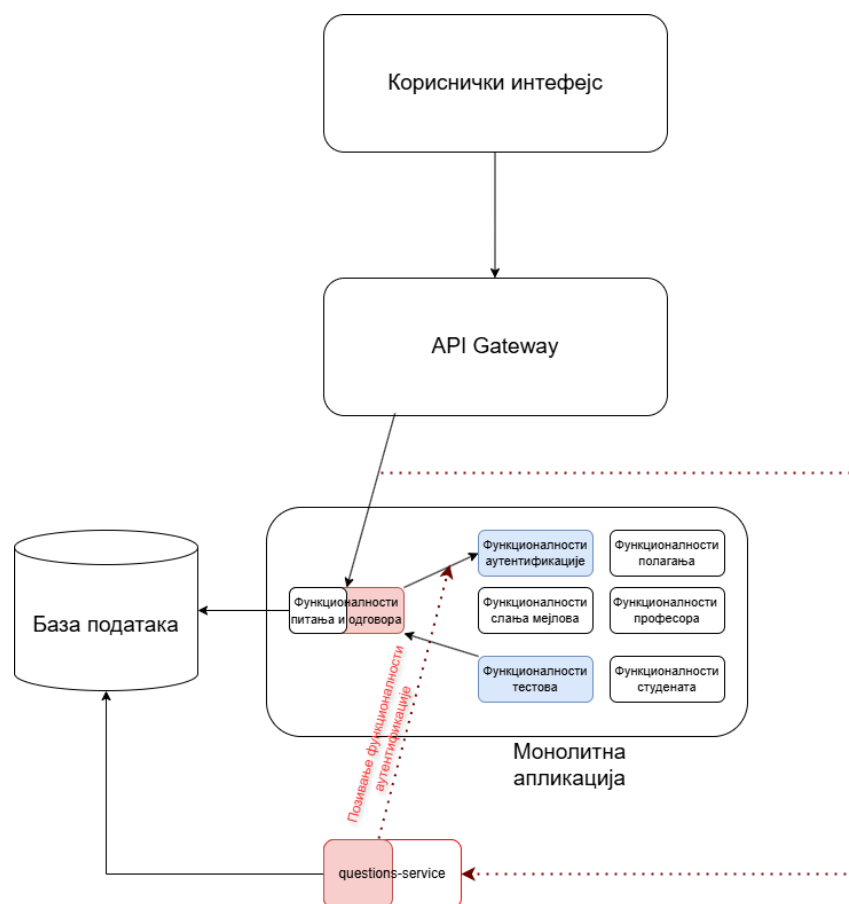


Слика 35. Умрежавање микросервиса са функционалностима монолита

Проблем који се јавља са оваквим примером преусмеравања и миграције јесте када постојећа монолитна функционалност позива функционалност која се мигрира. Врло важна особина „*strangler fig application*” патерна јесте могућност лаког пребацивања између коришћења нове и старе имплементације, односно коришћења микросервиса или монолита (Њуман, 2019, стр. 80).

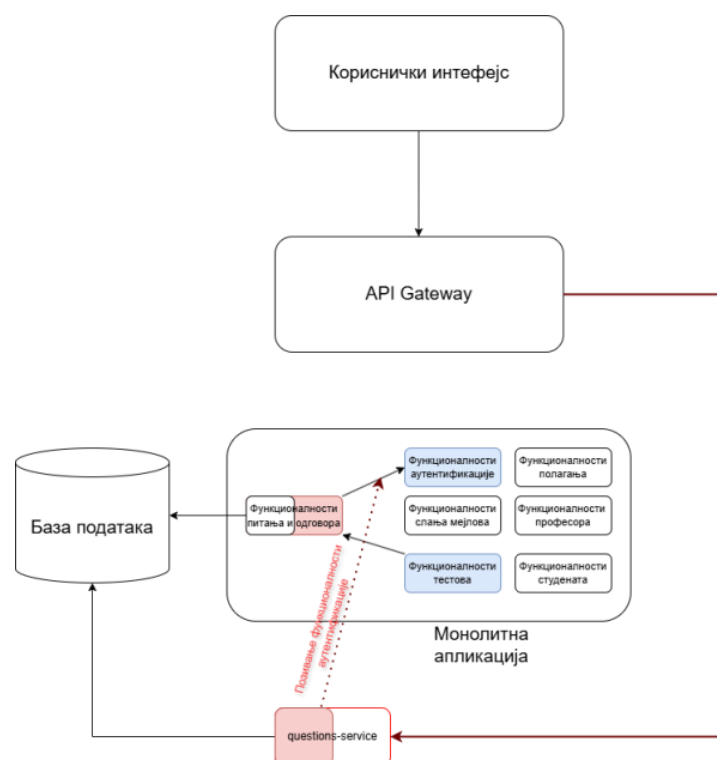
Међутим, уколико се у монолиту функционалност преусмери ка коришћењу функционалности микросервиса, приликом било какве појаве грешке на страни микросервиса не постоји никакав механизам повратка на комплетно коришћење монолита обзиром да функционалност фиксирано користи услуге микросервиса.

Постоје неколико решења оваког проблема. Уколико је функционалност сувише велика и повезана на различите начине са осталим функционалностима, Њуман (2019, стр. 88) предлаже инкрементану миграцију дела функционалности. Наиме, део функционалности, који зависи још увек од других функционалности монолита, остаје као део монолита, док преостали део функционалности се мигрира на нови креирани микросервис. На тај начин, у зависности од захтева, приликом преусмеравања на нови микросервис биће доступне услуге које се могу самостално извршавати, док остале које су зависне остају део монолита док не дође до миграције свих функционалности које су зависне од услуга које пружа. На следећој слици приказано је решење постепене миграције дела функционалности на нови микросервис:



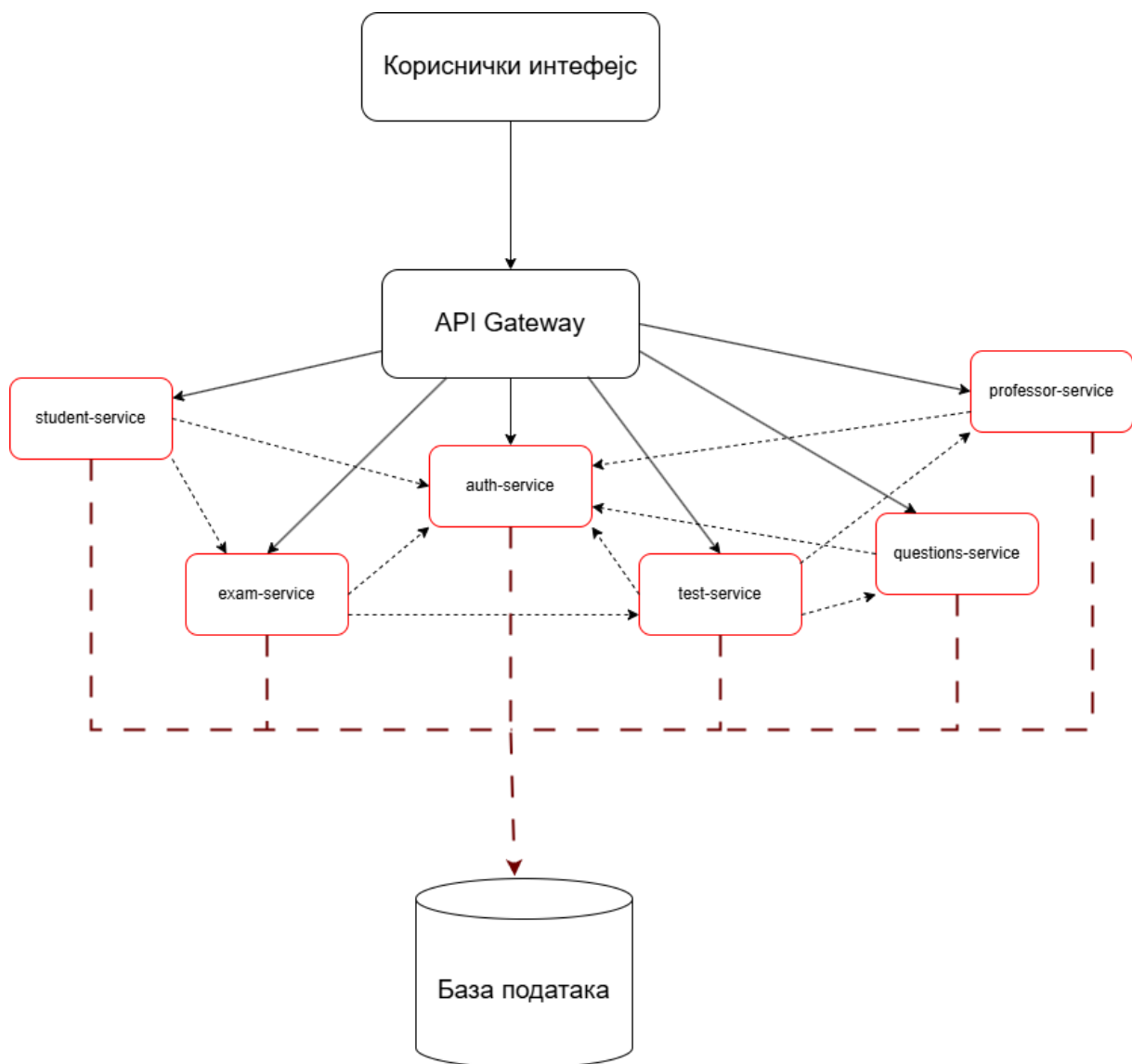
Слика 36. Миграција дела функционалности на нови микросервис и преусмеравање ка истом

Када тест функционалност буде мигрирана на микросервис, такође и целокупна функционалност питања и одговора ће моћи бити мигрирана на свој микросервис. Међутим, уколико је жеља успоставити миграцију целокупне функционалности на нови микросервис, и при том оставити могућност пребацивања између нове и старе имплементације функционалности, као боље решење долази други патерн миграције „*Branch by Abstraction*”. Патерн који омугаћава пребацивање „дубоких“ функционалности, односно оних делова који се не могу „*strangler fig*” патерном издвојити у нови микросервис. Наиме, „*Branch by Abstraction*” патерн нуди могућност лаког пребацивања између коришћења функционалности унутар монолита или микросервиса коришћењем различитих софтверских патерна који обрађују преусмеравања базираних на вредностима које им се пружају. Рад „*Branch by Abstraction*” патерна је приказан у наставку мастер рада на конкретном примеру, док је за решење поменутог проблема, за потребе конкретног примера мастер рада, коришћен принцип мигрирања делова функционалност „*strangler fig application*” начином миграције. Након мигрирања делова функционалности питања и одговора који се могу користити одвојено и од којих функционалност тестова не зависи, прелази се на преусмеравање позива ка новој, мигрираној, функционалности на микросервису. На следећој слици приказано је преусмеравање позива преко посредника, „*API Gateway-a*”, ка новом микросервису:



Слика 37. Преусмеравање позива на нови микросервис

Последњим кораком преусмеравања позива је комплетиран „*Strangler Fig Application*” патерн миграције. Обзиром да се истом техником служило за извлачење свих осталих функционалности монолита, у мастер раду је приказан само претходни пример реализације оваквог патерна. Као резултат реализације „*Strangler Fig Application*” патерна миграције добија се следећа микросервисна архитектура:



Слика 38. Резултат реализације *strangler fig application* патерна миграције над свим функционалностима

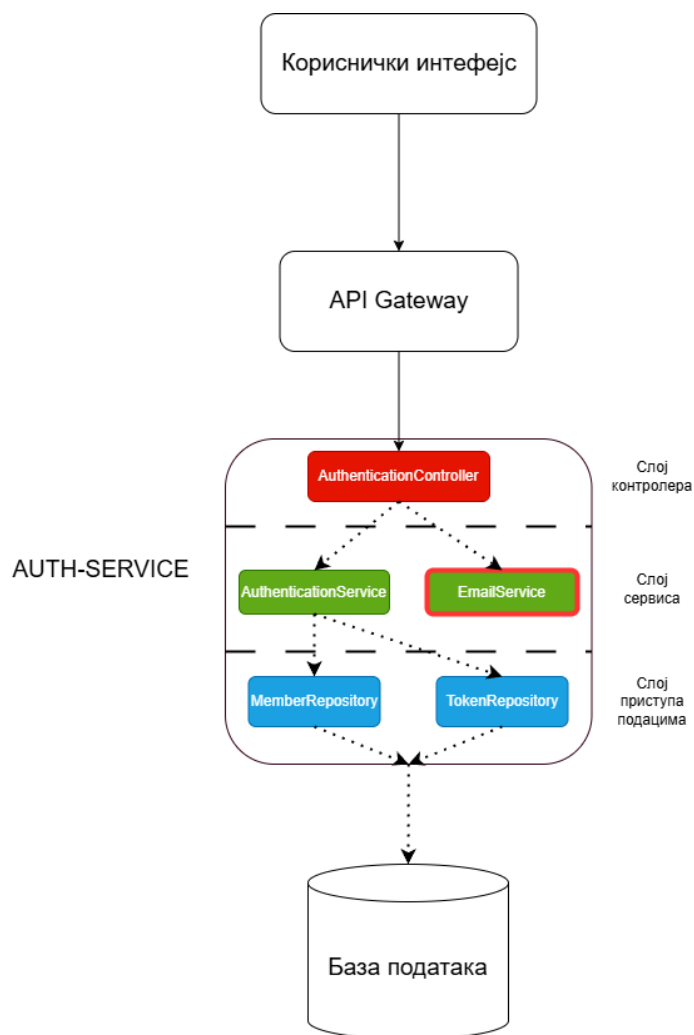
Претходна слика илуструје део резултата трансформације монолитне апликације у систем са микросервисном архитектуром. Према Њуману (2019, стр. 5) једна од важних особина микросервиса јесте поседовање засебног домена података, кроз базу података, који сваки микросервис садржи за себе. Обзиром да се за конкретни пример мастер рада одлучило за приступ трансформације разбијањем кода прво, додавање базе података сваком од микросервиса долази тек касније, након свих извршених патерна миграције и распоређивања функционалности. У наставку мастер рада обрађују се још два патерна миграције на конкретним примерима: „*Branch by Abstraction*” и „*Decorating Collaborator*”.

Разлагање „*Branch by Abstraction*” патерном миграције

Претходним патерном миграције добија се основна структура програмског система са микросервисном архитектуром над којом се могу примењивати и други патерни миграције. Један од таквих јесте „*Branch by Abstraction*” патерн. Функционалности које су тренутно издвојене јесу функционалности за које је постојала могућност миграције без утицаја на рад постојећег система. Међутим, у оквиру монолита постоје и функционалности које су дубоко постављене унутар других функционалности и чију миграцију није могуће реализовати „*Strangler Fig Application*” патерном миграције обзиром да таквим начином миграције би утицала на рад читавог монолитног система. Једна од таквих функционалности јесте слање мејлова корисницима. Слање мејлова корисницима је имплементирано у оквиру функционалности аутентификације и нуди следеће услуге:

- Слање мејла за промену слабе лозинке
- Слање мејла за промену заборављене лозинке
- Слање мејла за регистрацију

За пример реализације поменутог патерна узима се слање мејла за промену заборављене лозинке. На следећој слици приказана је архитектура сервиса за аутентификацију („*auth-service*”) и функционалности које садржи:



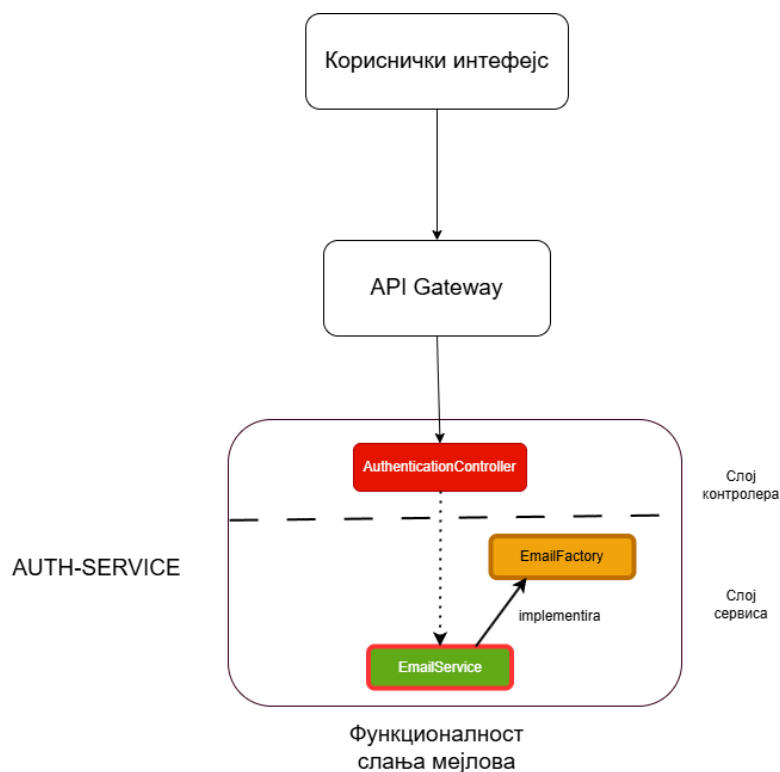
Слика 39. Архитектура микросервиса за аутентификацију „auth-service”

Са претходне слике почетне микросервисне архитектуре, може се закључити да сваки од издвојених микросервиса у себи садржи архитектуру слојевитог монолита са само једним доменом услуга који нуди. Међутим, микросервис аутентификације такође садржи и услуге слања мејлова што не спада под доменом саме функционалности аутентификације. Обзиром да услуге слања мејлова су изложене корисницима само путем контролера аутентификације, указује на то да се оваква функционалност налази дубље у оквиру саме функционалности микросервиса и да се „*Strangler Fig Application*” патерном не могу постићи жељени резултати. Начин на који се може тражена мејл функционалности мигрирати јесте мењањем базе кода тако да омогућава коришћење две имплементације, старе монолитне и нове микросервисне, без било каквог утицаја или прекида на рад читавог софтверског система. „*Branch by Abstraction*” патерн нуди такав приступ реализације миграције (Њуман, 2019, стр. 104-105)

На основу Њуманових (2019, стр. 105) пет корака за реализацију „*Branch by Abstraction*” патерна миграције, илустрован је пример извлачења функционалности слања мејлова за промену заборављене лозинке. Кроз сваки корак објашњен је поступак реализације конкретног корака. Резултат извршених корака миграције јесте нови мејл микросервис („*email-service*”), касније додат већ постојећој микросервисној архитектури.

Први корак

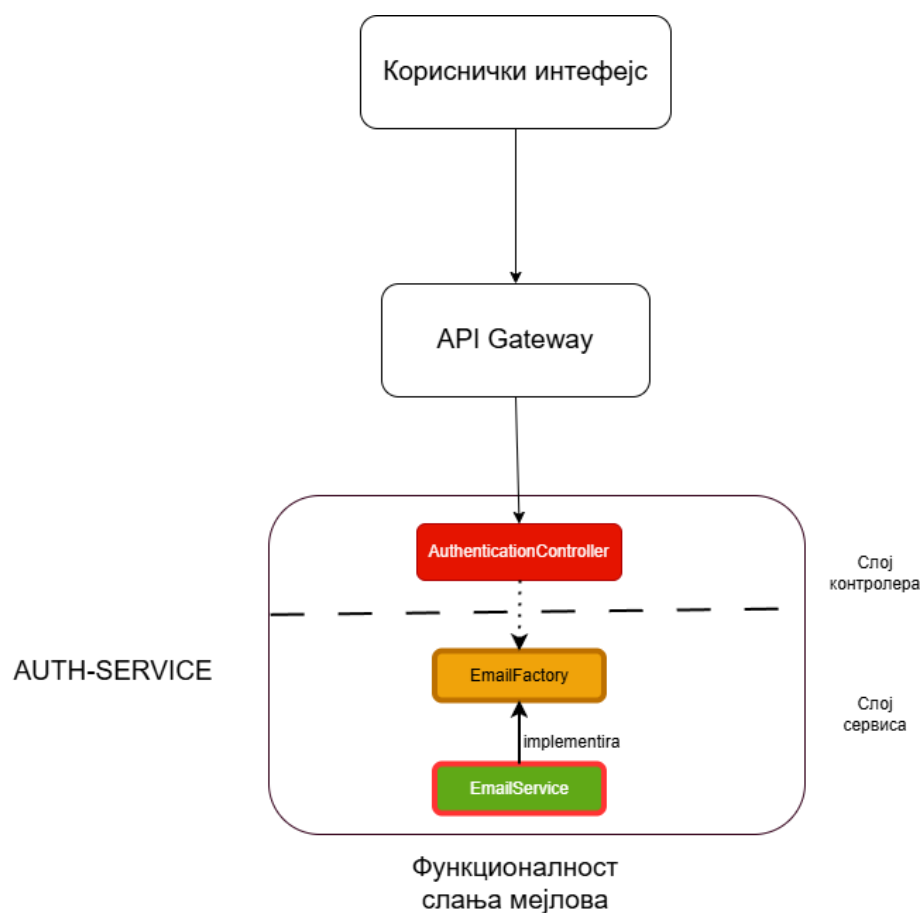
Према Њуману (2019, стр. 105-106), први корак „*Branch by Abstraction*” патерна миграције јесте креирање апстракције над жељеном функционалношћу, која ће представљати посредника између окружења које позива функционалност и саме тражене функционалности. Претходно је поменуто да је унутрашња архитектура микросервиса слојевита. На основу тога, слој апстракције се прави између слоја контролера и слоја сервиса, тачније између контролера аутентификације и мејл сервиса. Апстракција представља класу која управља позивањем неопходне имплементације у односу на захтеве који се траже. На следећој слици приказана је креирана апстракција и њен положај у оквиру архитектуре микросервиса аутентификације:



Слика 40. Креирање апстракције над сервисом слања мејлова

Други корак

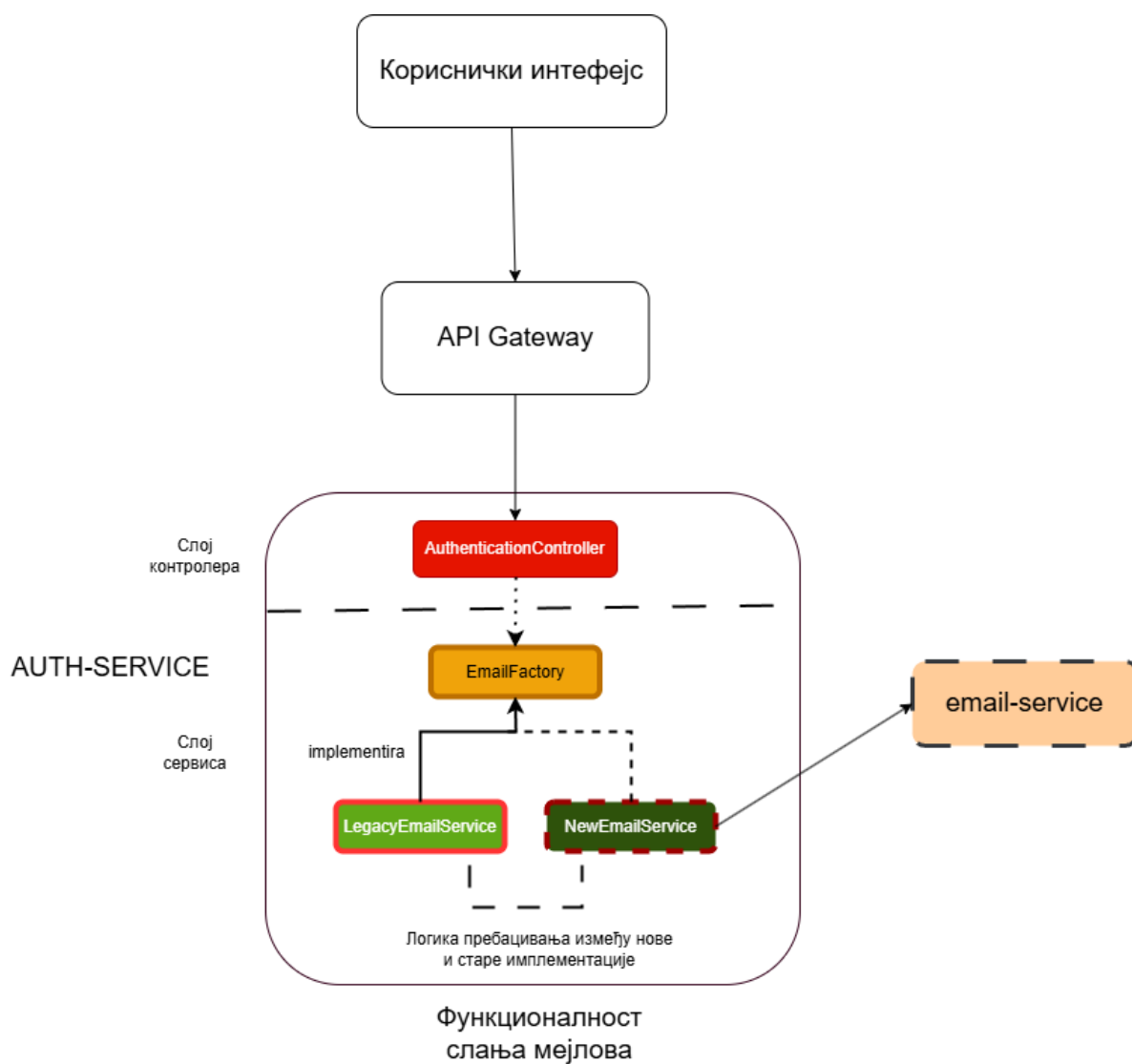
Други корак јесте коришћење креиране апстракције. Наиме, сви позиваоци функционалности над којом се апстракција прави сада комуницирају са новом апстракцијом. Имплементација сваког корака је мала и некомпличована али са оваквим инкременталним приступом постиже се сигурност у успостављању комуникације са новом имплементацијом и завршетак реализације патерна (Њуман, 2019, стр. 206). Поступак успостављања комуникације са апстракцијом је приказано на следећој слици:



Слика 41. Успостављање комуникације са апстракцијом слања мејлова

Трећи корак

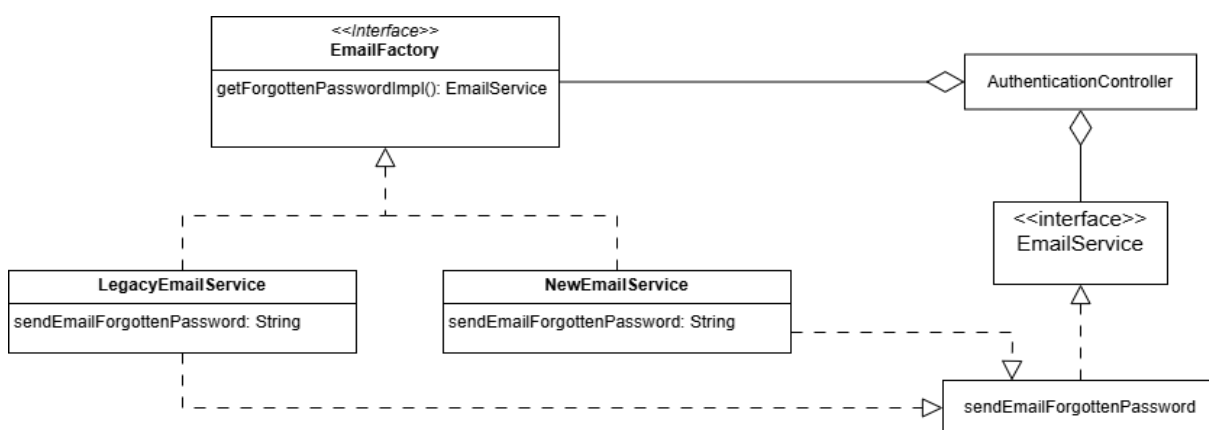
Са успостављеном комуникацијом апстракције са позиваоцима функционалности, прелази се на фазу креирања нове имплементације. Под креирањем нове имплементације за конкретни пример мастер рада подразумева се креирање нове имплементације сервиса у оквиру микросервиса аутентификације који позива нови креирани микросервис слања мејлова. Самим тим долази до креирања две целине: новог сервиса у оквиру слоја сервиса микросервиса аутентификације и изолованог микросервиса слања мејлова. Резултат реализације трећег корека је приказан на следећој слици:



Слика 42. Креирање нове имплементације и микросервиса слања мејлова

Идеално, као и у случају „*Strangler Fig Application*“ патерна миграције, опција коришћења старе имплементације би било пожељно имати приликом извлачења новог микросервиса. Овакав приступ омогућава сигурност у случају могуће недоступности, грешке у комуникацији и сличног. У оваквим ситуацијама апстракција може преусмерити извршавање логике ка старој имплементацији и на тај начин спречити кашњење одговора на захтев, пада софтверског система и другог (Њуман, 2019, стр. 108).

Логика пребацивања између коришћења старе и нове имплементације је пројектована уз помоћу дизајн патерна под називом „*Abstract Factory*“ патерн. „*Abstract Factory*“ патерн обезбеђује механизам креирања међусобно повезаних или зависних објеката путем заједничког интерфејса без конкретног инстанцирања њихових класа (Влајић, 2014, стр. 44). На конкретном примеру, заједнички интерфејс се приписује „*EmailFactory*“ интерфејсу који креира објекте нове или старе имплементације сервиса слања мејлова на основу вредности прослеђене окружењу над којом се сервис аутентификације покреће. Начин на који функционише јесте тај што, уколико се проследи окружењу да користи нову имплементацију методе слања заборављене шифре, користиће објекат класе „*NewEmailService*“. Свака метода која се позива са стране контролера аутентификације, прво се проверава над том методом, у односу на вредност прослеђеној варијабли окружења, да ли се користи нова или стара имплементација дате методе и на основу тога се позива тражена имплементација. На следећој слици је представљена логика пребацивања коришћења старе и нове имплементације коришћењем принципа „*Abstract Factory*“ дизајн патерна:

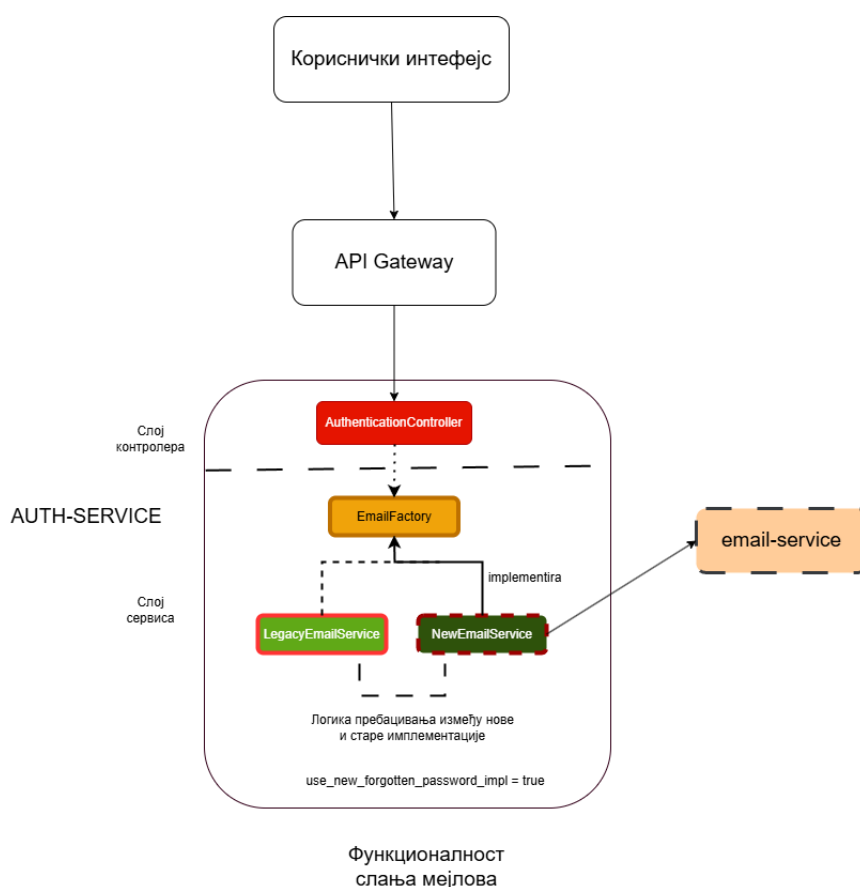


Слика 43. Логика пребацивања између старе и нове имплементације путем „*Abstract Factory*“ дизајн патерна

Са дијаграма се примећује да се конкретна имплементација налази у оквиру методе „*sendEmailForgottenPassword*” и да оба сервиса имају другачије имплементације. Сервис нове имплементације слања мејла позива микросервис слања мејлова за извршавање неопходне логике, док се код сервиса старе имплементације логика извршава у оквиру микросервиса аутентификације.

Четврти корак

Након коначне имплементације новог сервиса и микросервиса, прелази се на фазу коначног преусмеравања коришћења нове имплементације. Обзриом да је логика пребацивања реализована на претходно описани начин, прослеђивањем неопходне вредности варијабле окружења, микросервис аутентификације се конфигурише тако да користи сервис нове имплементације слања мејлова, односно да сада целокупна реализација логике је на страни микросервиса за слање мејлова. Преусмеравање контролера аутентификације на коришћење нове имплементације слања мејлова за промену заборављене лозинке је приказано на следећој слици:

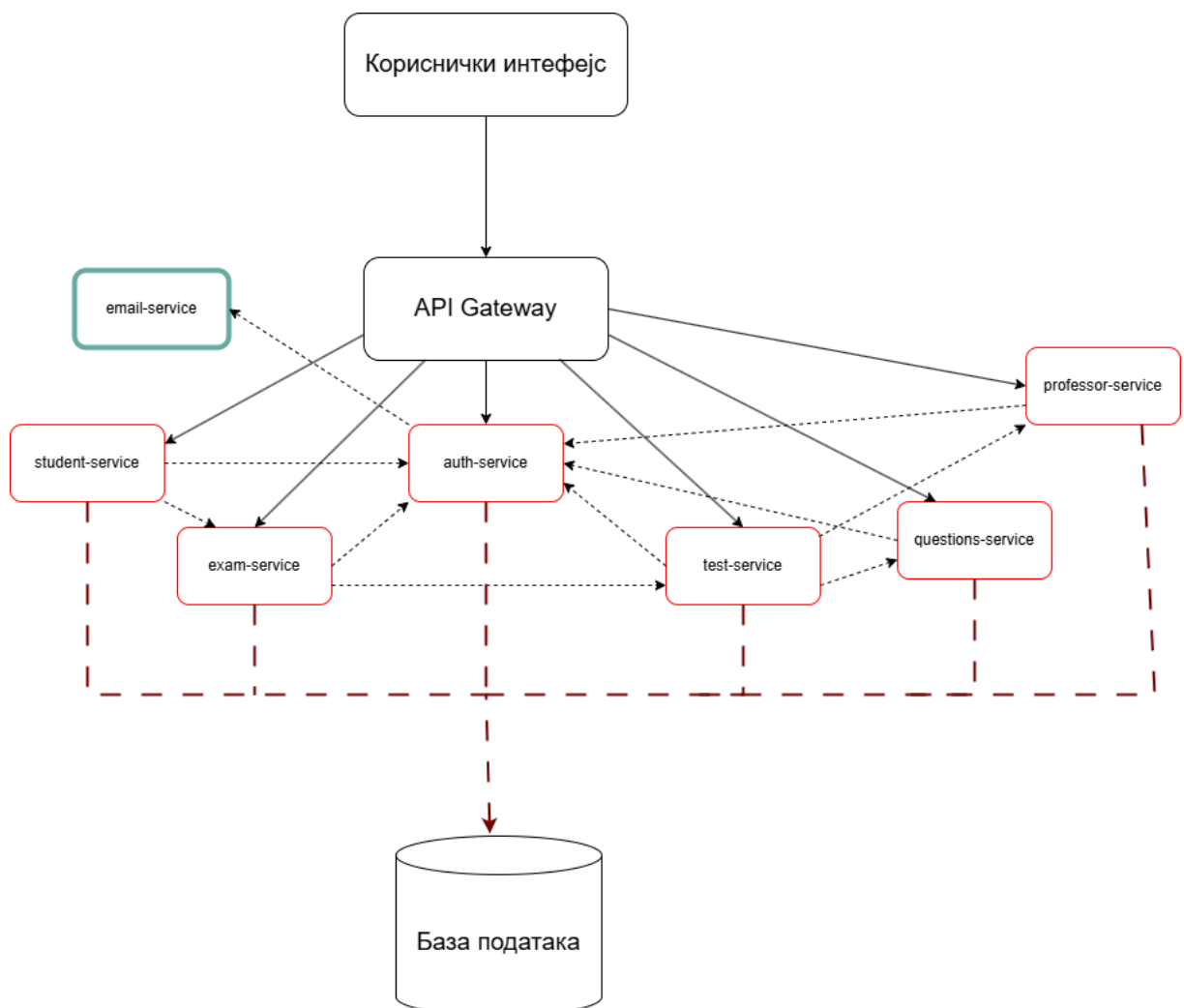


Слика 44. Преусмеравање контролера аутентификације на коришћење нове имплементације слања мејла за промену заборављене лозинке

Пети корак

Пети корак Њумановог „*Branch By Abstraction*” патерна миграције (Њуман, 2019, стр. 110-111) јесте комплетно брисање старе имплементације и преазак на коришћење нове, микросервисне имплементације. Међутим, за потребе конкретног примера мастер рада овај корак није реализован управо због механизма логике пребацивања који је претходно представљен.

Као резултат „*Branch By Abstraction*” патерна миграције, постојећој архитектури микросервиса, додаје се нови микросервис за слање мејлова „*email-service*”. Нова микросервисна архитектура приказана је на следећој слици:



Слика 45. Резултат реализације „*Branch By Abstraction*” патерна миграције

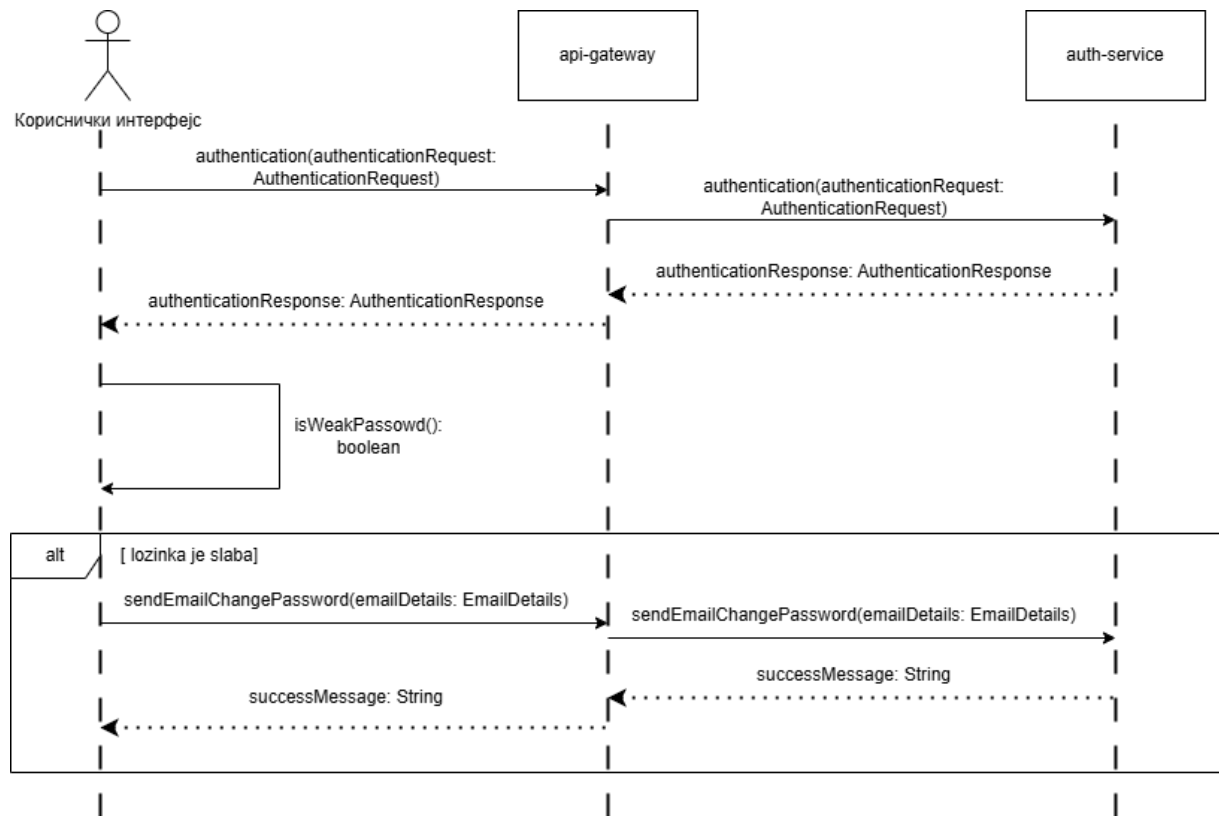
Разлагање „*Decorator Collaborator*“ патерном миграције

Претходно представљеним патернима миграције на конкретном примеру, описани су случајеви у којима се већ постојећа функционалност надограђује, извлачи и слично. Са патернима „*Strangler Fig Application*“ и „*Branch by Abstraction*“ тражена функционалност је мигрирана у нову целину микросервиса, док је постојећа имплементација функционалности постала неутребљива и обрисана, или надограђена како би користила услуге нове имплементације омогућене путем микросервиса. Самим тим за реализацију патерна миграције су биле неопходне одређене промене кода монолита. Међутим, у ситуацијама у којима је код монолита тражене функционалности немогуће променити, ова два патерна су непримењива.

За ситуације у којима се жели увести ново понашање или функционисање у систем, а да се при томе не врши измена постојећег кода монолита система, као решење се јавља „*Decorator Collaborator*“ патерн миграције. Овај патерн миграције омогућава надоградњу преко постојеће функционалности, без било каквог утицаја и интервенције над постојећим кодом монолита. На тај начин, нова функционалност се додаје као надоградња, док оригинални систем остаје непромењен и без знања о модификацији нове функционалности (Њуман, 2019, стр. 118).

Пример узет за реализацију „*Decorator Collaborator*“ патерна миграције јесте надоградња слања мејлова упозорења приликом пријављивања студената на софтверски систем. У претходном вербалном опису монолитне апликације описано је да студент може бити креиран путем личне регистрације или креирањем налога од стране професора. Уколико је налог креиран од стране професора, лозинка генерисана за тај налог је слаба и представља број индекса студента. Студент, који се пријави на софтверски систем путем налога који му је професор креирао, добија мејл упозорења у којима се наглашава важност промене његове слабе лозинке на другу, јачу, и упућује га ка даљим корацима реализације промене лозинке. Овим примером променама су погођена два микросервиса: микросервис аутентификације („*auth-service*“) и микросервис преусмеравања захтева („*api-gateway*“).

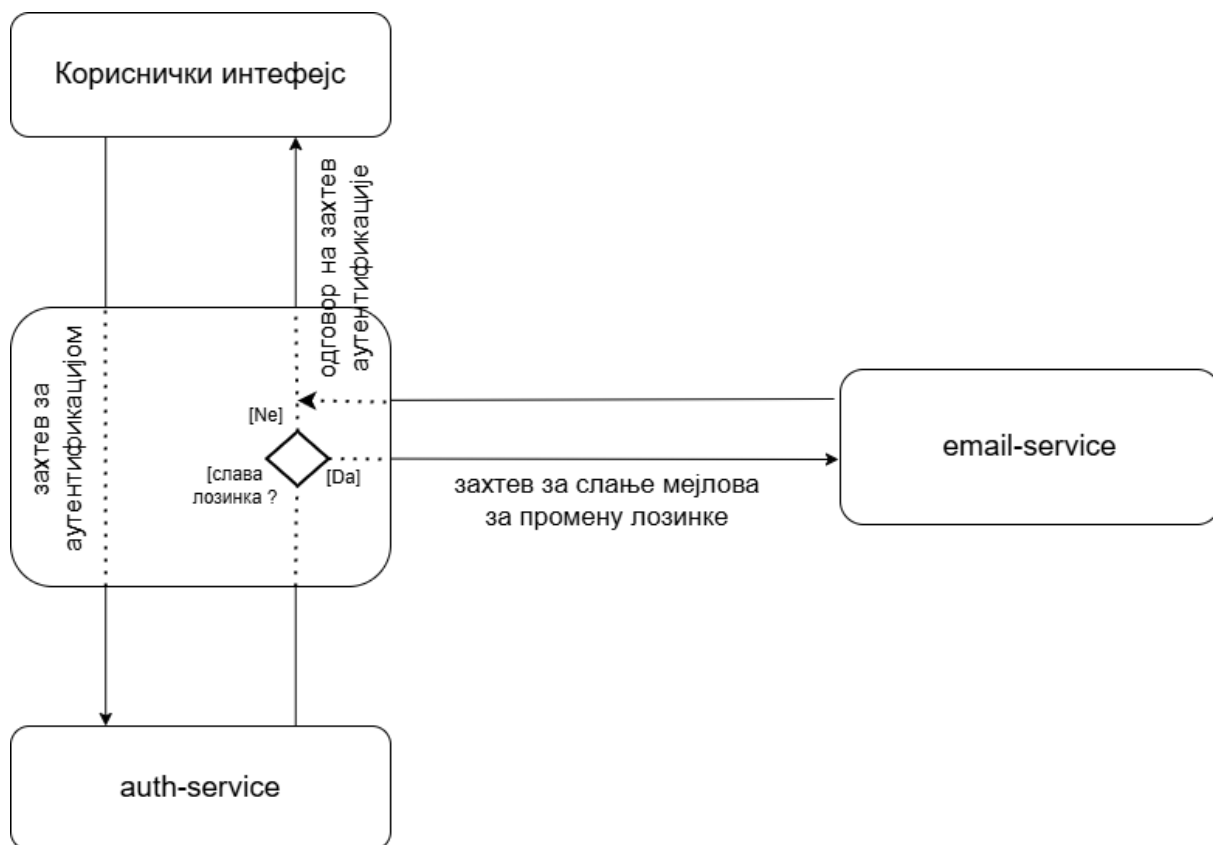
Циљ примене патерна миграције на функционалност аутентификације јесте издвојити функционалности слања мејлова за промену слабе лозинке, тиме што се таква функционалност неће позивати из микросериса аутентификације, нити са корисничког интерфјеса, већ преко „*api-gateway*“ микросервиса. Пре примене патерна миграције, софтверски систем је функционисао тако што се изврши функционалност аутентификације, корисничком интерфејсу врате неопходни подаци аутентификације, након чега кориснички интерфејс поново позива микросервис аутентификације, односно функционалност слања мејлова за промену слабе лозинке. На следећој слици је приказан дијаграм секвенце који описује ток свих захтева и одговора током извршења аутентификације корисника:



Слика 46. Дијаграм секвенце аутентификације корисника на систем

Са реализацијом патерна миграције, функционалност слања мејлова се пребацује у потпуности на микросервис за слање мејлова. Међутим, обзиром да је циљ „*Decorator Collaborator*“ патерна надоградња функционалности без утицаја на постојећу и њену изолацију од такве надоградње, микросервис аутентификације неће комуницирати са микросервисом за слање мејлова. Начин како се приступа функционалности слања мејла за промену слабе лозинке јесте путем пресретања одговора на захтев кроз „*api-gateway*“. Наиме, уместо да пропусти одговор директно корисничком интерфејсу, „*api-gateway*“ паузира слање одговора на захтев за аутентификацијом, обрађује податке добијене аутентификацијом и на основу њих шаље захтев микросервису за слање мејлова уколико закључи да је лозинка студента слаба. Захтев који шаље јесте за слање мејлова за промену лозинке где, након успешног извршавања слања мејла, прослеђује паузиран одговор захтева корисничком интерфејсу. На овај начин кориснички интерфејс се ослобађа терета поновног слања захтева ка микросервису аутентификације за слање мејлова, док се микросервис аутентификације специфицира да извршава услуге за које је првобитно и направљен – услуге аутентификације и регистрације. Пресретањем захтева не утиче се на конкретне функционалности које се позивају и чијег тока је део, већ се олакшава реализација услуге аутентификације тиме што се смањује број позива ка микросервисима и кроз један захтев реализују све неопходне активности.

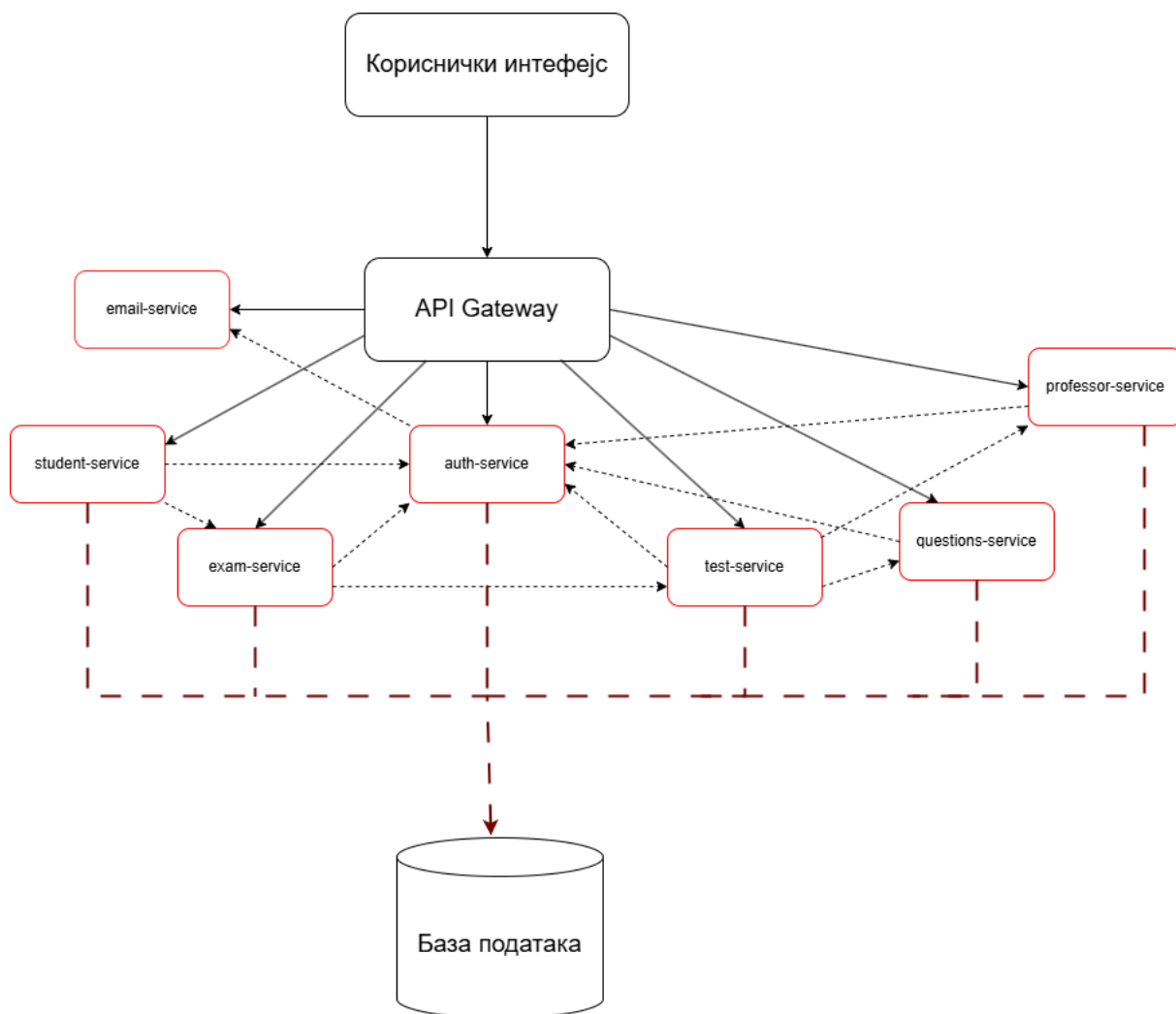
Међутим, пре него што се крене са пресретањем захтева неопходно је функционалности имати на микросервису за слање мејлова. Након премештања функционалности, на следећој слици се може видети процес пресретања одговора и слање захтева са „*api-gateway-a*“ на нову функционалност микросервиса за слање мејлова:



Слика 47. Конкретан пример „Decorator Collaborator“ патерна миграције

Оваквим патерном се функционалност надограђује успешно и без икаквих утицаја на рад осталих микросервиса. Једина промена која се захтева јесте у оквиру самог посредника, „*api-gateway-a*“, који сам по себи не представља микросервис, већ проху заслужен за преусмеравање захтева и враћање одговора корисницима. Самим тим, нова имплементација преусмеравања ка микросервису за слање мејлова спада у оквиру његовог домена одговорности према целокупном софтверском систему.

Резултат „Decorator Collaborator“ патерна миграције, на целокупну архитектуру програмског система са микросервисима, јесте нова веза између „*api-gateway-a*“ и микросервиса за слање мејлова. На следећој слици је приказана микросервисна архитектура софтверског система након реализације овог патерна миграције:

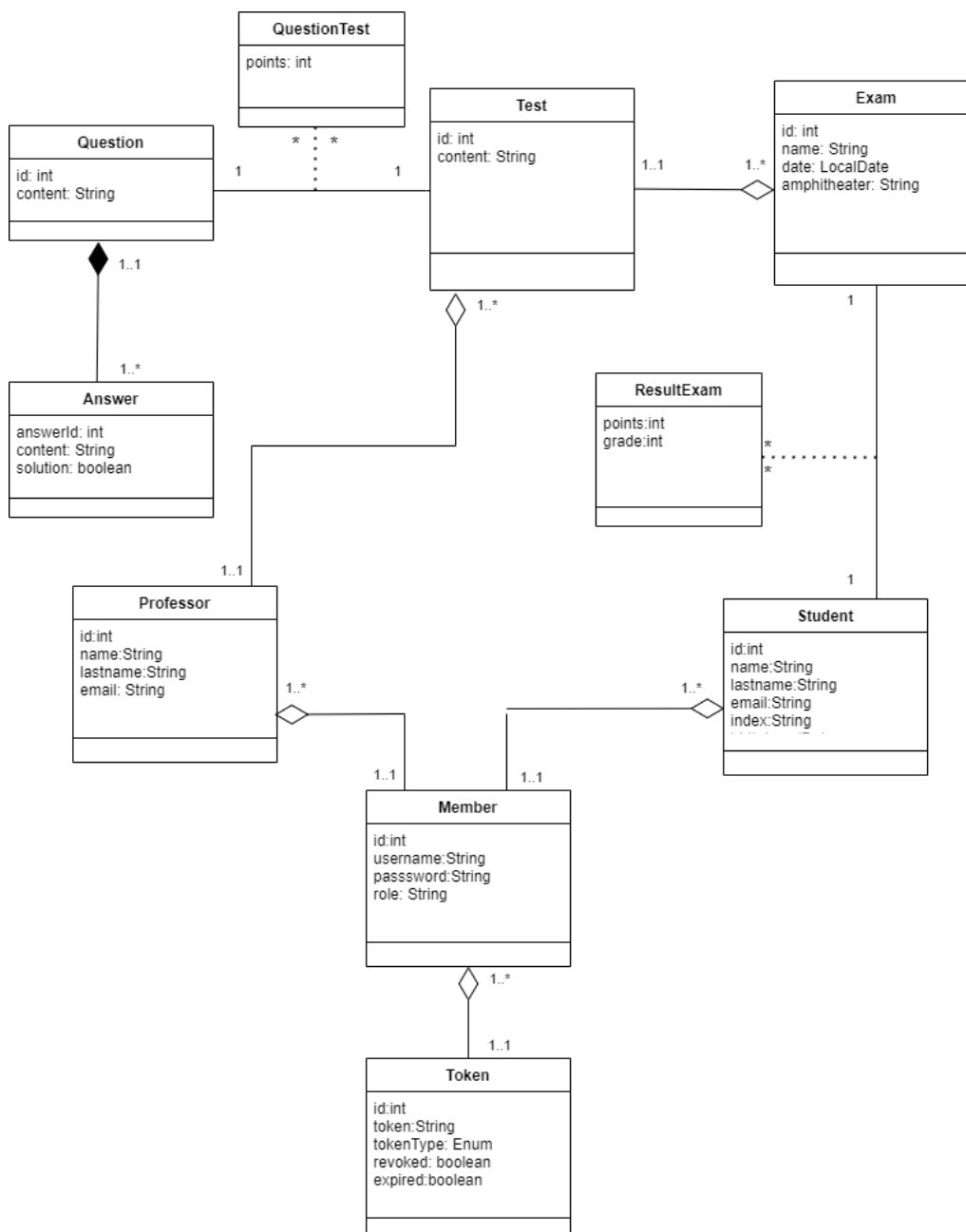


Слика 48. Приказ микросервисне архитектуре након реализације „*Decorator Collaborator*” патерна миграције

Са претходне архитектуре се уочава коришћење једне базе за потребе свих микросервиса. Обзиром да је једна од особина микросервиса постојање засебног домена података са независном и изолованом базом података, у наставку мастер рада је приказан поступак разлагања базе података монолита у базе података сваког микросервиса.

Разлагање монолитне базе података

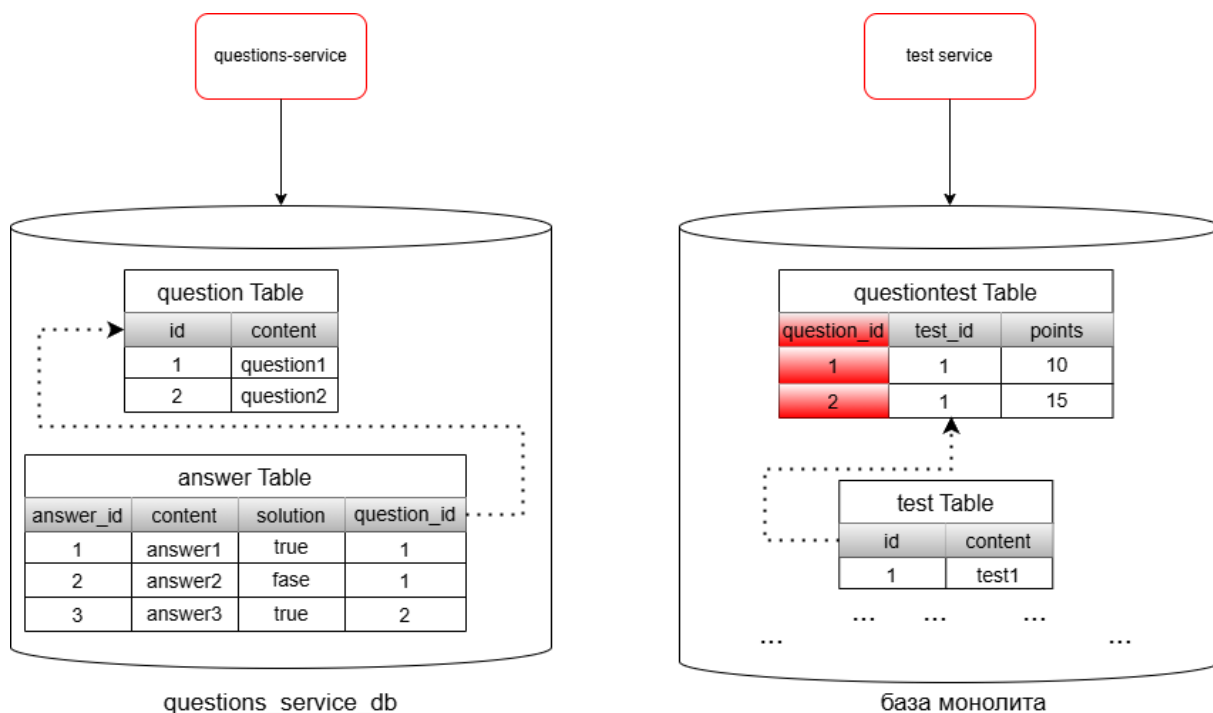
У оквиру мастер рада објашњен је значај поседовања засебног домена и базе података сваког микросервиса. Обзриом да претходна архитектура микросервиса се везује за коришћење једне, монолитне, базе података, у наставку мастер рада приказане су миграције и разлагање базе података на поједине домене. Међутим, пре почетка разлагања неопходно је имати увид у релациону структуру монолитне базе података. На следећој слици дат је концептуални дијаграм доменских класа које образују релацион шему монолитне базе података:



Слика 49. Концептуални дијаграм доменских класа монолитног софтверског система

Проблем који настаје на почетку разлагања монолитне базе података је идентичан проблему описаним приликом реализације „*Strangler Fig Application*” патерна, а то је одабир првог домена података за разлагање. Претходно је споменуто да најбољи први кандидат за издвајање функционалности „*Strang Fig Application*” патерном је онај који је најизолованији од остатка домена функционалности. Слично је и са разлагањем базе података. Уместо разматрања услуга и веза које се остварују са осталим функционалностима, разматрају се референцијални интегритети који се могу нарушити. Током имплементације примера мастер рада креће се од домена података, табеле базе података, чијим извлачењем се у што мањој мери утиче на референцијални интегритет и конзистентност базе података. Нарушавање референцијалног интегритета и конзистентности базе података је неизбежно приликом разлагања базе података, тј. током читаве миграције у микросервисну архитектуру. Због тога, мастер радом се и покривају различити принципи разлагања базе података који имају за циљ очување конзистентности и решавање проблема референцијалног интегритета. Међутим, одабиром најизолованијег домена података за првог кандидата разлагања базе података доприноси инкременталном разлагању базе података и лакоћу приликом реализације.

Са тим у вези, за први домен разлагања базе података одабран је домен података питања и одговора. Разлог због које је одабран јесте управо због његове изолованости и везом са остатком система кроз домен података тест функционаности. База података питања у себи садржи две табеле, табелу питања и одговора. Са концептуалног дијаграма доменских класа се примећује и асоцијативна класа, односно табела асоцијације између класе питања и тестова. Задатак асоцијативне класе јесте да омогући повезивање једног објекта питања са више различитих објеката тестова и обрнуто. Међутим проблем који настаје приликом издвајања базе података питања и одговора јесте у референцијалном интегритету која асоцијативна табела има према табели питања и теста. Издвајање табеле питања и одговора је лако обзиром да су те две табеле међусобно везане и заједно иду у посебну базу података. Међутим, ситуација са асоцијативном табелом је та што подаци такве табеле обухватају два домена података. На следећој слици је приказана издвојена база података за микросервис питања као и веза са асоцијативном табелом у бази монолита између функционалности теста и питања:

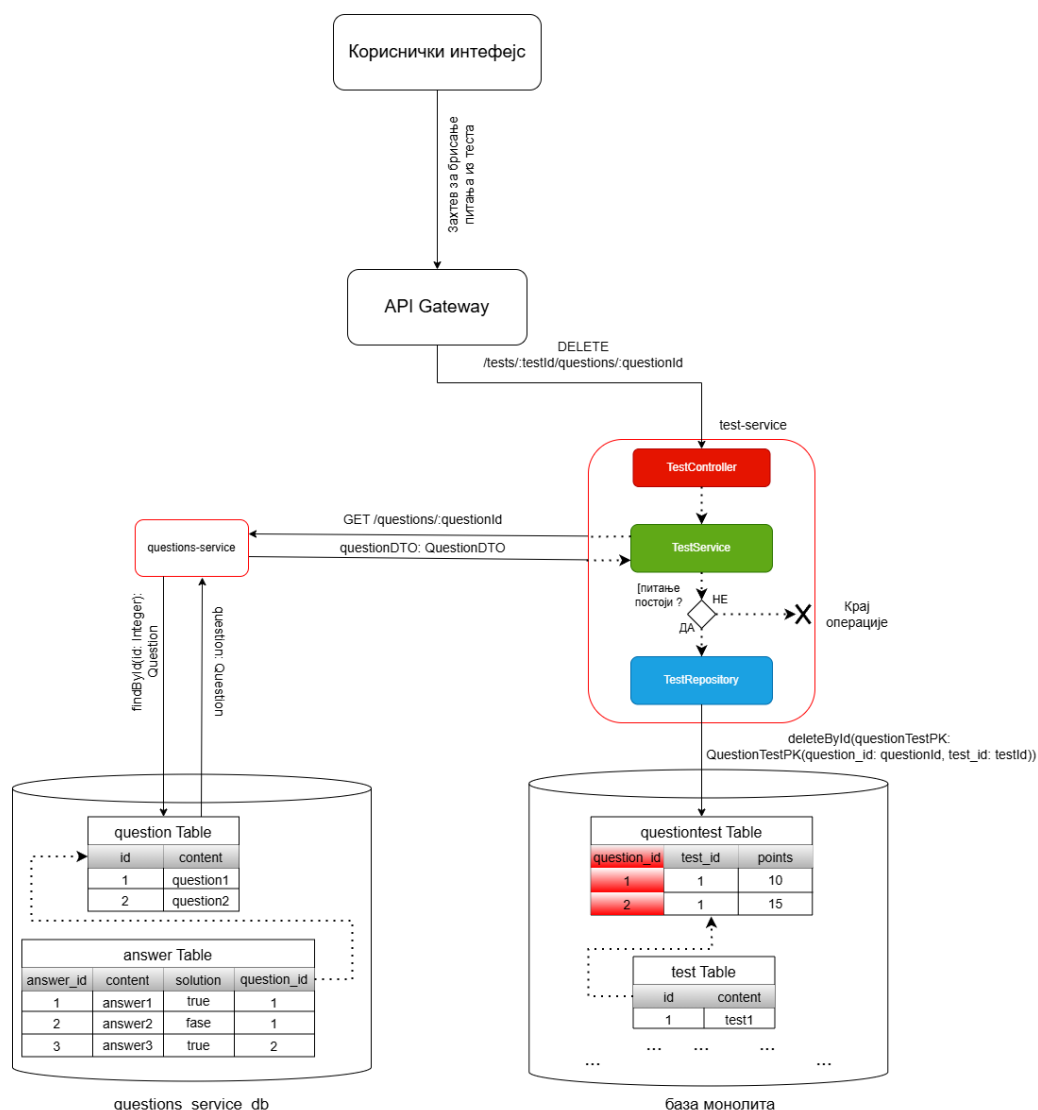


Слика 50. Раздвајање домена података микросервиса питања и проблем везе са асоцијативном табелом питања и тестова

Главни проблем разлагања базе података, са табелама између којих важе правила референцијалног интегритета, јесте конзистентност података база података. На примеру монолитне базе, брисањем рекора асоцијативне табеле питања и тестова би се прво поштовала правила и ограничења постављена између релација. Уколико је постављено ограничење „*Cascade*”³, онда би се брисањем рекорда питања брисали и рекорди асоцијативне табеле за коју је повезан и слично. Међутим, одвајањем табела у одвојене базе података таква ограничења више не постоје. На такав начин долази и до неконзистентности података између домена који комуницирају (Њуман, 2019, стр. 176). Претходно у мастер раду су представљена различита решења неконзистентности података, док се за конкретан пример користило решење провере података између различитих домена функционалности.

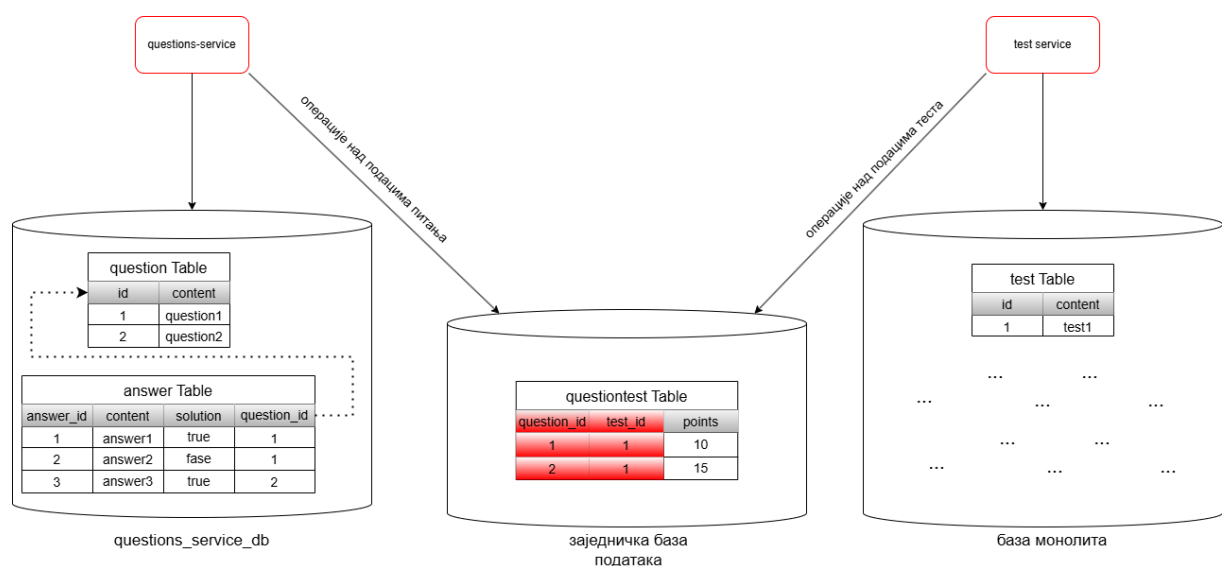
³ *Cascade* - представља једно од ограничења приликом рада са релационим базама података која дефинише акцију базе података на одређене промене попут брисања и ажурирања рекорда табеле које имају спољашње везе са другим табелама. Конкретно, *CASCADE* је везан за акције које се дешавају над родитељском табелом у односу на табелу где је постављен тако што приликом брисања родитељског рекорда, аутоматски брише и рекорд друге табеле који се за рекорд родитељске табеле надовезује [38].

Приликом руковања са операцијама уписивања, ажурирања или брисања података базе података, неопходно је проверити да ли је рекорд који се брише повезан са рекордима осталих база података (Њуман, 2019, стр. 176). На конкретном примеру питања и тестова неопходно је, приликом извршавања операција над асоцијативном табелом питања и одговора, проверити податке операције са доменом података питања и одговора. Односно, проверити приликом уписивања новог питања на тесту, брисања и ажурирања постојећег питања са теста, да ли такво питање постоји у оквиру издвојене базе података микросервиса питања и одговора. На следећој слици је приказан целокупан процес провере података између микросервиса теста и питања:



Слика 51. Приказ провере података на микросервису питања приликом избацивања питања са теста

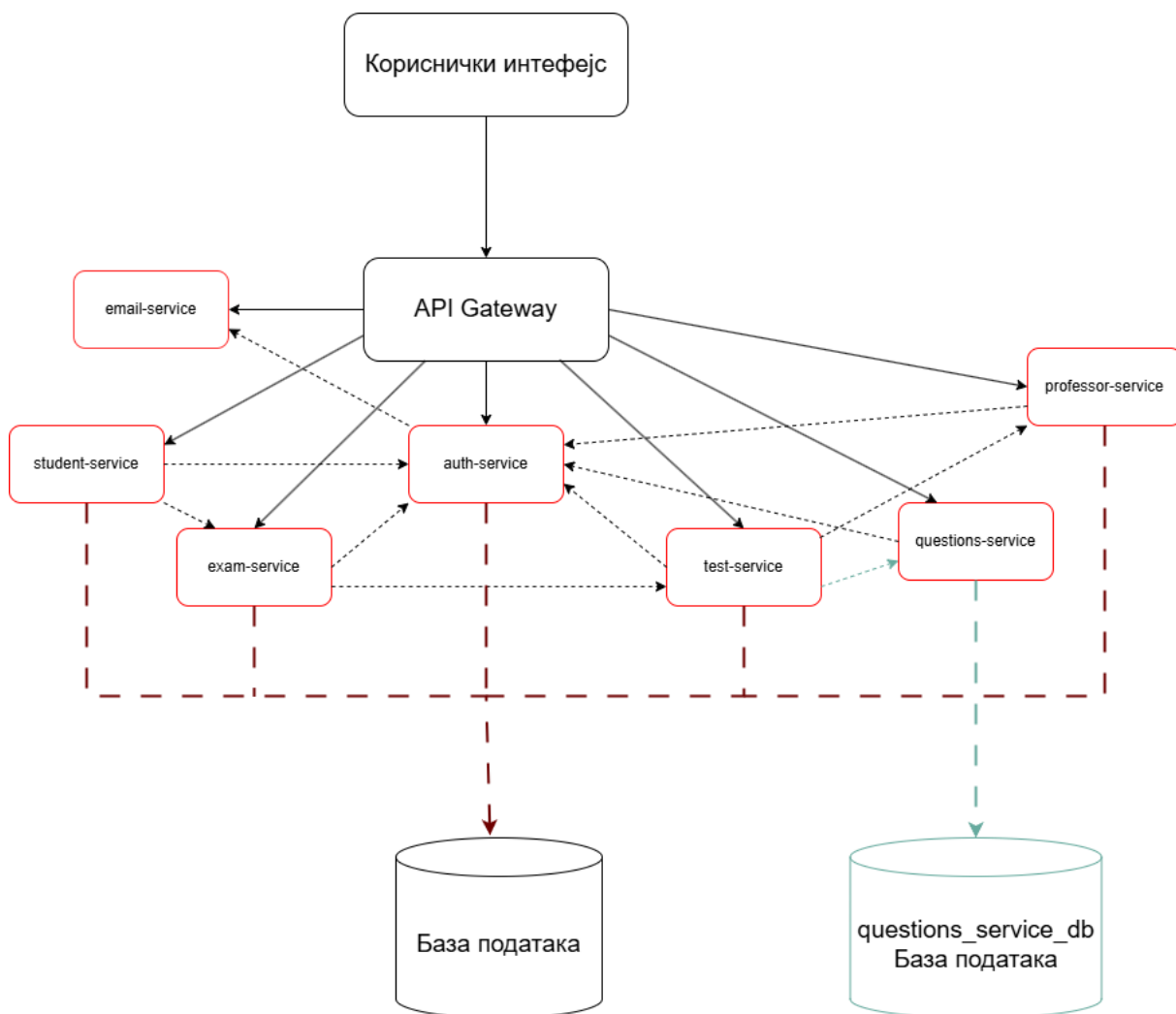
Претходни дијаграм функционише уколико се асоцијативна табела питања и тестова, и целокупне услуге и операције које долазе, приписују домену функционалности теста. На конкретном примеру мастер рада такво раздвајање табела је и урађено. Међутим, постоје и други начини како се ова табела може раздвојити. Један од начина јесте и коришћењем заједничке, изоловане, базе података којој ће приступати функционалности питања и тестова. На тај начин ова два микросервиса би имали приступ својој бази података, као и дељеној заједничкој бази података одговорној за чување веза између питања и одговора (асоцијативна табела). Међутим, уколико се одлучи о коришћењу засебне шеме базе података за чување података асоцијативне табеле, неопходно је разматрати и изазове који долазе за оваквим приступом. Један од изазова јесте учесталост промена података. Брисање, ажурирање и додавање података у базу података проузрокује додатне константне операције у оквиру микросервиса питања и теста. Са тим се број операција и дужина извршавања захтева повећава и може проузроковати значајна кашњења током извршавања наредби. Због тога, репрезентативан пример оваквог начина разбијања би били статични подаци, односно подаци који се не мењају често, који се углавном само читају из базе и користе за специфичне сврхе (Њуман, 2019, стр. 181). У случају коришћења изоловане шеме базе података, следећа слика представља пример како може да изгледа на конкретном примеру питања и тестова:



Слика 52. Заједничка база података за асоцијативну табелу питања и тестова

Такође, постојањем дељене базе података нарушава се особина изолованости микросервиса и постојања засебног домена података обзиром да податке деле са још једним доменом функционалности.

Међутим, за потребе конкретног примера мастер рада, асоцијативна табела питања и тестова придружена је домену функционалности теста. На основу вербалног описа закључује се да питања добијају смисао у оквиру софтверског система једино уколико су укључени у оквиру тестова. Укљученост питања у оквиру теста се чита из асоцијативне табеле питања и тестова и представља разлог зашто је прикључено овом домену функционалности. Резултат таквог разлагања базе података јесте микросервисна архитектура са једном издвојеном базом података, базом података питања и одговора што је и представљено на следећој слици:

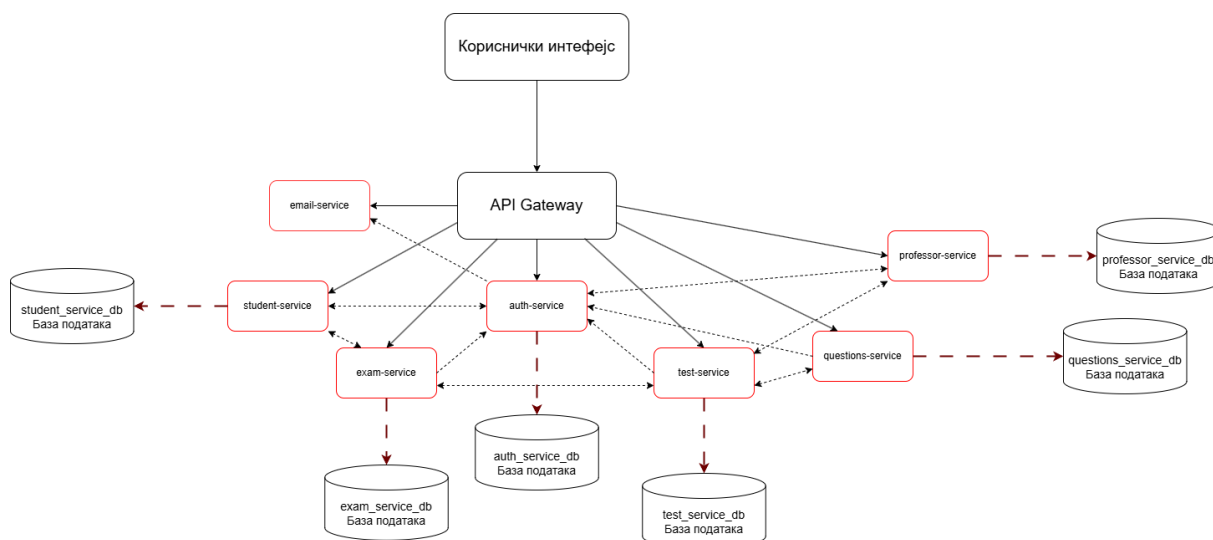


Слика 53. Резултат издвајања базе података питања и повезивања са микросервисом питања

Обзиром да је путања миграције монолита у микросервисну архитектуру таква да се прво издвајају функционалности разбијањем кода монолита, значајно је олакшано разбијање базе монолита у другом кораку обзиром да су све везе између функционалности већ остварене у коду у оквиру првог корака разбијања.

Претходна стратегија и патерн миграције разбијања базе монолита и издвајање посебних база резервисаних за потребе микросервиса се идентично примењују за све домене функционалности.

Резултат имплементације патерна разбијања монолитне базе података и издвајања базе података микросервиса, добија се коначна микросервисна архитектура добијена комбинацијом првог корака разбијања домена функционалности путем кода. Следећа слика илуструје коначну архитектуру добијену миграцијом монолита у програмски систем са микросервисном архитектуром.



Слика 54. Коначна микросервисна архитектура као резултат свих миграција монолита

Са слике се уочавају двосмерне комуникације неких микрсервиса које алудирају на важност међусобне провере података приликом рада са табелама које су имале одређени вид референцијалног интегритета са осталим доменима података.

Иако на први поглед делује као умршена мрежа различитих целина чије поједине операције зависе од операција и домена података других микросервиса, поента оваквих миграција није жеља за оптималним софтверским системом. Циљ мастер рада је показати патерне миграције и начине како се могу реализовати тако да се на олакшан начин изврши миграција монолита у програмски систем са микросервисном архитектуром. Са таквим патернима долазе и проблеми који су покривени конкретним примером што овај монолит чини још бољим избором, односно овакав пример монолита покрива различите проблеме на које се може наићи приликом трансформације монолитне апликације у микросервисну архитектуру. Коначна микросервисна архитектура је добијена реализацијом патерна миграције што је и циљ и идеја ове теме мастер рада, док се о питањем смисла реализације на конкретном примеру не бави обзиром да смисао сваке трансформације зависи од врсте и функционалности монолита, као и од спецификација и околности које долазе са тим.

5. Закључак

Мастер радом покривене су особине, принципи, правила, предности, као и недостаци који долазе са применом микросервисне архитектуре. Динамичко тржиште и стална промена потреба и захтева условљавају компаније да своје пословање прилагођавају и мењају како би остале у кораку са, како конкуренцијом, тако и са задовољством своје идентификоване групе корисника. Идеја мастер рада јесте пружити увид у проблеме и предности које долазе са променом пословања са монолитне структуре ка микросервисној, не само у правцу софтверске структуре, већ и организационе структуре које имплементирају. Услуге које пословања нуде се разликују, због чега правац писања мастер рада је усмерен ка пружању општих информација које се могу разматрати приликом доношења одлуке о трансформације структуре пословања. Међутим, са дефинисаним информацијама неопходно је пружити увид у конкретне примере на којима се такви извори информација могу и користити. Управо је то циљ имплементације конкретног примера у мастер раду, односно пружити увид у начине и конкретне кораке реализације идеје и циља саме трансформације монолита. Са друге стране, ефикасност реализације таквих циљева није представљена темом мастер рада. Разлог томе јесте до разноликости услуга које сваки монолит са којег се мигрира може да пружи. У почетку мастер рада представљени су различити фактори које је неопходно узети у обзир приликом упуштања у реализацију трансформације структуре. Међутим, индекс успешности реализације и перформансе нове микросервисне архитектуре нису представљени темом мастер рада.

Иако је садржај мастер рада техничке природе који се тичу различитих софтверских решења и проблема које са трансформацијом долазе, менаџмент страна мигрирања структуре пословања је такође од великог значаја за читав процес трансформације. Мастер радом су покривени проблеми, недостаци и потенцијална решења која долазе са комплетном миграцијом читаве структуре пословања. Међутим, конкретне акције, начини, методологије које се могу користити су остављене са стране обзиром да је циљ мастер рада био на техничкој страни реализације миграције монолитне апликације у систем са микросервисном архитектуром. Како би трансформација била успешна, обе стране морају функционисати и реаговати заједно, како би сам процес миграције био у што већој мери олакшан, бржи, прецизнији и ефективнији.

Софтверски инжењери морају иза себе имати прописан план, структуру и начин рада како би своје задатке реализовали на што бољи начин и са својим тимом пословање довели до значајног успеха. Упуштање у тако велики пројекат као што је миграција структура пословања захтева прецизно планирање и усаглашеност свих нивоа и надлежности организације. Од самих софтверских инжењера од којих се очекује испорука производа, па све до самог врха менаџмента који одлучује о путањи пројекта и његовом одобрењу, процес трансформације монолита у микросервисну архитектуру има далеко већи утицај и значење од самог програмирања и испоруке софтверског система.

Циљ мастер рада није представити микросервисну архитектуру као најбољу опцију архитектуре пословања. Врло је важно направити преглед и утврдити разлог и смисао ступања у трансформацију и имплементацију микросервисне архитектуре. У одређеним ситуацијама, имплементација микросервисне архитектуре може донети више недостатака него предности у односу на архитектуру монолита. Значајни утицај на донешење одлуке о примени микросервисне архитектуре имаће и надоградња мастер рада, која подразумева увођење параметара и формула. Такви параметри и формуле ће послужити као алати за процену одабира правца имплементације архитектуре софтверског система. Иако популарност микросервисне архитектуре расте и њена употреба је све већа, то не треба бити главни разлог њеног увођења. Наиме, свака одлука о трансформацији архитектуре, било ка микросервисном или неком другом архитектуралном стилу, мора бити пропратно испраћена и детаљно разматрана пре састављања даљих корака увођења специфичне архитектуре.

На основу претходних потенцијалних надоградњи мастер рада, идентификују се будућа истраживања која могу значајно унапредити циљ и садржину ове теме матер рада. Покривеношћу разлитих техника реализације миграције монолита пружа се увид у потенцијалне начине реализације и решавања проблема који са миграцијом долазе. Међутим, ефикасност и перформансе резултата реализације таквих техника мастер радом није покривено. Циљ будућих истраживања јесте утврдити различите индексе, параметре и чиниоце који могу бити репрезентативан пример успешности реализације и објективне процене резултата траснфорамције монолитне апликације у систем са микросервисном архитектуром.

Самим тим, дефинисањем различитих формула или патерна, уз помоћу дефинисаних параметара, моћи ће се на лак начин доћи до увида у успешност трансформације. Сама процена успешности спада, како и на тим испоруке производа, тако и на менаџмент који такође има важну улогу у процесу трансформације. Овим истраживањем се доводи до следећег будућег истраживања који се тиче детаљнијег описа улоге менаџмента, организационе структуре на процес трансформације монолита у програмски систем са микросервисном архитектуром. Наиме, описивањем методологија и унутрашњих процеса које је неопходно спровести значајно би се процес читаве трансформације олакшао и учинио ефективнијим. Иако важни аспекти техничког дела трансформације су мастер радом покривени, неопспориво је важна и друга страна организационих процеса које омогућавају да технички аспект трансформације буде реализован.

Литература

- [1] Bass, L., Clements, P., & Kazman, R. (2013). Software Architecture in Practice: Third Edition. Addison-Wesley
- [2] Синиша Влајић: Софтверски патерни, Издавач Златни пресек, ISBN: 978-86-86887-30-6, Београд, 2014.
- [3] Gamma, E., Helm, R., Johnson, R., & Vlissides, J., (1995). Elements of reusable object-oriented software. Design Patterns. What is Design Pattern? (Chapter 1, pp. 1-2). Addison-Wesley
- [4] Ali Gelenler. Medium. (2023). Architectural Styles vs Architectural Patterns. <https://medium.com/@ali.gelenler/architectural-styles-vs-architectural-patterns-7fab51713470>. (приступано 26.06.2024)
- [5] Richards, M. (2015). Software architecture patterns: *Understanding Common Architectural Styles and When to Use Them*: Second Edition. 1005 Gravenstein Highway North, Sebastopol, CA 95472: O'Reilly Media, Incorporated.
- [6] др Милан Милановић. Medium. (2023). Most Common Software Architecture Styles. <https://medium.com/@techworldwithmilan/most-common-software-architecture-styles-86881d779683>. (приступано 27.06.2024)
- [7] Милан Јовановић. MJTech. (2024). What Is a Modular Monolith? <https://www.milanjovanovic.tech/blog/what-is-a-modular-monolith>. (приступано 30.06.2024)
- [8] Mehmet Ozkaya. Medium. (2023). Microservices Killer: Modular Monolithic Architecture. <https://medium.com/design-microservices-architecture-with-patterns/microservices-killer-modular-monolithic-architecture-ac83814f6862>. (приступано 30.06.2024)
- [9] Paul Kirvan. Tech Target. (2023). Loose Coupling. <https://www.techtarget.com/searchnetworking/definition/loose-coupling>. (приступано 30.06.2024)
- [10] Andre Newman. Gremlin. (2020). Is your microservice a distributed monolith? <https://www.gremlin.com/blog/is-your-microservice-a-distributed-monolith>. (приступано 30.06.2024)
- [11] Naresh Waswani. Medium. (2024). Microservices Architecture: The Hard Parts: Trap of Distributed Monolith. <https://medium.com/simpplr-technology/microservices-architecture-the-hard-parts-trap-of-distributed-monolith-7d707858aa32>. (приступано 30.06.2024)

30.06.2024)

- [12] AWS. What is SOA (Service-Oriented Architecture)? <https://aws.amazon.com/what-is/service-oriented-architecture/>. (приступано 21.7.2024)
- [13] IBM. What is an enterprise service bus (ESB)? <https://www.ibm.com/topics/esb>. (приступано 21.7.2024)
- [14] GeeksForGeeks. (2023). Service-Oriented Architecture. <https://www.geeksforgeeks.org/service-oriented-architecture/>. (приступано 22.7.2024)
- [15] Matthew Tyson. InfoWorld. (2020). What is service-oriented architecture? <https://www.infoworld.com/article/2158174/what-is-service-oriented-architecture.html>. (приступано 22.7.2024)
- [16] Barney, N., Nolle, T. TechTarget. (2023). Service-oriented architecture (SOA). <https://www.techtarget.com/searchapparchitecture/definition/service-oriented-architecture-SOA>. (приступано 22.7.2024)
- [17] TIBCO. What is event-driven architecture? <https://www.tibco.com/glossary/what-is-event-driven-architecture>. (приступано 22.7.2024)
- [18] Seetharamugn. Medium. (2023). The Complete Guide to Event-Driven Architecture. <https://medium.com/@seetharamugn/the-complete-guide-to-event-driven-architecture-b25226594227>. (приступано 22.7.2024)
- [19] Anton Selin. Medium. (2023). Understanding Space-Based Architecture. <https://medium.com/the-pragmatic-tech-review/understanding-space-based-architecture-15281953c24c>. (приступано 24.7.2024)
- [20] Margaret Rouse. Techopedia. (2012). Tuple Space. <https://www.techopedia.com/definition/28553/tuple-space>. (приступано 24.7.2024)
- [21] Mark Richards. O'Reilly. (2015). Software architecture patterns. <https://www.oreilly.com/content/software-architecture-patterns/>. (приступано 24.7.2024)
- [22] AWS. What are Microservices? <https://aws.amazon.com/microservices/>. (приступано 24.7.2024)
- [23] Newman, S. (2015). *Building Microservices, Designing Fine-Grained Systems*, 1st ed. United States of America: O'Reilly Media Inc
- [24] Newman, S. (2019). *Monolith to microservices: evolutionary patterns to transform your monolith*. O'Reilly Media.

- [25] Kalske, M., Mäkitalo, N., & Mikkonen, T. (2018). *Challenges when moving from monolith to microservice architecture*. In Current Trends in Web Engineering: ICWE 2017 International Workshops, Liquid Multi-Device Software and EnWoT, practi-O-web, NLPIT, SoWeMine, Rome, Italy, June 5-8, 2017, Revised Selected Papers 17 (pp. 32-47). Springer International Publishing.
- [26] Martin Fowler. martinFowler.com. (2020). Domain Driven Desing. <https://martinfowler.com/bliki/DomainDrivenDesign.html>. (приступано 29.7.2024)
- [27] Hashitha Subhashana. Medium. (2021). Circuit Breaker Pattern (Design Patterns for Microservices). <https://medium.com/geekculture/design-patterns-for-microservices-circuit-breaker-pattern-276249ffab33>. (приступано 29.7.2024)
- [28] Martin Fowler. martinFowler.com. (2004). Strangler Fig Application. <https://martinfowler.com/bliki/StranglerFigApplication.html>. (приступано 2.8.2024)
- [29] Microsoft. Strangler Fig pattern. <https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>. (приступано 2.8.2024)
- [30] Robert Witkowski. Medium. (2018). UI in Microservices World – Micro Frontends pattern and Web Components. <https://medium.com/@witek1902/ui-in-microservices-world-micro-frontends-pattern-and-web-components-23607a569363>. (приступано 2.8.2024)
- [31] Katle Lawson. bloomreach. (2024). What Are Single Page Applications and Why Do People Like Them So Much?. <https://www.bloomreach.com/en/blog/what-is-a-single-page-application>. (приступано 3.8.2024)
- [32] Cam Jackson. martinFowler.com. (2019). Micro Frontends. <https://martinfowler.com/articles/micro-frontends.html>. (приступано 3.8.2024)
- [33] Martin Fowler. martinFowler.com. (2014). Branch by Abstraction. <https://martinfowler.com/bliki/BranchByAbstraction.html>. (приступано 3.8.2024)
- [34] Steve Smith. On Tech. (2013). Application pattern: Verify Branch by Abstraction. <https://www.stevesmith.tech/blog/application-pattern-verify-branch-by-abstraction/>. (приступано 3.8.2024)
- [35] Priyal Walpita. Medium. (2021). Monolith to Microservices – Migration patterns. <https://priyalwalpita.medium.com/monolith-to-microservices-migration-patters-a9e32d5adc9a>. (приступано 3.8.2024)

- [36] Abhishek Kapoor. Medium. (2021). Patterns to know before migrating your monolith to microservices. <https://levelup.gitconnected.com/patterns-to-know-before-migrating-your-monolith-to-microservices-72fcbcc7846e>. (приступано 4.8.2024)
- [37] Fowler, M. (2012). *Patterns of enterprise application architecture*. (Chapter 15, pp. 401-411). Addison-Wesley.
- [38] GeeksForGeeks. (2024). Cascade in SQL. <https://www.geeksforgeeks.org/cascade-in-sql/>. (приступано 29.12.2024)